中图分类号：TP311

论 文 编 号：10006ZY2206319

北京航空航天大學

# 硕 士 学 位 论 文

# 基于预训练模型的即时缺陷预测技术研究

作 者 姓 名　郭钰香

学 科 专 业　计算机技术

指 导 教 师　高小鹏　教授

培 养 学 院　计算机学院

# Research on Just-in-Time Defect Prediction Based on Pre-trained Models

A Dissertation Submitted for the Degree of Master

**Candidate: Yuxiang Guo**

**Supervisor: Prof. Xiaopeng Gao**

School of Computer Science & Engineering

Beihang University, Beijing, China

**中图分类号：TP311**

**论文编号：10006ZY2206319**

# 硕　士　学　位　论　文

# 基于预训练模型的即时缺陷预测技术研究

| | | | |
|---|---|---|---|
| 作者姓名 | 郭钰香 | 申请学位级别 | 工学硕士 |
| 指导教师姓名 | 高小鹏 | 职　　称 | 教授 |
| 学科专业 | 计算机技术 | 研究方向 | 智能化软件工程 |
| 学习时间自 | 2022 年 9 月 1 日 起 | 至 | 2024 年 12 月 9 日止 |
| 论文提交日期 | 2024 年 11 月 29 日 | 论文答辩日期 | 2024 年 12 月 9 日 |
| 学位授予单位 | 北京航空航天大学 | 学位授予日期 | 年　月　日 |

# 关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：_____ 日期：2024 年 12 月 13 日

# 学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：_____ 日期：2024 年 12 月 13 日

指导教师签名：_____ 日期：2024 年 12 月 13 日

# 摘 要

现代软件开发周期倾向于在短期内发布软件产品，应对这样的短期软件开发周期对现代软件质量保证（SQA）实践构成重大挑战，即时（Just-in-time）缺陷预测的提出主要是为了预测项目开发周期中一个新的提交在未来是否会引入缺陷。这种提交级别的预测对于帮助从业者在软件开发过程中将有限的 SQA 资源优先放在风险最大的提交上非常有用。

近年来随着深度学习的发展，越来越多的工作将端到端的深度学习模型应用于即时缺陷预测任务，改写了传统的基于人工特征和机器学习的分类方法。而预训练模型的出现使得相关任务在性能表现上进一步提高。

然而大多数研究者仅对预训练模型做简单的微调应用，对于即时缺陷预测任务来说，与预训练模型的微调相关的影响因素仍然未知。因此，系统地探讨不同因素如何影响基于预训练模型的 JIT 缺陷预测任务的性能至关重要。此外，即时缺陷预测任务中存在可用缺陷数据不足的问题，即可以通过预训练的方式学习到的信息是有限的。检索增强技术作为一种提高生成性任务表现的良好方案，已经在代码相关任务如注释补全和代码补全中获得显著的性能提升。对于缺陷预测这类分类理解任务来说，如何利用检索增强的范式提升任务性能仍未得到充分探索。

针对以上问题，本文做了如下工作：

1. 本文提出一个通用的基于预训练模型的即时缺陷预测架构，并基于不同预训练模型构建缺陷预测模型，实证性地探究不同 Transformer 结构预训练模型和不同输入在即时缺陷预测任务上的敏感性。通过消融实验揭示了 commit code 在缺陷预测中的关键作用，建议在模型优化中重点关注提交代码处理。此外，本文发现平衡数据集在 few-shot 场景下能提升模型性能。
2. 本文基于提出的通用架构，分别在基于 encoder 和 decoder 架构的两个经典预训练模型中系统性研究调优策略的影响，实验结果表明第一编码器/解码器层在模型微调中至关重要，冻结该层参数会显著降低性能，为模型调优策略提供了重要参考。本研究结合 LoRA 提出了一种高效微调方案，优化训练资源与性能的平衡，保留第一层编参数的可更新性，配合权重衰减策略，实现计算效率与性能的提升，为 JIT 缺陷预测模型的优化提供了有效手段。

3. 本文基于提出的架构和实证的发现，引入基于 commit code 的检索增强方法，在即时缺陷预测任务中设计并实现了两种检索增强架构（信息堆叠与跨模型向量融合），实验结果显示这两种方法均能显著提升缺陷预测任务的表现，证明了合理的检索数量增强能够有效提高模型性能。
4. 本文提出的跨模型向量融合架构通过将召回文本转化为召回向量，有效解决了信息堆叠架构中显存限制和信息丢失的问题。为了进一步验证基于跨模型向量融合方式的检索增强方法泛化性，本文在另外两组模型上进行了更广泛的实验验证，即与 CodeGPT 同架构，但参数量为 355M 的 GPT2-medium，以及当前另一 decoder 架构参数量为 1.23B 的 Llama-3.2-1B 模型。

总体来说，本文面向基于预训练模型的即时缺陷预测任务，提出了通用的模型架构并对任务影响因素进行详尽的实证分析，基于实证发现设计并实现两种检索增强架构（信息堆叠与跨模型向量融合）以提升 JIT 缺陷预测任务的性能。本研究不仅提供了深入的模型分析和评估，还通过实验验证了检索增强在缺陷预测任务中的实际效果。并给出了在基于检索增强缺陷预测的结构中召回数量的分析和建议。

**关键词：**即时缺陷预测，预训练模型，检索增强

# Abstract

The modern software development cycle tends to prioritize rapid software product releases, posing significant challenges to contemporary Software Quality Assurance (SQA) practices. Just-in-time (JIT) defect prediction has emerged to forecast whether a new commit during a project development cycle will introduce defects. Such commit-level predictions are invaluable for practitioners, enabling them to allocate limited SQA resources to the most risky submissions throughout the software development process.

In recent years, the advancement of deep learning has led to an increasing number of end-to-end deep learning models being applied to JIT defect prediction tasks, transforming traditional methods based on manual features and machine learning. The introduction of pre-trained models has further improved performance on related tasks.

However, most researchers have merely conducted simple fine-tuning of pre-trained models, leaving the influencing factors related to fine-tuning in the context of JIT defect prediction largely unexplored. Therefore, it is crucial to systematically investigate how various factors impact the performance of JIT defect prediction tasks based on pre-trained models. Additionally, the challenge of insufficient available defect data limits the information that can be learned through pre-training. Retrieval-augmented techniques have proven effective in enhancing generative task performance and have shown significant improvements in code-related tasks such as comment completion and code completion. However, the exploration of how to leverage retrieval augmentation paradigms to enhance performance in classification tasks like defect prediction remains inadequate.

The main contribution of this work is as follows:

1. This paper propose a general architecture for JIT defect prediction, constructing different defect prediction models using various pre-trained models and empirically investigating the sensitivity of various Transformer architectures and input parts on JIT defect prediction tasks. Ablation studies reveal the critical role of commit code in defect prediction. Additionally, we find that balancing datasets can enhance model performance in few-shot scenarios.

2. Based on the proposed architecture, we systematically study the impact of tuning strategies in two typical pre-trained models CodeGPT and CodeBERT. Experimental results provide important insights for model tuning strategies. We propose an efficient fine-tuning scheme combining LoRA to optimize the balance between training resources and performance.
3. Leveraging the proposed architecture and empirical findings, we introduce a retrieval-augmented method based on commit code and design two retrieval-augmented architectures (information stacking and cross-model vector fusion) for the JIT defect prediction task.
4. The cross-model vector fusion architecture proposed in this study transforms recalled sequences into recalled vectors, effectively addressing the issues of memory limitations and information loss encountered in the information stacking architecture. To further validate the generalizability of the retrieval-augmented approach, we conducted extensive experimental verification on two additional model groups: GPT2-medium and and the Llama3.2-1B model.

In summary, this paper establishes a general model architecture for JIT defect prediction based on pre-trained models and conducts a thorough empirical analysis of task-influencing factors. We design and implement two retrieval-augmented architectures to enhance performance in JIT defect prediction task. This research not only provides in-depth model analysis and evaluation but also experimentally validates the practical effectiveness of retrieval augmentation in defect prediction tasks, offering analyses and recommendations regarding the number of recalls in retrieval-augmented defect prediction structures.

**Key words**: Just-In-time defect prediction, pre-trained models, retrieval augment

# 目 录

第一章 绪论 ..... 1
1.1 研究背景和意义 ..... 1
1.2 国内外研究现状 ..... 5
1.2.1 即时缺陷预测技术及相关研究 ..... 5
1.2.2 预训练模型调优研究 ..... 6
1.2.3 基于检索增强的技术研究 ..... 8
1.3 研究目标及内容 ..... 9
1.3.1 预训练模型结构对缺陷预测任务的实证研究 ..... 9
1.3.2 预训练微调策略对缺陷预测任务的实证研究 ..... 10
1.3.3 基于检索增强的即时缺陷预测技术优化 ..... 11
1.4 本文组织结构 ..... 12
第二章 相关技术基础介绍 ..... 14
2.1 即时缺陷预测 ..... 14
2.1.1 传统即时缺陷预测技术 ..... 14
2.1.2 基于预训练微调的即时缺陷预测技术 ..... 16
2.2 检索增强 ..... 17
2.2.1 向量相似性计算 ..... 17
2.2.2 基于预训练模型的向量相似性 ..... 18
2.2.3 检索增强技术应用案例 ..... 19
2.3 本章小节 ..... 21
第三章 预训练模型结构对缺陷预测任务影响的实证研究 ..... 22
3.1 实证研究目的与内容 ..... 22
3.2 基于预训练模型通用框架实现 ..... 23
3.3 实验设置与数据 ..... 24
3.4 实证研究实验与分析 ..... 25
3.4.1 基于六种不同预训练模型结构的 JIT 缺陷预测模型的性能探究 ..... 25

3.4.2 不同模型的训练效果与预测结果分布之间的关系探究 ..... 28
3.4.3 不同模型对输入数据的敏感性探究 ..... 31
3.4.4 不同模型在 few-shot 场景下的具体表现 ..... 33
3.5 本章小节 ..... 35
第四章 预训练微调策略对缺陷预测任务的实证研究 ..... 38
4.1 实证研究目的与内容 ..... 38
4.2 实证研究方法概述 ..... 39
4.3 实验设置与数据 ..... 39
4.4 实证研究实验结果与分析 ..... 40
4.4.1 参数冻结和初始化对模型性能影响探究 ..... 40
4.4.2 模型对不同特征提取器的敏感性探究 ..... 44
4.4.3 优化器策略对模型的具体影响探究 ..... 45
4.4.4 综合微调策略的经济有效训练设置探究 ..... 47
4.5 本章小节 ..... 48
第五章 基于检索增强的即时缺陷预测技术优化 ..... 49
5.1 模型结构 ..... 49
5.1.1 无检索模型的构建以及向量表示的获取 ..... 49
5.1.2 两种基于检索增强的模型架构 ..... 51
5.2 实验的目的与内容 ..... 53
5.3 实验设置与数据 ..... 54
5.4 实验结果与分析 ..... 55
5.4.1 基于信息堆叠的检索增强模型（RIS）具体表现 ..... 55
5.4.2 基于跨模型向量融合的检索增强模型（RVF）具体表现 ..... 58
5.4.3 基于跨模型向量融合的检索增强模型泛化性探究 ..... 61
5.4.4 相较于基线模型的总体性能提升 ..... 67
5.5 本章小节 ..... 69
总结与展望 ..... 71

参考文献 ..... 73

攻读硕士期间取得的成果 ..... 80

致谢 ..... 81

# 图 清单

图 1　在 QT 项目中一次有 bug 提交修改的文件示例 ..... 1
图 2　QT 项目中一次有 bug 提交的 commit message 示例 ..... 2
图 3　在 QT 项目中一次有 bug 的 commit code ..... 3
图 4　基于预训练模型的即时缺陷预测技术研究内容 ..... 9
图 5　基于不同预训练模型的即时缺陷预测模型框架 ..... 23
图 6　模型在两个数据集中的预测分布 ..... 27
图 7　模型在 qt 数据集中 accuracy 随训练时长变化 ..... 27
图 8　模型在 qt 数据集中 recall 随训练时长变化 ..... 28
图 9　模型在 openstack 数据集中 accuracy 随训练时长变化 ..... 28
图 10　模型在 openstack 数据集中 recall 随训练时长变化 ..... 29
图 11　模型在 qt 数据集中 loss 随训练步长变化情况 ..... 30
图 12　模型在 openstack 数据集中 loss 随训练步长变化情况 ..... 30
图 13　两个数据集上进行消融实验的 accuracy 以及 precision 结果 ..... 31
图 14　在两个数据集上进行消融实验的 recall 以及 f1 score 结果 ..... 32
图 15　不同 shot 场景下 qt 以及 openstack 数据集中的 auc 和 accuracy ..... 35
图 16　不同 shot 场景下 qt 以及 openstack 数据集中的 precision 和 recall ..... 36
图 17　JIT 模型框架及微调策略 ..... 39
图 18　六个数据集在 CodeGPTJIT 中冻结前 n 层解码器参数的 auc score 变化 ..... 41
图 19　CodeBERTJIT 中冻结前 n 层编码器参数的 auc score 变化 ..... 42
图 20　CodeGPTJIT 中初始化前 n 层编码器参数的 auc score 以及 f1 score ..... 43
图 21　CodeBERTJIT 中初始化前 n 层编码器参数的 auc score 以及 f1 score ..... 43
图 22　CodeLlama 结构 ..... 50
图 23　基于信息堆叠的检索增强模型架构 ..... 52
图 24　基于向量融合的检索增强模型架构 ..... 53
图 25　CodeGPT-base 模型平均性能随召回向量个数变化 ..... 58
图 26　GPT2-medium 模型平均性能随召回向量个数变化 ..... 62
图 27　Llama3.2-1B 模型平均性能随召回向量个数变化 ..... 65

# 表 清单

表 1 数据集统计信息 ..... 25
表 2 训练超参设置 ..... 25
表 3 相比于基线模型在 qt 项目中表现 ..... 26
表 4 相比于基线模型在 openstack 项目中表现 ..... 26
表 5 模型的训练耗时 ..... 29
表 6 模型预测分布 T 检验结果 ..... 30
表 7 模型 2000-shot 场景下在 qt 数据集中的实验结果 ..... 33
表 8 模型 2000-shot 场景下在 openstack 数据集中的实验结果 ..... 34
表 9 CodeGPTJIT 在 qt 数据集 2000-shot 场景下超越其他两个基线模型 ..... 35
表 10 GPT2JIT 在 openstack 数据集 2000-shot 场景下超越其他两个基线模型 ... 36
表 11 6 个开源项目数据统计信息描述 ..... 40
表 12 训练超参设置 ..... 40
表 13 CodeGPTJIT 中冻结前 n 层 decoder block 参数的 f1 score ..... 42
表 14 CodeBERTJIT 中冻结前 n 层 encoder block 参数的 f1 score ..... 43
表 15 CodeGPTJIT 中使用不同特征提取器的 auc score 以及 f1 score ..... 44
表 16 CodeBERTIT 中使用不同特征提取器的 auc score 以及 f1 score ..... 45
表 17 CodeGPTJIT 中使用不同权重衰减率的 auc score 以及 f1 score ..... 46
表 18 CodeBERTJIT 中使用不同权重衰减率的 auc score 以及 f1 score ..... 46
表 19 OCJITLoRA 相比于其他基线模型在六个数据集上的 auc score ..... 47
表 20 OCJITLoRA 相比于其他基线模型在六个数据集上的 f1 score ..... 47
表 21 实验所用数据集统计信息 ..... 55
表 22 实验环境 ..... 55
表 23 原始无检索模型在六个项目中表现 ..... 56
表 24 引入召回代码输入的检索增强模型表现 ..... 57
表 25 扩大 commit code 文件数量的无检索增强模型表现 ..... 57
表 26 引入召回向量数量为 1 的检索增强模型表现 ..... 59

表 27　引入召回向量数量为 4 的检索增强模型表现 ..... 59
表 28　引入召回向量数量为 16 的检索增强模型表现 ..... 59
表 29　引入召回向量数量为 64 的检索增强模型表现 ..... 60
表 30　GPT2-medium 训练超参设置 ..... 61
表 31　GPT2-medium 引入召回向量数量为 0 的检索增强模型表现 ..... 62
表 32　GPT2-medium 引入召回向量数量为 1 的检索增强模型表现 ..... 63
表 33　GPT2-medium 引入召回向量数量为 4 的检索增强模型表现 ..... 63
表 34　GPT2-medium 引入召回向量数量为 16 的检索增强模型表现 ..... 64
表 35　GPT2-medium 引入召回向量数量为 64 的检索增强模型表现 ..... 64
表 36　LoRA 训练设置 ..... 64
表 37　Llama3.2-1B 引入召回向量数量为 0 的检索增强模型表现 ..... 66
表 38　Llama3.2-1B 引入召回向量数量为 1 的检索增强模型表现 ..... 66
表 39　Llama3.2-1B 引入召回向量数量为 4 的检索增强模型表现 ..... 66
表 40　Llama3.2-1B 引入召回向量数量为 16 的检索增强模型表现 ..... 67
表 41　Llama3.2-1B 引入召回向量数量为 64 的检索增强模型表现 ..... 67
表 42　所提优化方案与基线模型相比的 auc score ..... 68
表 43　所提优化方案与基线模型相比的 f1 score ..... 68
表 44　所提优化方案与基线模型相比的 MCC ..... 69

# 第一章　绪论

## 1.1 研究背景和意义

现代软件开发周期倾向于在短期内发布软件产品，为了应对这样的短期软件开发周期对现代软件质量保证（SQA）实践构成的重大挑战，持续的代码质量工具（例如，CI/CD，现代代码审查，静态分析）被用于早期检测软件缺陷。然而，由于有限的 SQA 资源，SQA 团队无法有效地检查每个提交，即时（Just-in-time）缺陷预测[1]的提出主要是为了预测一个提交在未来是否会引入缺陷。这种提交级别的预测对于帮助从业者在软件开发过程中将有限的 SQA 资源优先放在风险最大的提交上非常有用。

开发人员在对项目进行一次提交时，往往涉及修改多个文件，图 1 是 QT 项目一次提交中修改的文件示例（其中分别修改了 10 个文件，Patchset1 到 Patchset10，每个修改的文件有唯一标识的 id，例如 Patchset4 的 id 为 4016ee1b95e67f6ce126c97e2635947564ec0d0c）。

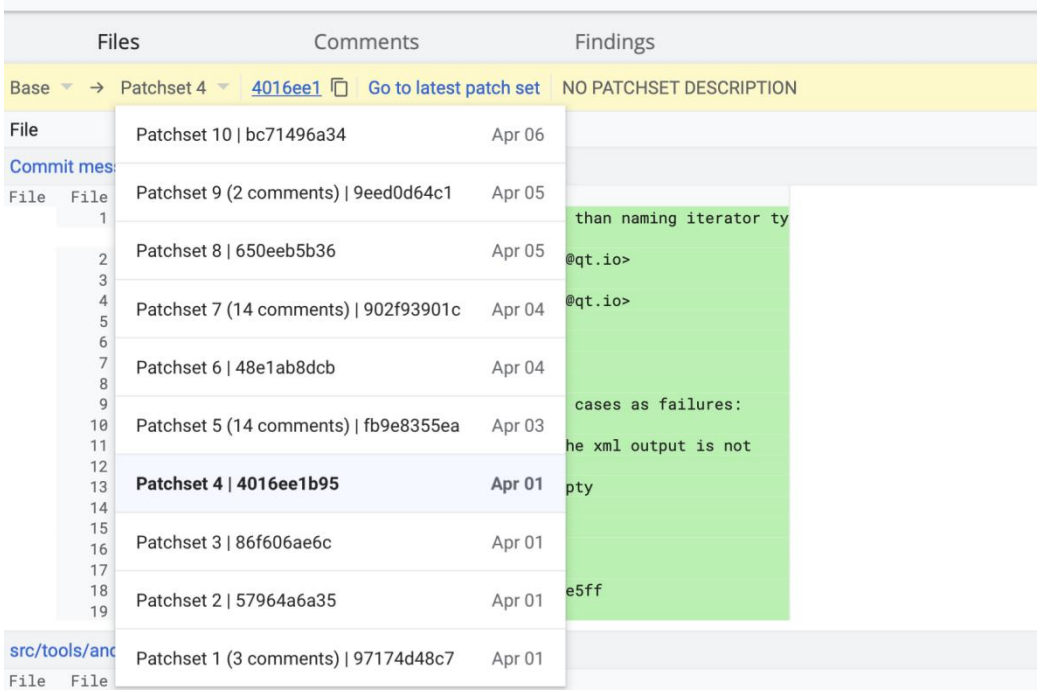


**图 1　在 QT 项目中一次有 bug 提交修改的文件示例**

图 2 和图 3 是 QT 项目中开发人员一次提交时对一个文件修改的示例，包括一次代码及包含的提交信息示例，图 2 是本次提交中的提交信息（Commit message），在提交的信息部分包含了提交本次代码的开发人员信息（第 4 行），提交时间（第 5 行）

以及任务序号（第 17 行）和变更 ID（第 18 行）。在提交信息部分中比较重要的部分是开发人员的注解（第 7-13 行），这部分由自然语言文本构成，很大程度可以体现出开发人员本次提交的意图，它也是基于深度学习方法德即时缺陷预测任务中常用到的有效特征。

图 3 是变更 ID 为 Ifbb3c31fa22159df0c39b264e95e2f1ccd16e5ff 的一次提交中变更的部分代码示例，其中绿色和红色部分标识了该项目下文件路径为 src/tools/androidtestrunner/main.cpp 的变更代码，其中绿色部分为增加的代码，红色部分为删除的代码。

Commit message

File File

```
1  Parent:     657cc31b (QSettings: use auto rather than naming iterator ty
   pes)
2  Author:     Assam Boudjelthia <assam.boudjelthia@qt.io>
3  AuthorDate: 2022-04-01 16:45:10 +0300
4  Commit:     Assam Boudjelthia <assam.boudjelthia@qt.io>
5  CommitDate: 2022-04-01 18:01:36 +0300
6
7  Android: fix androidtestrunner failure parsing
8
9  Make sure androidtestrunner report the following cases as failures:
10 * XPASS test case
11 * The test output doesn't have closing line or the xml output is not
12   complete and missing some closing blocks.
13 * The test result read from output.txt is not empty
14
15 Pick-to: 6.2 6.3
16 Fixes: QTBUG-100312
17 Task-number: QTBUG-100470
18 Change-Id: Ifbb3c31fa22159df0c39b264e95e2f1ccd16e5ff
19
```

**图 2　QT 项目中一次有 bug 提交的 commit message 示例**

已有研究者往往利用一次提交的传统特征和传统机器学习方法开展即时缺陷预测的研究。随着深度学习的发展[2-4]，研究者们开始将各种深度模型引入即时缺陷预测的任务中中[5-15]。开发人员的一次提交，既包含自然语言文本的提交注解，也包括变更的代码源码。研究人员利用基于自然语言的 commit message 和 commit code 作为输入，提出了两个即时缺陷预测深度模型：DeepJIT[13]和 CC2Vec[15]，实验表明它们具有良好的预测性能，但是还在一些不足，如 CC2Vec 的非即时性、非完全上下文特性等，为了进一步探究即时缺陷预测技术的具体影响因素，Zeng 等人开展了大规模的实证研究，并贡献了近 10w 条数据的缺陷数据集，并提出一种依赖变更代码行数特征的分类器 LApredict[16]。

src/tools/androidtestrunner/main.cpp

```
464  495
465  496 static bool pullFiles()
466  497 {
467          bool ret = true;
     498     bool result = true;
468  499     for (auto it = g_options.outFiles.constBegin(); it != g_options.outFiles.end(); ++it) {
469  500         QByteArray output;
470  501         if (!execCommand(QStringLiteral("%1 shell run-as %2 cat files/output.%3")
471  502                          .arg(g_options.adbCommand, g_options.package, it.key()), &output)) {
472  503             // Cannot find output file. Check in path related to current user
473  504             QByteArray userId;
474  505             execCommand(QStringLiteral("%1 shell cmd activity get-current-user")
475  506                         .arg(g_options.adbCommand), &userId);
476  507             const QString userIdSimplified(QString::fromUtf8(userId).simplified());
477  508             if (!execCommand(QStringLiteral("%1 shell run-as %2 --user %3 cat files/output.%4")
478  509                          .arg(g_options.adbCommand, g_options.package, userIdSimplified, it.key()),
479  510                           &output)) {
480  511                 return false;
481  512             }
482  513         }
     514
     515         // If the output is empty then it either wasn't written
     516         // or we were not able to get it, so fail anyway.
     517         if (output.isEmpty())
     518             return false;
     519
483  520         auto checkerIt = g_options.checkFiles.find(it.key());
484          ret = ret && checkerIt != g_options.checkFiles.end() && checkerIt.value()(output);
     521     result &= (checkerIt != g_options.checkFiles.end()) && checkerIt.value()(output);
485  522         if (it.value() == QStringLiteral("-")){
486  523             fprintf(stdout, "%s", output.constData());
487  524             fflush(stdout);
488  525         } else {
489  526             QFile out{it.value()};
490  527             if (!out.open(QIODevice::WriteOnly))
491  528                 return false;
492  529             out.write(output);
493  530         }
494  531     }
495          return ret;
     532     return result;
```

**图 3　在 QT 项目中一次有 bug 的 commit code**

（附注 1：其中绿色部分为增加的代码，红色部分为删除的代码）

随着 Transformer 的出现，预训练模型大量应用于软件工程下游任务并获得了相较于以往的性能提升。预训练模型的出现引入了一种新的两步预训练和微调范式，以提升典型任务表现[17,18]。研究人员提出了不同架构的预训练模型，如基于 Transformer encoder 的预训练模型 RoBERTa[19]， CodeBERT[20]，基于 Transformer encoder 的模型 GPT2 [21]， CodeGPT，基于 Transformer encoder 和 decoder 的模型如 BART [22]和 PLBART[23]。研究人员将这些预训练模型应用于编程语言任务，如文本分类、代码搜索和漏洞检测，并显示出在这些任务上的显著改进[24-27]。

BERT 风格模型是 Devlin 等人的工作[19]中提出的 BERT 模型的变体。一些研究者通过探索一些优化方法来对原来的 BERT 模型继续预训练。RoBERTa 在预训练 BERT 时通过探索不同训练策略改进预训练过程，CodeBERT 是 RoBERTa 的一个变体，它使用自然语言和编程语言两种模态的训练语料，可以适应于与代码相关的任务。这些

BERT 风格的模型为解决某个问题提供了新的基础。

最初，研究人员经常选择对新的下游任务进行全参数微调。BERT 微调过程的不稳定性自 BERT 被提出以来一直存在，研究人员提出各种方法解决这一问题[28-35]，但这些研究只关注常见的自然语言理解任务中 BERT 模型的微调影响。而在代码相关任务领域，以往的研究大多只关注预训练模型在下游任务中的应用，与预训练模型的微调相关的因素的影响仍然未知。因此，系统地探讨不同因素如何影响基于预训练模型的 JIT 缺陷预测任务的性能是至关重要的。

检索增强技术作为一种提高生成性任务表现的良好工具，往往被使用在开放域问答和机器阅读理解领域。在陈丹琦 17 年的工作中，将基于检索增强的此类任务系统分为两大块。对给定的问题，文档检索器从所有维基百科文章中检索，并将检索到的文章切分成段落，然后用一个称之为文档阅读器的模块在不同段落中预测答案位置并给出分数。这一工作为后续其他基于检索增强的研究提供了一个基本的框架和思路[36]。检索增强的研究工作往往基于向量相似性研究，基于此，一些研究人员对预训练模型如何获得更好的向量表示进行探究[37-40]。

预训练模型提升在自然语言领域任务表现的同时，也带来了解决程序语言相关任务的新范式。基于预训练模型，深度学习已广泛应用于软件漏洞检测、源代码总结，深度代码搜索，源代码补全等多种软件工程任务。由于复杂的数据依赖关系和控制依赖关系容易引发软件漏洞，Devign[41]将不同类型的程序结构信息与 Code Property Graph 结合到 Graph Neural Network 中进行漏洞检测。GraphCodeBERT[42]进一步结合数据流信息对程序中的变量关系进行编码进行预训练，并在四个下游任务上证明了其有效性。Liu 等人[43]将检索到的代码－摘要对结合起来生成高质量的摘要。尽管这些自定义网络在特定任务上取得了显著的改进，但泛化性能仍然很低。为了解决这一限制，一些研究人员提出利用无监督技术利用大量数据进行一般模型的预训练，然后针对不同的下游任务对其进行微调[44-49]。例如，CuBERT[50]在收集的大型 Python 语料库（7.4 万个文件）上预训练 BERT，然后在变量误用识别和错误二进制操作符识别等不同任务上对其进行微调。CodeBERT 对开源代码的六种编程语言进行预训练。

上述提到的研究都是在训练之前引入知识，一些研究者发现，将检索到的知识引入模型训练过程中同样可以获得模型性能的提升[51-57]。基于语言模型可以在推理时简

单地读取和记忆新数据从而立即获得新知识的假设，类似 Memorizing Transformer 的架构被提出，通过对最近（键，值）对的近似 kNN 的查找实现语言模型的扩展，使其具有记忆过去输入的能力。和这一工作比较类似，Borgeaud 提出了一个 transfomer 模型变体，RETRO，基于检索知识的前提下在训练过程中引入外部知识[58]。

受益于检索增强技术，一些代码相关的任务同样也可以依赖这种方式进行改进，然而大部分都应用于生成式相关的任务，比如代码补全，注释生成，代码摘要生成等等。Yu 等人研究了 Bash 代码注释生成问题，提出了一种基于两阶段训练策略的 BASHEXPLAINER 自动生成方法[59]。Lu 等人利用“外部”上下文，提出了一个检索增强的代码补全框架并在 python 及 java 语言上进行评估[60]。

尽管当前基于预训练模型的缺陷预测任务的研究相较于之前的技术已经有很大的改进，但这些研究大多只关注预训练模型在下游任务中的应用，而预训练模型微调相关因素如何影响下游任务仍然未知。因此，系统地探讨不同因素如何影响 JIT 缺陷预测任务性能以及如何对基于预训练模型的即时缺陷预测任务做出优化是至关重要的。此外，即时缺陷预测任务中存在可用的缺陷数据不足的问题，即可以通过预训练的方式学习到的信息是有限的，而检索增强作为一种引入额外信息的方式，已经在生成式任务中获得良好的表现，但检索增强应用于缺陷预测这种理解任务仍是未探索的方向。

因此，本文面向即时缺陷预测分类任务，基于不同预训练模型构建缺陷预测模型，探究不同 Transformer 结构预训练模型在即时缺陷预测任务上的敏感性，以及系统性研究预训练模型调优影响因素，并结合检索增强方式提出两种新的缺陷预测模式，以提高即时缺陷预测任务的表现。此外本文所提出的基于跨模型向量融合的检索增强方法在不同参数量以及模型架构模型中进行了实验验证，均有效果提升。

## 1.2 国内外研究现状

### 1.2.1 即时缺陷预测技术及相关研究

2015 年之前的软件缺陷研究都是基于粗粒度的，随着 Hinton 等人在深度学习领域的研究[2-4]，各个领域之间交叉融合的趋势更加明显。依赖于近些年深度学习的发展[5-8]，各种网络结构的提出为变更代码进行特征提取提供了新的思路[9]。在传统即时缺陷预测技术研究中，研究者往往使用提取出的特征连接一个分类器进行模型构建，分类器

通常为机器学习中常见的算法，例如随机森林，支持向量机，逻辑回归等等[10][11]。然而前面提到的这些研究都是基于传统的人工提取特征，近些年随着自然语言处理领域的发展，研究者更多地意识到，可以从代码和提交信息本身出发，进行自动地特征提取[13][14]。

很多研究者发现一次变更提交中，提交信息由提交者提供，本身就是自然语言，而代码则是半结构化语言，也具有自然性。这些特点天然提供了即时缺陷预测技术和自然语言处理的相关性。受益于卷积神经网络（CNN）和循环神经网络（RNN）[12]等网络的提出，即时缺陷预测技术也不仅仅局限于人工提取的传统特征。Hoang 提取出了一种基于卷积神经网络的端到端即时缺陷预测框架 DeepJIT，该框架通过利用 CNN 对一次提交中的提交信息和变更代码进行特征提取，再将融合后的特征输入一层全连接进行分类，实现脱离人工提取特征的即时缺陷预测[13]。

2020 年 Hoang 提出了一种代码变更的分布式向量表示 CC2Vec，通过 GRU 和层次注意力网络（HAN）[14]，利用提交注解作为目标对象进行代码变更的训练，从而获得表示代码变更的嵌入式向量[15]。这两种利用深度学习方法的即时缺陷预测模型在 auc 的评估维度上都获得了良好的表现。针对 CC2Vec 的优秀表现，Zeng 等人对该模型做了更加详细的分析和对比，并在原有基础上进行了数据集的扩充，在肯定了 CC2Vec 性能的同时也对其缺陷进行了分析，并提出一种使用简单传统特征的逻辑回归分类器 LApredict，证实了深度学习方法还有很多可研究的空间[16]。

当前深度即时缺陷预测技术的研究还不够成熟，随着 Transformer 的提出[17]，许多依赖 Transformer 结构的预训练模型也相继出现[18][19]。当前预训练模型在分类任务中的良好的表现，也为深度缺陷预测技术提供了新的思路方法[20]。Zhou 等人对 CodeBERT 在编程语言相关的任务上进行了评估，并在 QT 和 Openstack 两个数据集上进行评估，展示出了 CodeBERT 应用于对应任务上的良好性能。尽管当前模型相较于之前的技术已经有很大的改进，但是目前即时缺陷预测任务中仍存在可用的缺陷数据不足的问题，即可以通过预训练的方式学习的信息是有限的。

### 1.2.2 预训练模型调优研究

预训练模型的出现引入了一种新的两步预训练和微调范式，以解决典型任务。目前已经提出了一些基于 Transformer 编码器的预训练模型，如 RoBERTa [19] ，CodeBERT

[20]，基于解码器的预训练模型，如 GPT2 [21] ，CodeGPT，基于编码器－解码器的模型，如 BART [22]和 PLBART[23]。研究人员将预训练模型应用于编程语言任务，如文本分类、代码搜索和漏洞检测，已经显示出显著的改进[24][27] 。

BERT 风格模型是 Devlin 等人的工作[19]中提出的 BERT 模型的变体。一些研究者通过探索一些优化方法来对原来的 BERT 模型继续预训练。RoBERTa 在预训练 BERT 时探索设计决策改进预训练过程，CodeBERT 是 RoBERTa 的一个变体，它使用自然语言和编程语言两种模态的训练语料，可以适应于与代码相关的任务。这些 BERT 风格的模型为解决相关问题提供了新的基础。

最初，研究人员经常选择基于预训练模型对下游任务进行全参数微调。BERT 微调过程的不稳定性自 BERT 被引入以来一直得到承认，研究人员提出各种方法来解决这个问题。Phang 等人[28]证明了在大型中间任务上对预训练模型进行微调可以稳定随后在小数据集上的微调。Lee 等人[29]引入了一种新的正则化方法来约束微调模型，使其与预训练的权值保持接近，显示了其对微调的稳定作用。Mosbach 等[30]表明 BERTADAM 会导致微调过程中的不稳定。

与在完全微调场景中探索调优稳定性相比，Aghajanyan 等人[31]发现，更大的网络或在更多数据上预训练的网络需要更小的修改来学习新任务。这一观察结果解释了参数高效微调（PEFT）方法的成功，并推动了低秩微调方法的发展，如 LoRA[32]和 Compater[33]。与完整的微调范式不同，参数高效的微调方法基于这样的假设：在复杂网络训练的早期阶段，实现与整个网络相同的性能是可能的。LoRA 是一种高效的 PEFT 方法，它将原来的权重矩阵转化为两个低秩矩阵。这种方法可以减少计算资源和内存资源的消耗。

以往的研究大多只关注预训练模型在下游任务中的应用，而预训练模型如何影响某些任务仍然是未知的。虽然 Guo 等人[34]进行了实证研究，探讨了不同的基于体系结构的预训练模型在 JIT 缺陷预测任务上的性能，但他们只关注外部因素：Transformer 体系结构、输入数据等。与预训练模型的微调相关的因素的影响仍然未知。Zhang 等人[35]探索了 BERT 上下文表示的微调，但他们只关注常见的自然语言理解任务。因此，系统地探讨不同因素如何影响 JIT 缺陷预测任务的预训练模型的性能是至关重要的。

### 1.2.3 基于检索增强的技术研究

关于检索增强技术，首先需要给对应任务提供外部数据库，并基于信息检索（Information Retrieval，IR）的技术，从外部数据库中检索出和问题相关的信息，然后让模型结合相关信息生成结果。这种模式有时候也被称为检索增强生成（Retrieval Augmented Generation，RAG）。

预训练模型提升在自然语言领域任务表现的同时，也带来了程序语言相关任务的新范式，基于预训练模型，深度学习已广泛应用于软件漏洞检测、源代码总结，深度代码搜索，源代码补全等多种软件工程任务。由于复杂的数据依赖关系和控制依赖关系更容易引发软件漏洞，Devign 将不同类型的程序结构信息与 Code Property Graph 结合到 Graph Neural Network 中进行漏洞检测[41]。Liu 等人将检索到的代码－摘要对结合起来生成高质量的摘要[43]。尽管这些自定义网络在特定任务上取得了显著的改进，但泛化性能仍然很低。为了解决这一限制，一些研究人员提出利用无监督技术对大量数据进行通用模型[41,43-49]的预训练，然后针对不同的下游任务对其进行微调。

训练代码相关的预训练模型本质是为了提供一种更好的源代码向量表示。但一个重要的问题是这些模型表示代码语义时是否具有鲁棒性。Liu 等人的研究证明，最先进的预训练模型对简单的标签保留程序突变（如变量重命名）非常敏感[39]。他们利用克隆检测（POJ-104）的测试数据在预训练模型 CodeBERT 和 GraphCodeBERT 上做重命名变量的代码相似度比较，结果显示 CodeBERT 和 GraphCodeBERT 都受到重命名操作的严重影响。基于此，Liu 等人提出了 ContraBERT，一个基于无监督对比学习的框架，以增强现有预训练模型在代码场景中的语义鲁棒性从而获得更好的代码向量表示[51]。

检索增强技术往往作为一种提高生成性任务表现的良好技术[53-58]。受益于检索增强技术，一些代码相关的任务同样也可以依赖这种范式进行改进，然后现有研究主要将检索增强应用于生成式相关的任务，比如代码补全，注释生成，代码摘要生成等等。

Yu 等人研究了 Bash 代码注释生成问题，提出了一种基于两阶段训练策略的 BASHEXPLAINER 自动生成方法[59]。在第一阶段，在构建的 Bash 代码语料库上微调 CodeBERT 来训练 Bash 编码器。在第二阶段，首先根据语义和词法相似性从代码存储库中为目标代码检索最相似的代码。然后使用经过训练的 Bash 编码器生成两个向量表

示，通过融合层融合这两个向量表示并通过解码器生成代码注释。

Lu 等人利用“外部”上下文实对应任务的性能提上，这一灵感来自人类在编写代码时从相关代码片段复制的行为进行代码补全任务的探究。具体来说，他们提出了一个检索增强的代码补全框架，利用词法复制和检索引用具有相似语义的代码获得额外的信息，并采用了一种分阶段训练方法，该方法结合了源代码检索器和编程语言的自回归语言模型，并在 python 及 java 语言上进行评估[60]。

## 1.3 研究目标及内容

本文旨在研究面向基于预训练模型的即时缺陷预测分析与优化技术，图 4 展示了本文的整体研究思路，通过探究不同模型结构以及模型调优等各方面因素，分析即时缺陷预测任务面临的问题和并探究可优化的影响因素，并结合基于不同架构预训练模型和基于不同微调策略的实证研究中的发现，基于检索增强的范式提升任务的表现，从而使得开发人员能在开发过程中尽早发现缺陷代码并作出对应的修改。

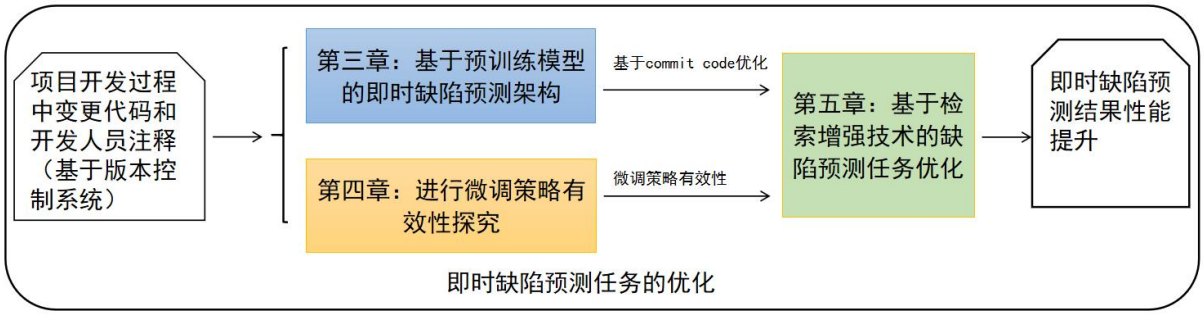


**图 4　基于预训练模型的即时缺陷预测技术研究内容**

### 1.3.1　预训练模型结构对缺陷预测任务的实证研究

本研究中使用三种不同结构的预训练模型：基于 Transformer encoder，基于 Transformer decoder 以及基于 Transformer encoder-decoder 的预训练模型以构造 JIT 缺陷预测模型，为了全面评估这些预训练的 JIT 缺陷预测模型，本文选择了三种不同 Transformer 结构预训练模型的自然语言和编程语言相关的两个不同版本作为 JIT 缺陷预测模型的骨干网络。利用相应的预训练模型和 CNN 网络构建了 6 个深度 JIT 缺陷预测模型：RoBERTaJIT、CodeBERTJIT、GPT2JIT、CodeGPTJIT、BARTJIT 和 PLBARTJIT。本文的主要动机是研究不同 Transformer 结构在 JIT 缺陷预测任务中的性能，以及它们在不同设置下的具体表现，研究中用到的数据集来自 qt 和 openstack 两个开源项目，

实验评价指标主要为 auc，accuracy，precision，recall 以及 f1 score。

研究内容主要分为以下不同的方面：

（1）基于六种不同预训练模型结构的 JIT 缺陷预测模型的性能探究

针对这个研究问题，本文尝试探索在全微调场景下六种不同模型在 JIT 缺陷预测任务中的整体性能。

（2）不同模型的训练效果与预测结果分布之间的关系探究

本文问题考察以预训练模型为骨架的不同 JIT 缺陷预测模型的整体训练效率，以及模型之间预测结果的分布是否存在关联。

（3）不同模型对输入数据的敏感性探究

在这个研究问题中，本文的目标是探索在 JIT 缺陷预测模型中，commit message 和 commit code 作为模型输入如何影响缺陷检测的性能。

（4）不同模型在 few-shot 场景下的具体表现

在这个研究问题中，本文的目的是探索使用平衡数据集的 few-shot 场景下，基于不同骨干模型的 JIT 缺陷预测模型的性能。

### 1.3.2 预训练微调策略对缺陷预测任务的实证研究

本研究为了综合评价影响不同架构模型的 JIT 缺陷预测模型性能的微调因素，进行了全面的实证研究。本文选择 CodeGPT 和 CodeBERT 两个典型的 decoder-only 和 encoder-only 的模型，以往缺陷预测研究中应用这两个模型在 qt 和 openstack 项目中获得了比最先进的方法更好的性能。本文使用 commit message 和 commit code 作为模型输入，并在近 10 万个扩展缺陷预测数据集上评估这两个模型的性能，比较它们在不同微调设置下的性能。实验评价指标主要为 auc 以及 f1 score。

研究内容主要分为以下不同的方面：

（1）参数冻结和初始化对模型性能影响探究

以前基于预训练模型的缺陷预测工作倾向于进行全参数微调操作。局部参数微调对缺陷预测性能的影响尚不清楚。本文研究了两种不同的部分参数微调范式：冻结模型参数更新和在下游任务微调之前初始化参数。

（2）模型对不同特征提取器敏感性探究

在这个研究问题中，本文的目标是研究不同的特征提取器对 BERT 风格 JIT 缺陷

预测模型的影响。研究中选择了四种不同的特征提取器：FCN（全连接网络）、LSTM、Transformer 和 CNN 以进行实验探究。

（3）优化器策略对模型的具体影响探究

这个研究问题的目的是探索不同的超参数设置在 Adam 优化器的有效性，通过对 Adam 设置不同的权重衰减率来探究模型的性能差异。

（4）综合微调策略的经济有效训练设置探究

这个研究问题的目标是探索是否存在一种方法更有效地微调预训练模型并无损模型性能。

### 1.3.3 基于检索增强的即时缺陷预测技术优化

本研究针对缺陷预测任务中存在的可用信息不足的问题，利用检索增强的方式构建缺陷预测的分类模型，并针对现有直接进行文本拼接的检索增强形式做出改进，在 CodeGPT 模型上进行初步实验验证，并使用裁剪后的 Codellama-7B 作为向量编码器，在近 10 万个扩展缺陷预测数据集上评估这模型的性能。此外，我们还将基于跨模型向量融合的检索增强模型在更大参数量的 GPT2-medium 模型以及不同架构的 Llama3.2-1B 模型上进行验证实验，以证明所提方法的有效性和泛化性。评价指标为 auc，f1 score 以及马修斯相关系数 MCC。

研究内容主要分为以下不同的方面：

（1）基于向量相似召回的向量表示获取方式

首先，应用检索增强技术的前提需要获得代码及文本的合理的向量表示，目前广泛的方式是利用一个编码器模型直接获取输入的向量表示，但常用的 BERT 架构模型在获取向量表示时受到长度限制，基于切块的向量表示获取可能导致获取的向量表示存在信息截断和损失，因而本研究选择了 Codellama-7B 作为基模型，该模型可以处理 16384 长度的文本，并进行模型裁剪以保证向量获取的效率。

（2）两种检索增强即时缺陷预测方法的提出

在获取向量表示后，需要根据向量对相似信息做出检索，本文通过选择基于 L2 distance 进行召回。并基于上一研究构造的向量表示提出了基于信息堆叠的检索增强模型架构（Retrieval-based Information Stack，RIS）和基于跨模型向量融合的检索增强模型架构（Retrieval-based Vector Fusion，RVF），结合第二章中提出的模型架构，基

于 CodeGPT 模型在接近 10 万条公开数据集中进行实验验证，证明检索增强应用于即时缺陷预测任务的有效性。

（3）基于跨模型向量融合的检索增强在不同参数量及不同架构下的泛化性探究

信息融合是模型优化中需要考虑的一环，涉及了模型训练的效率以及性能表现，基于信息堆叠检索增强的模型架构受限于模型处理上下文的长度以及显存随检索个数的线性增长，而基于跨模型向量融合的检索增强架构的有效性和泛化性，在同架构更大参数量 GPT2-medium 和不同架构的 Llama3.2-1B 模型上进行实验验证，以证明该架构的有效性和泛化性。

## 1.4 本文组织结构

本文的组织结构划分如下：

第一章 绪论

本章介绍了本文的研究背景和研究意义，概述了基于预训练模型的即时缺陷预测技术以及检索增强技术的相关研究，明确了本文的研究目标和内容。

第二章 相关技术基础介绍

本章介绍了本文所涉及领域的相关基础知识，概述了即时缺陷预测，预训练模型调优以及检索增强国内外研究现状的基本知识。

第三章 预训练模型结构对缺陷预测任务的影响研究

本章详细阐述了预训练模型结构对缺陷预测任务的影响研究的实证方案。该方案主要通过构建基于预训练模型的通用即时缺陷预测模型架构，在通用数据集上全面评估这些基于预训练模型的 JIT 缺陷预测模型，以及影响其性能的具体因素，包括预训练模型架构，模型输入以及缺陷数据占比的影响。

第四章 预训练微调策略对缺陷预测任务的影响探究

本章详细阐述了不同微调策略对基于预训练模型的 JIT 架构性能影响的研究方案。该方案主要通过基于第三章的通用 JIT 架构，探究 encoder-only 和 decoder-only 的两个经典模型 CodeBERT 及 CodeGPT 在应用不同参数冻结，参数初始化以及不同优化器策略下的性能变化，并在接近 10 万条通用数据集上进行评估，并结合上述研究提出一个基于 LoRA 的轻量级微调方式，在保持与全参数微调相比性能无损情况下，实现更

经济有效的微调。

第五章 基于检索增强的即时缺陷预测技术研究

本章结合第三章及第四章的实证研究发现，针对缺陷预测任务中的信息不足问题，提出了两种基于 commit code 的检索增强的方法，即基于信息堆叠（RIS）的和基于跨模型向量融合（RVF）的两种架构来构建 JIT 模型。其中两种架构都在基于第三章提出 CodeGPTJIT 的模型上进行了初步实验，使用裁剪后的 Codellama-7B 作为向量编码器，验证了其性能的提升。此外，基于跨模型向量融合的架构可以在参数量更大的 GPT2-medium 模型和不同架构的 Llama3.2-1B 模型上体现其有效性和泛化。

总结和展望

本章包含对本论文的工作总结和对本领域内未来可行工作的展望。对本论文的工作总结主要概述了本文在基于预训练模型的即时缺陷预测技术领域所做出的贡献，并对本文所实现方案的局限性给出分析；对未来可行工作的展望主要包括对当前工作局限性改进策略的猜想，和此领域进一步研究发展可能性的探究。

# 第二章 相关技术基础介绍

本章主要介绍本文工作的技术背景，主要包括即时缺陷预测任务以及检索增强相关技术的基本知识。

## 2.1 即时缺陷预测

即时缺陷预测方法旨在及时发现软件缺陷，降低修复成本并提升软件质量。通过在开发过程中追溯变更代码，进而实现提交级缺陷预测。这一过程通常包括数据标注、特征提取、模型构建和实验评估。传统技术依赖人工特征提取，并使用机器学习算法进行模型训练。近年来，深度学习的发展使得研究者能够自动从代码中提取特征，从而提升预测效果。此外，随着预训练模型及大模型的出现，微调技术被应用于编程语言任务显示出在这些任务上的显著改进。然而，当前即时缺陷预测中仍存在数据可用性不足以及现有模型架构利用信息不足的问题。

### 2.1.1 传统即时缺陷预测技术

及时发现缺陷，有助于减少修复缺陷的成本，提高软件的质量。研究人员已经提出了大量的缺陷预测技术，根据预测粒度不同，主要包括模块级、文件级和变更级缺陷预测。缺陷预测的研究具有十分重大的意义，因此吸引了很多软件工程领域研究者的关注[1]。其中，Mockus 和 Weiss 首次提出基于代码自身变更属性进行代码失败的预测，这些属性包括一些代码行的变化，修改代码的文件数，开发者的经验等[67-71]。Kim 在 2008 年提出了使用机器学习方法的代码分类方法，即将代码分类为有 bug 和无 bug 类别，对最近代码进行是否有缺陷的分类预测[61]。2013 年 Kamei 正式将这种基于软件变更特征进行缺陷预测的研究方法称为即时缺陷预测技术（Just-In-Time defect prediction）[72]。

即时缺陷预测技术的一般过程通常包括数据标注、特征提取、模型构建以及实验评估。其中数据标注是用来判断代码的变更是否引入缺陷。Sliwerski 等人在 2005 年第一次提出了识别引入缺陷的变更代码的算法，即 SZZ 算法，这种算法可以通过版本控制系统追踪变更的提交历史，并完成缺陷的识别和判定[63]。之后很多研究者又对该算法做出了改进，如 AG-SZZ（Annotation graph SZZ）算法，解决了将注释行、空

行以及注释等非缺陷引入代码的错误识别，即避免将这些代码识别为引入缺陷[64]。MA-SZZ（Meta- change SZZ）算法则在 AG-SZZ 算法的基础上，考虑了分支的改变被错误识别为缺陷引入代码的情况[65]。RA-SZZ（Refactoring aware SZZ）算法则在 MA-SZZ 的基础上，考虑了代码重构的问题被错误识别为缺陷引入代码的情况[66]。

特征提取和模型构建是当前即时缺陷预测研究的重点之一，传统的即时缺陷预测技术大多依赖人工提取特征，并使用不同的机器学习方法进行分类器的训练。即时缺陷预测中使用的特征主要包括人工提取的传统特征和使用深度学习方法提取的特征。Mockus [69]等人第一次提出了人工提取特征的选取，并将其划分为代码行的变化、修改代码的文件数、开发者的经验等不同类别的特征。这些特征具有坚实的实证研究基础。后续的一些研究工作中对传统人工特征进行了扩展，使用代码复杂度、变更文件的数量、日志和代码等构件的词频率等特征来表示变更以及使用相关代码文件修改前后抽象语法树的节点数量的差值表示变更等等。

在传统的即时缺陷预测技术研究中，研究者往往使用提取出的特征连接一个分类器进行模型的构建，分类器通常选择为机器学习中常见的算法，例如随机森林，支持向量机，逻辑回归等等[10,11]。然而上述研究都往往基于传统的人工提取特征，随着自然语言处理领域的发展，研究者更多地意识到，可以从代码和提交信息本身出发，进行自动特征提取[13,14]。

2015 年之前的软件缺陷研究都是基于粗粒度的，随着 Hinton 等人在深度学习领域的研究[2-4]，各个领域之间交叉融合的趋势更加明显。依赖于近些年深度学习的发展[5-8]，各种网络结构的提出为变更代码进行特征提取提供了新的思路。2015 年 Yang 等人提出使用深度学习的方法对即时缺陷预测进行改进，该研究使用了深度信念网络对人工提取的特征进行维度的处理和映射，这是即时缺陷预测技术中首次用到深度学习[9]。很多研究者发现一次变更提交中，提交信息由提交者提供，本身就是自然语言，而代码则是半结构化语言，也具有自然性。这些特点天然提供了技术缺陷预测技术和自然语言处理的相关性。受益于卷积神经网络（CNN）和循环神经网络（RNN）[12]等网络的提出，即时缺陷预测技术也不仅仅局限于人工提取的传统特征。Hoang 提取出了一种基于卷积神经网络的端到端即时缺陷预测框架 DeepJIT，实现脱离人工提取特征的即时缺陷预测[13]。

2020 年 Hoang 提出了一种代码变更的分布式向量表示 CC2Vec，通过 GRU 和层次注意力网络（HAN）[14]，利用提交注解作为目标对象进行代码变更的训练，从而获得表示代码变更的嵌入式向量[15]。这两种利用深度学习方法的即时缺陷预测在 auc 的评估维度上都获得了良好的表现。针对 CC2Vec 的优秀表现，Zeng 等人对该模型做了更加详细的分析和对比，并在原有基础上进行了数据集的扩充，在肯定了 CC2Vec 性能的同时也对其缺陷进行了分析，并提出一种使用简单传统特征的逻辑回归分类器 LApredict，证实了深度学习方法还有很多可研究的空间[16]。

### 2.1.2 基于预训练微调的即时缺陷预测技术

预训练模型的出现引入了一种新的两步预训练和微调范式，以解决下游任务。目前已经提出了一些基于 Transformer 编码器的预训练模型，如 RoBERTa [19]，CodeBERT [20]，基于解码器的预训练模型，如 GPT2 [21]， CodeGPT，基于编码器－解码器的模型，如 BART [22]和 PLBART[23]。研究人员将预训练模型应用于编程语言任务，如文本分类、代码搜索和漏洞检测，已经显示出显著的改进[24-27]。

当前深度即时缺陷预测技术的研究还不够成熟，2017 年 Google 提出了一种神经网络结构 Transformer，这种结构提供了一种不利用 CNN 和 RNN 的全注意力机制[17]，Devlin 等人利用 Transformer 结构提出了 BERT 预训练模型[18]。Liu 等人对 BERT 模型做出改进，优化了参数的选择并扩大了 BERT 模型的语料库，提出了 RoBERTa 模型[19]。由 Feng 等人提出的 CodeBERT，提供了自然语言和编程语言的双模态预训练模型，可以用于软件工程和自然语言的一些下游任务[20]。当前预训练模型在分类任务中均具有良好的表现，也为缺陷预测技术提供了新的思路方法。Zhou 等人对 CodeBERT 在编程语言相关的任务上进行了评估，在 QT 和 Openstack 两个数据集上的评估结果展示出 CodeBERT 应用于对应任务上的良好性能。尽管当前研究相较于之前的技术已经有很大的改进，但是即时缺陷预测任务中仍存在可用的缺陷数据不足的问题，即可以通过预训练的方式学习的信息是有限的。

以往的研究大多只关注预训练模型在下游任务中的应用，而预训练模型如何影响某些任务仍然是未知的。虽然 Guo 等人[34]进行了一项实证研究，探讨了基于不同架构的预训练模型在 JIT 缺陷预测任务上的性能，但他们只关注外部因素：Transformer 架构、输入数据等。与预训练模型的微调相关的因素的影响仍然未知。Zhang 等人[35]探

索了 BERT 上下文表示的微调，但他们只关注常见的自然语言理解任务。因此，系统地探讨不同因素如何影响 JIT 缺陷预测任务的预训练模型的性能是至关重要的。

## 2.2 检索增强

基于预训练模型的探索已广泛应用于软件漏洞检测、源代码总结，深度代码搜索，源代码补全等多种软件工程任务。由于复杂的数据依赖关系和控制依赖关系容易引发软件漏洞，Devign[41]将不同类型的程序结构信息与 Code Property Graph 结合到 Graph Neural Network 中进行漏洞检测。Liu 等人[43]将检索到的代码－摘要对结合起来生成高质量的摘要。尽管这些自定义网络在特定任务上取得了显著的改进，但泛化性能仍然很低。为了解决这一限制，一些研究人员提出利用无监督技术和大量数据进行一般模型[44-49]的预训练，然后针对不同的下游任务进行微调[50,51]。受益于此，检索增强技术可以基于预训练模型构造的向量，进行信息相似度的判定，并召回额外相关信息对输入上下文进行增强。这一方法已经被广泛地应用于现存的大模型下游任务应用如代码补全任务，本节将简要介绍检索增强技术的核心要素及检索增强在一些在代码相关任务上的具体应用。

### 2.2.1 向量相似性计算

向量的其中一个特性是可以通过距离计算，来判断向量之间的相似度，在推荐系统/广告中，一般使用 embedding 做相似召回。自然语言内容理解也经常会根据一些内容特征预训练得到一个 item embedding，并选择在排序侧，将预训练好的 item embedding 作为模型的特征输入。大多数的检索技术通常基于向量的相似度进行设计。

最常见的向量相似计算包括欧氏距离，余弦距离，汉明距离，杰卡德相似系数，BM25 算法等等。以常见的欧式距离来算，假设向量的维度为 k，库总共有 n 个 item。则搜索一次的时间复杂度为 O(k*n)，且整体的查询耗时与 item 库的大小线性相关，线上查询要遍历成百万上千万的暴力计算并不满足需求。考虑到向量检索的大部分场景都是返回与该元素相似的前 k 个元素即可，比如推荐中的召回，只需召回近似 100 个 item，也不关注 item 间的相对顺序，也无需 100%准确。一些研究者通过近似最近邻搜索（Approximate Nearest Neighbor Search）来解决精度和效率上的问题。

FAISS[52]是一个开源的 Python 库，封装了许多常见的检索技术，FAISS 通过更好

地利用 GPU 来解决检索的问题。虽然 GPU 擅长数据并行任务，但先前的方法受限于显示出较少并行性的算法，例如 k-最小选择，或者对内存层次结构的使用很差。通过设计一种 k 选择方式，可以使最近邻实现比以前的 GPU 技术状态快 8.5 倍。

ScaNN[53]基于量化将最大内积搜索扩展到大规模数据库。传统的量化方法以最小化数据库点的重构误差为目标。基于对给定查询的观察，具有最大内积的数据库点更相关，ScaNN 通过一组各向异性量化损失函数来设计检索。从而实现近似的最近邻搜索。

### 2.2.2 基于预训练模型的向量相似性

预训练模型的出现是传统自然语言处理任务的一个分水岭，将文本输入预训练模型，可以获得对应模型的向量表示，BERT 论文中对 STS-B 数据集进行有监督训练[18]，最终达到了 85.8 的斯皮尔曼相关值。对于语义相似度任务来说：在有监督范式下，BERT 需要将两个句子合并成一个句子再对其进行编码，如果需要计算很多文本两两之间的相似度，BERT 将其排列组合后送入模型，这种组合极大地增加了模型的计算量。而在无监督范式下，BERT 表征的句向量会出现携带语义相似度信息较少的问题。Sentence-BERT[37]是一篇采用孪生 BERT 架构的工作。Sentence-BERT 作者指出，如果想用 BERT 表示计算出 10000 个句子之间的两两相似度，排列组合的方式在 V100 GPU 上测试需要花费 65 小时；如果先表示出 10000 个句子，再计算余弦相似度矩阵，则只需要花费 5 秒左右。因此，作者提出了通过孪生网络架构训练 BERT 句向量的方式。

Sentence-BERT 一共采用三种不同的方式训练孪生 BERT 架构，分别为 Cross-entropy loss，MSE loss 以及 Triple loss，基于对比学习的模式增强 BERT 模型对文本的向量表示。BERT-flow 是一篇通过对 BERT 句向量做后处理的工作。作者认为，直接用 BERT 句向量来做相似度计算效果较差的原因并不是 BERT 句向量中不包含语义相似度信息，而是其中包含的相似度信息在余弦相似度等简单的指标下无法很好地体现出来。

BERT-flow[38]基于两个前提，即语言模型和掩码语言模型都基于最大化给定的上下文与目标词的共现概率（也就是上下文和目标贡献概率）。如果两句话预测出的目标词一致，那么两句话的上下文向量很有可能也是相似的。其次，BERT 的句向量空间是各向异性的，且高频词距离原点较近，低频词距离较远，且分布稀疏。因此 BERT

句向量无法体现出其中包含的相似度信息。BERT-flow 通过一个基于流的生成模型对 BERT 句向量空间进行映射，从而训练出一个标准的高斯分布，使得该分布中的点可以与 BERT 句向量中的点一一映射。由于该方法采用的映射方式是可逆的，因此可以通过给定的 BERT 句向量信息，映射回标准高斯空间，然后再做相似度计算。由于标准高斯空间是各向同性的，因此能够将句向量中的语义相似度信息更好地展现出来。

SimCSE[39]则基于对比学习学习文本的语义相似度。对比学习相较于文本对之间的匹配，可以在拉近与其相似的正例的同时，同时拉远与更多负例之间的距离，从而训练出一个更加均匀的超球体向量空间。SimCSE 利用模型自身的 dropout 来生成两个不同的 sentence embedding 作为正例进行对比。

ConSERT[40]同样基于对比学习获取基于 BERT 模型的文本相似度。ConSERT 采用多源数据增强的方式构造正例，其中包括对抗攻击，打乱文本中的词顺序，Cutoff 以及 Dropout。与 SimCSE 不同的是，ConSERT 的数据增强操作只停留在嵌入层，而 SimCSE 则采用了对 BERT 所有层进行 Dropout。

这些预训练模型本质是为了提供一种更好的文本向量表示。但一个重要的问题是这些模型是否具有鲁棒性来表示代码语义。Liu 等人的研究证明，最先进的预训练模型对简单的标签保留程序突变（如变量重命名）不具有鲁棒性。他们利用克隆检测（POJ-104）的测试数据[39]（检测两个函数在不同实现下是否语义等价的任务），在预训练模型 CodeBERT 和 GraphCodeBERT 上做重命名变量的代码相似度比较，结果显示 CodeBERT 和 GraphCodeBERT 都受到重命名操作的严重影响。

### 2.2.3 检索增强技术应用案例

检索增强技术往往作为一种提高生成式任务表现的良好方式。Khandelwal[54]等人针对语言模型中 next-token-prediction 的任务引入了一种 kNN-LM 的结构，它通过 k 近邻（kNN）模型线性插值来扩展预训练的语言模型。根据预训练语言模型嵌入空间中的距离计算最近邻。检索回的结果被应用到一个强大的语言模型上，从而实现不用任何训练的情况下降低模型困惑度的效果。

开放域问答和机器阅读理解也是检索方法经常使用的研究领域。陈丹琦 17 年的工作中[36]，将基于检索增强的此类任务系统分为两大块，对给定的问题，文档检索器从所有维基百科文章中检索对应的信息；检索到的文章会被切分为段落，文档阅读器的

模块会在段落中预测答案位置并给出分数。这一工作为后续的其他研究提供了基本的框架和思路。

Karpukhin[55]等人在开放领域问答中利用稠密的向量表示进行检索，而非传统的基于 token 的 TF-IDF 或 BM25 方法。他们的工作表明，检索可以实际地使用稠密向量表示来实现，其中嵌入向量的获得基于一个简单的双编码器框架，并从少量的问题和段落中学习。REALM[56]作者认为，传统的完全依赖于模型内部权重进行知识下学习是一种非常低效率的做法，正确的做法是让模型学会找和理解。基于这一思路以及预训练模型的发展，他将这一过程分为了两个组件，检索器分别用 BERT 获取查询和文档的 embedding，通过向量相似度计算后联合召回的知识以及原来的问题构成新的输入。

上述提到的工作都是在训练之前引入知识，一些研究者发现，将检索到的知识引入模型训练过程中同样可以获得模型性能的提升。

Wu 等人认为，语言模型在获取新知识时通常需要进行训练或微调，这涉及模型权重的更新[57]。基于语言模型可以在推理时简单地读取和记忆新数据从而立即获得新知识的假设，Wu 等人提出了 Memorizing Transformer，通过对最近（键，值）对的不可微分记忆的近似 kNN 查找，扩展语言模型，使其具有记忆过去输入的内部表示的能力。

和 Wu 的工作比较类似，Borgeaud 也利用检索的方式改进了 Transformer 模型，并提出 RETRO 框架。该方法从巨量文本中采样出部分数据集，然后用同样的采样数据构建数据库，并在训练验证阶段利用所有巨量文本数据构建数据库。RETRO 采用了冻结的 BERT 模型做检索器并在计算注意力时引入了 cross-chunked attention， 把检索到的 embedding 并入自注意力计算中，从而实现在训练过程中引入外部知识[58]。

受益于检索增强技术，一些代码相关的任务同样也可以依赖这种方式进行改进。大部分研究都将检索增强应用于生成式相关的任务，比如代码补全，注释生成，代码摘要生成等等。

Yu 等人研究了 Bash 代码注释生成问题， 并提出了一种基于两阶段训练策略的 BASHEXPLAINER 自动生成方法。在第一阶段，该方法在构建的 Bash 代码语料库上微调 CodeBERT 进行以 Bash 编码器的训练。在第二阶段，该方法会语义和词法相似性从代码存储库中为目标代码检索最相似的代码，然后使用经过训练的 Bash 编码器生

成两个向量表示，融合层融合这两个向量表示并将通过解码器生成代码注释[59]。

Lu 等人利用“外部”上下文进行知识的召回，这一灵感来自人类在编写代码时从相关代码片段复制的行为进行代码补全任务的探究。具体来说，他们提出了一个检索增强的代码补全框架，利用词法复制，并检索引用具有相似语义的代码，这一框架采用了一种分阶段训练方法，并结合了源代码检索器和编程语言的自回归语言模型在在 python 及 java 语言上进行评估[60]。

## 2.3 本章小节

本章首先介绍了即时缺陷预测技术的实现框架流程，以及基于传统的和深度学习的两种不同范式。随后，本章介绍了当前应用预训练模型进行微调的相关研究以及如何基于预训练模型对即时缺陷预测任务进行探索。最终，本章介绍了现有检索增强技术的关键技术要素以及检索增强在代码相关任务中的应用，概述了检索增强可用于即时缺陷预测中的可能性，提供了本文基于预训练模型进行即时缺陷预测技术探究和改进的必要知识。

# 第三章 预训练模型结构对缺陷预测任务影响的实证研究

鉴于现有研究对基于预训练模型的 JIT 缺陷预测架构的性能影响因素尚未充分探讨，本文构建了基于六种不同预训练模型结构的 JIT 缺陷预测模型，包括 RoBERTaJIT、CodeBERTJIT、GPT2JIT、CodeGPTJIT、BARTJIT 和 PLBARTJIT。研究主要围绕以下四个方面展开：不同模型架构，训练数据分布，code 和 message 的各自影响，以及模型在 few-shot 场景下的性能，来对基于预训练模型的 JIT 缺陷预测任务进行全面的实证研究。本章首先会介绍实验的目的和内容，随后介绍实施实验的环境和设计，模型框架的具体实现，最后给出实验的结果和分析。

## 3.1 实证研究目的与内容

本章使用三种不同结构的预训练模型：基于 Transformer encoder，基于 Transformer decoder 以及基于 Transformer encoder-decoder 的预训练模型构造 JIT 缺陷预测模型，为了全面评估这些预训练的 JIT 缺陷预测模型，本小节选择了三种不同 Transformer 结构预训练模型的自然语言和编程语言相关的两个不同版本作为 JIT 缺陷预测模型的骨干网络。利用相应的预训练模型和 CNN 网络构建了 6 个深度 JIT 缺陷预测模型：RoBERTaJIT、CodeBERTJIT、GPT2JIT、CodeGPTJIT、BARTJIT 和 PLBARTJIT。本研究的主要动机是研究不同 Transformer 结构在 JIT 缺陷预测任务中的性能，以及它们在不同情况下的行为，研究中用到的数据集来自 qt 和 openstack 两个开源项目，实验评价指标主要为 auc score，accuracy，precision，recall 以及 f1 score。

研究内容主要分为以下不同的方面：

（1）探究基于六种不同预训练模型结构的 JIT 缺陷预测模型的性能

针对这个研究问题，本文尝试探索在全微调场景下六种不同模型在 JIT 缺陷预测任务中的整体性能。

（2）探究不同模型的训练效果与预测结果的分布之间关系

本文问题考察以预训练模型为骨架的不同 JIT 缺陷预测模型的整体训练效率，以及模型之间预测结果的分布是否存在关联。

（3）不同的模型对输入数据的敏感性

在这个研究问题中，本文的目标是探索在 JIT 缺陷预测模型中，commit message 和 commit code 作为模型输入如何影响缺陷检测的性能。

（4）不同的模型在 few-shot 场景下的具体表现

在这个研究问题中，本文的目的是探索使用平衡数据集的 few-shot 场景下，基于不同骨干模型的 JIT 缺陷预测模型的性能。

## 3.2 基于预训练模型通用框架实现

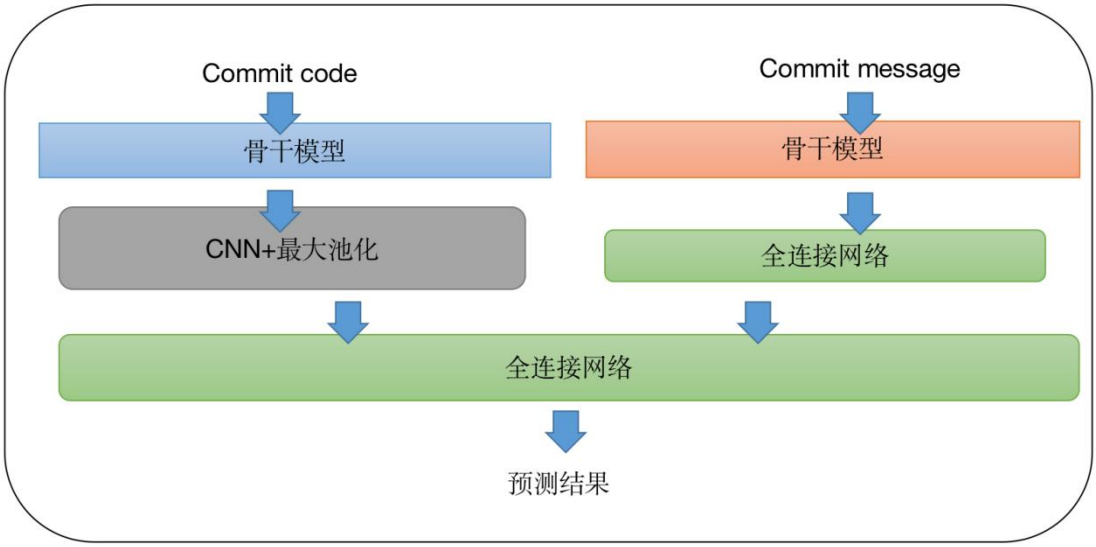


**图 5　基于不同预训练模型的即时缺陷预测模型框架**

图 5 说明了本研究中六个 JIT 缺陷预测模型的框架，所有这些模型都使用 commit code 和 commit message 作为模型输入。在处理 commit code 的过程中，本研究采用与 DeepJIT 模型类似的方法，从每个提交中选择最近的四个修改文件。提交代码表示为 $C = [P_1, P_2, \ldots, P_{|C|}]$，其中$|C|$代表提交中的修改文件数量，$|C| = 4$。如果提交中的修改文件数量少于 4 个，则使用“empty patch”填充剩余的位置。然后将每个补丁$P_i$作为提交代码中的一个序列，作为代码输入骨干预训练模型，以获得嵌入表示$e_i$，如公式（3.1）。

$$e_i = \mathrm{BackboneModel}(P_i) \tag{3.1}$$

当提交代码中的每个补丁由预训练模型处理后，本研究使用⊕操作得 commit code 的向量表示，遵循公式（3.2）。

$$C_e = e_1 \oplus e_1 \oplus \ldots \oplus e_{|C|} \tag{3.2}$$

对于 commit message 的处理，本研究将整个提交信息视为序列 $M = [w_1, w_2, w_3, \ldots, w_{|m|}]$。M 是由一组单词组成的序列，|m|是序列的长度。然后本研究将提交信息 M 输入骨干模型以获得嵌入表示，遵循公式（3.3）。

$$e_m = \mathrm{BackboneModel}(M) \tag{3.3}$$

本研究使用 CNN 和最大池化来提取代码嵌入的特征。其中定义一个过滤器 $f \in R^{k\,x\,d}$ 并使用$k$个窗口来提取提交代码的新特征。对于代码补丁$e_i$，本研究根据公式（3.4）得到新的向量$x_i$。

$$x_i = \mathrm{ReLU}(f * e_{i:i+k-1} + b_i) \tag{3.4}$$

在本文中，符号*表示元素积之和，ReLU 是非线性激活函数，$b_i \in R$ 是偏置值。通过将过滤器$f$应用于提交代码的每 k 文件来生成新特征$x_i$。新特征$x_i$串联成向量$X$，如公式（3.5）所示。

$$X = [x_1, x_2, \ldots x_{|C|-k+1}] \tag{3.5}$$

对于新的特征向量$X$，本研究使用最大池化方法处理以获得最显著的特征$Z_c$，因为 commit message 的序列长度比 commit code 的序列长度短，所以在提交信息嵌入的特征提取中仅使用一层全连接网络。根据公式（3.6）以获得 commit message 特征向量$Z_m$。

$$Z_m = W_m \cdot e_m + b \tag{3.6}$$

在获得 commit code 的特征向量$Z_c$和 commit message 的特征向量$Z_m$后，本研究将$Z_c$和$Z_m$融合以获得一次提交的完整特征向量$Z$，如公式（3.7）所示。

$$Z = Z_c \oplus Z_m \tag{3.7}$$

将 Z 输入一层全连接网络作为分类器，通过 sigmoid 函数将高维向量映射到一维向量，完成二分类的预测任务，如公式（3.8）所示。

$$y = \mathrm{sigmoid}(\mathrm{ReLU}(W_h Z + b_h)) \tag{3.8}$$

## 3.3 实验设置与数据

本研究旨在探究不同 Transformer 结构在 JIT 缺陷预测任务中的有效性，并在 qt 和 openstack 两个开源项目构造的数据集中评估这些模型的性能，所用数据集统计信息如表 1 所示，其中 qt 项目中包含 25704 条提交数据，openstack 项目中包含 13304 条实验数据，两个项目的缺陷代码率分别是 7.1%和 12.2%。

**表 1　数据集统计信息**

| 项目 | Commit 数目 | 缺陷数目 | 缺陷占比（%） |
|---|---|---|---|
| qt | 25704 | 825 | 7.1 |
| openstack | 13304 | 1627 | 12.2 |

实验运行环境基于 Ubuntu 18.04，以及配备 24GB 显存的 RTX3090，并使用 Python 的官方版本 3.7 实现实验代码。

实验超参设置如表 2 所示，其中所有模型分别在两个数据集中训练 3 个 epoch，且学习率均设定为 1e-5，使用固定大小 batch size=16，在 Adam 优化器设置下进行训练，其中 80%为训练数据，10%为验证数据，另外 10%为测试数据。

**表 2　训练超参设置**

| 项目 | 轮次 | 学习率 | 优化器 | 批次大小 |
|---|---|---|---|---|
| qt | 3 | 1e-5 | Adam | 16 |
| openstack | 3 | 1e-5 | Adam | 16 |

## 3.4 实证研究实验与分析

### 3.4.1　基于六种不同预训练模型结构的 JIT 缺陷预测模型的性能探究

针对这一研究问题，本研究旨在系统探索不同模型在即时缺陷预测（JIT）任务中的整体性能表现。我们对六种模型进行了全面比较，包括 RoBERTaJIT、CodeBERTJIT、GPT2JIT、CodeGPTJIT、BARTJIT 和 PLBARTJIT，以评估它们在 JIT 缺陷预测任务中的有效性。此外，为了进一步验证这些模型的性能，研究引入了两个当前最先进的基线模型 DeepJIT 和 CC2Vec，将它们与六个自定义构造的 JIT 缺陷预测模型进行对比分析。传统研究大多采用单一的 auc（Area Under Curve）作为 JIT 缺陷预测模型的评价指标。然而，单一评估度量可能会导致结果的片面性或偏倚。为全面评估模型的性能，本研究不仅考虑了 auc，还引入了其他评估指标，通过综合多维度的性能评估指标，本研究对各模型的优劣势进行了深入分析，并提供详尽的结果解读，以揭示模型在不同场景下的适用性与局限性。

**表 3 相比于基线模型在 qt 项目中表现**

| 模型 | auc score | accuracy | precision | recall | f1 score |
|---|---|---|---|---|---|
| DeepJIT | 0.79 | 0.73 | 0.17 | 0.70 | 0.27 |
| CC2Vec | 0.81 | 0.56 | 0.13 | 0.87 | 0.23 |
| RoBERTaJIT | 0.81 | 0.87 | 0.27 | 0.48 | 0.35 |
| CodeBERTJIT | 0.81 | 0.87 | 0.26 | 0.42 | 0.32 |
| GPT2JIT | 0.81 | 0.81 | 0.21 | 0.58 | 0.31 |
| CodeGPTJIT | 0.81 | 0.89 | 0.29 | 0.39 | 0.33 |
| BARTJIT | 0.82 | 0.82 | 0.23 | 0.62 | 0.34 |
| PLBARTJIT | 0.82 | 0.90 | 0.32 | 0.34 | 0.33 |

本研究构造的 6 个模型实验结果如表 3 和表 4 所示，相比于两个基线模型 DeepJIT 和 CC2Vec，本研究构建的基于预训练模型的架构获得了除了召回率之外的其他评估指标上的显著增强，本研究观察到不同 Transformer 架构的模型都拥有出色性能。

**•结论 1:** 用三种不同的 Transformer 结构构建的所有 JIT 缺陷预测模型在两个数据集中都表现出明显改进。

**表 4 相比于基线模型在 openstack 项目中表现**

| 模型 | auc score | accuracy | precision | recall | f1 score |
|---|---|---|---|---|---|
| DeepJIT | 0.74 | 0.61 | 0.17 | 0.76 | 0.28 |
| CC2Vec | 0.76 | 0.27 | 0.11 | 0.98 | 0.19 |
| RoBERTaJIT | 0.79 | 0.80 | 0.32 | 0.50 | 0.39 |
| CodeBERTJIT | 0.80 | 0.80 | 0.33 | 0.56 | 0.42 |
| GPT2JIT | 0.79 | 0.71 | 0.26 | 0.76 | 0.39 |
| CodeGPTJIT | 0.81 | 0.76 | 0.30 | 0.70 | 0.42 |
| BARTJIT | 0.81 | 0.71 | 0.27 | 0.77 | 0.40 |
| PLBARTJIT | 0.81 | 0.79 | 0.32 | 0.62 | 0.42 |

图 6 展示了六个模型在 qt 和 openstack 数据集上的预测结果分布。很明显，这些模型倾向于将 qt 数据集中的大多数提交预测为无缺陷提交，而这种行为在 openstack 数据集中有所不同。这一观察结果也反映在表 7 和表 8 中，其中 openstack 数据集上六个模型的召回值明显优于 qt 数据集。而前两种最先进的模型 DeepJIt 和 CC2vec 在

召回性能上的差异非常小。

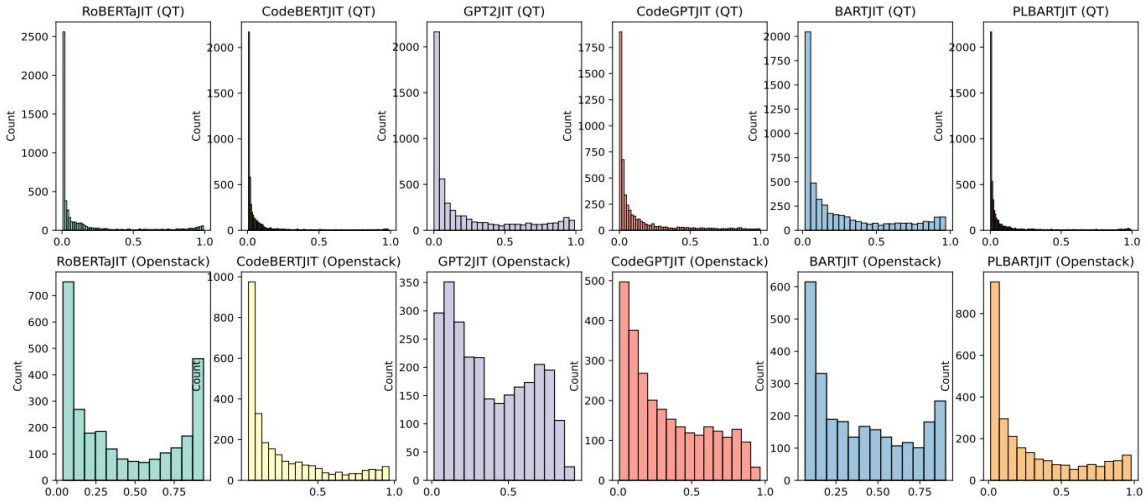


**图 6　模型在两个数据集中的预测分布**

为了进一步研究这一现象，本研究将现有结果与延长模型训练时间后得到的结果进行了比较。图 7，图 8，图 9 和图 10 分别描述了原始结果与使用较长训练时间获得的结果在 qt 和 openstack 两个数据集中的差异。很明显，随着训练时间的增加，六个模型的准确率都提高了，而召回值却下降了。这可以归因于基于预训练模型的 JIT 缺陷预测模型由于其深度和复杂的网络结构而具有较强的学习能力。因此，在训练过程中，当面对一个不平衡的数据集时，模型的参数倾向于拟合拥有更多数据的类。这就解释了为什么这些大型的预训练模型在 JIT 缺陷预测任务中表现出较低的召回率。

•**结论 2:** 复杂模型对数据集的平衡性比较敏感。

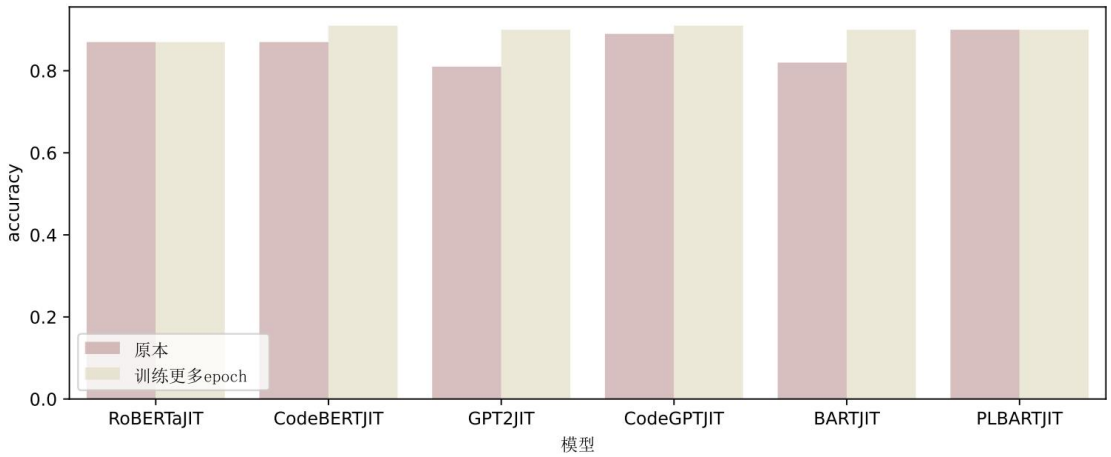


**图 7　模型在 qt 数据集中 accuracy 随训练时长变化**

### 3.4.2 不同模型的训练效果与预测结果分布之间的关系探究

本研究内容考察了以不同预训练模型为骨架的 JIT 缺陷预测模型的整体训练效率，以及不同模型预测结果的分布之间是否存在关联。本小节对比实验过程中需要的训练时间、参数大小和训练损失的变化，并对实验预测结果进行 t 检验来分析 6 个模型之间的分布相似度。

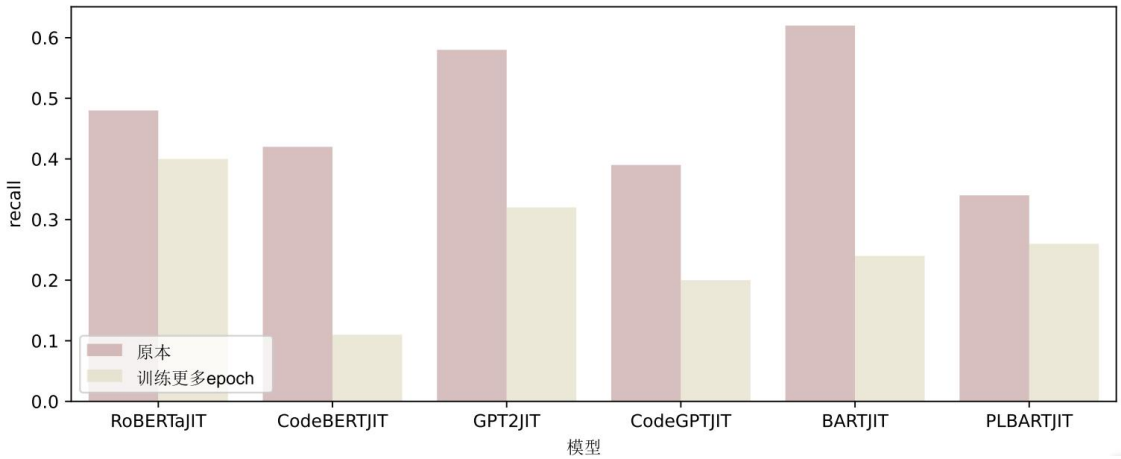


**图 8 模型在 qt 数据集中 recall 随训练时长变化**

本研究首先比较了不同 JIT 缺陷预测模型在训练时间、参数大小和训练损失的差异。表 5 分别给出了 RoBERTaJIT、CodeBERTJIT、GPT2JIT、CodeGPTJIT、BARTJIT 和 PLBARTJIT 模型的训练时间，可以发现，在一般情况下，不同模型的训练时间和模型大小相差不大。在两个数据集的相同训练设置下，BARTJIT 和 PLBART 比其他模型更快一些。此外，具有相似主干模型的 JIT 缺陷预测模型在训练时间上具有更相关的可比性。例如，在训练时长上 RoBERTaJIT 接近 CodeBERTJIT， GPT2JIT 接近 CodeGPTJIT，而 BARTJIT 接近 PLBARTJIT。

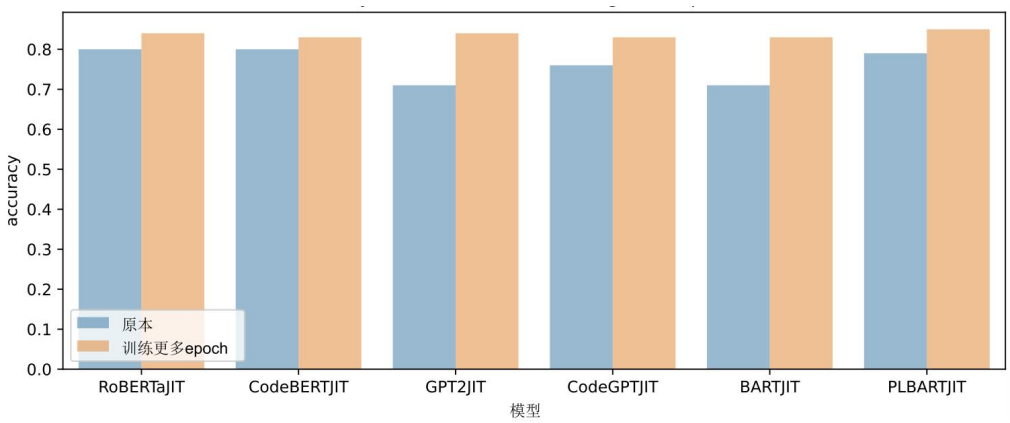


**图 9 模型在 openstack 数据集中 accuracy 随训练时长变化**

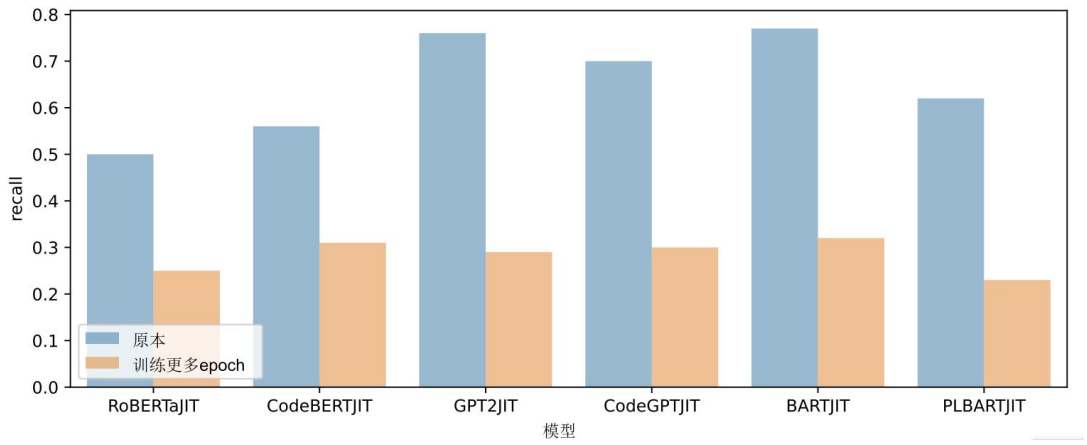


**图 10　模型在 openstack 数据集中 recall 随训练时长变化**

图 11 和图 12 分别显示了这六种模型在 qt 和 openstack 项目中的训练损失变化。就收敛效果而言，RoBERTaJIT、GPT2JIT 和 BARTJIT 略低于 CodeBERTJIT、CodeGPTJIT 和 PLBARTJIT。这表明代码相关的预训练模型在 JIT 缺陷预测任务中提供了更好的初始表示。

**表 5　模型的训练耗时**

| 模型 | QT | Openstack |
|---|---|---|
| RoBERTaJIT | 1938s | 1005s |
| CodeBERTJIT | 1942s | 1007s |
| GPT2JIT | 2184s | 1132s |
| CodeGPTJIT | 2150s | 1114s |
| BARTJIT | 1460s | 757s |
| PLBARTJIT | 1489s | 808s |

•**结论 1:** 具有相似主干模型的不同 JIT 缺陷预测模型的训练时间趋于接近，与代码相关的主干模型比与其对应的非代码相关模型收敛性更好。

在上面的探究中，本研究探讨了不同模型在训练效率方面的关系，但这些模型的预测结果是否具有相同的分布尚不清楚。为了进一步搞清楚这一点，本研究在六个模型的预测结果之间进行了 t 检验实验。表 6 给出了六个模型预测结果的检验结果，左边的数字表示两个模型之间的 t 值，右边的数字表示对应的 p 值，本研究将结果保留到小数点后两位。符号⊘ 表示 t 值和 p 值计算中省略的重复结果。根据 T 实验结果表明，尽管在构建 JIT 缺陷预测模型时，一些架构用到了相似的骨干模型如 RoBERTa 和 CodeBERT，BART 和 PLBART，GPT2 和 CodeGPT，但这些 JIT 缺陷预测模型在统

计意义上仍然产生不同的预测结果分布。

**表 6 模型预测分布 T 检验结果**

| 模型 | RoBERTaJIT | CodeBERTJIT | GPT2JIT | CodeGPTJIT | BARTJIT | PLBARTJIT |
|---|---|---|---|---|---|---|
| RoBERTaJIT | ⊘ | 18.41/0.0 | -8.73/0.0 | 6.48/0.0 | -11.95/0.0 | 14.88/0.0 |
| CodeBERTJIT | ⊘ | ⊘ | -29.21/0.0 | -13.22/0.0 | -33.01/0.0 | -3.21/0.001 |
| GPT2JIT | ⊘ | ⊘ | ⊘ | 16.38/0.0 | -3.3/0.001 | 25.01/0.0 |
| CodeGPTJIT | ⊘ | ⊘ | ⊘ | ⊘ | -19.98/0.0 | 9.45/0.0 |
| BARTJIT | ⊘ | ⊘ | ⊘ | ⊘ | ⊘ | 27.6/0.0 |
| PLBARTJIT | ⊘ | ⊘ | ⊘ | ⊘ | ⊘ | ⊘ |

**•结论 2:** 尽管一些骨干预训练模型具有相似性，但 JIT 缺陷预测任务的结果分布存在显著差异。

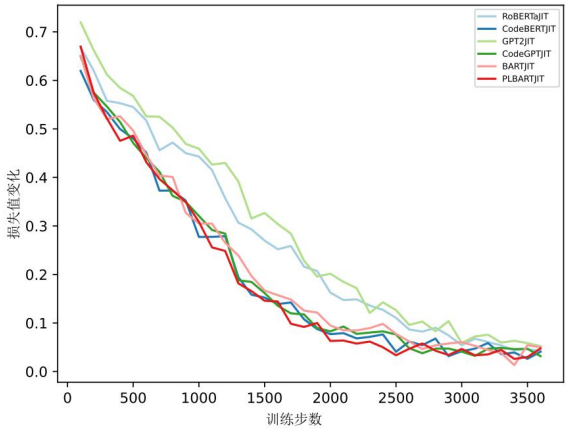


**图 11 模型在 qt 数据集中 loss 随训练步长变化情况**

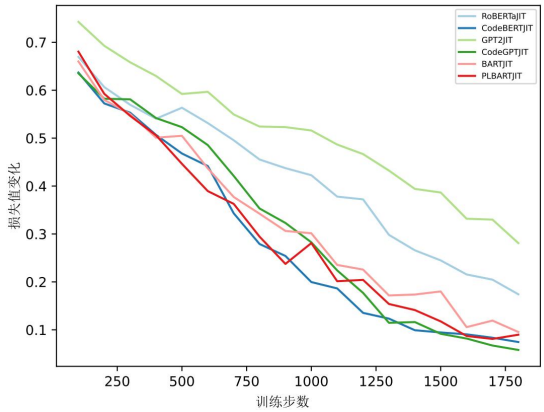


**图 12 模型在 openstack 数据集中 loss 随训练步长变化情况**

### 3.4.3 不同模型对输入数据的敏感性探究

这个研究问题的目标是研究在 JIT 缺陷预测模型中，commit code 以及 commit message 作为模型输入如何影响缺陷检测的性能。为此，本小节进行了消融实验，在预测时对输入分别做消融 commit code 以及 commit message 操作，并分析对应消融操作对模型结果召回率和精度的影响，这两个评估指标是缺陷检测和误报的最直观表示。

图 13 和图 14 展示了六个 JIT 缺陷预测模型在消融设置下的实验结果。具体而言，“-msg”操作指的是仅使用 commit code 作为模型输入，而“-code”操作指的是仅使用 commit message 作为模型输入。消融实验表明，在消除 commit code 后，所有模型的召回率都大幅下降。

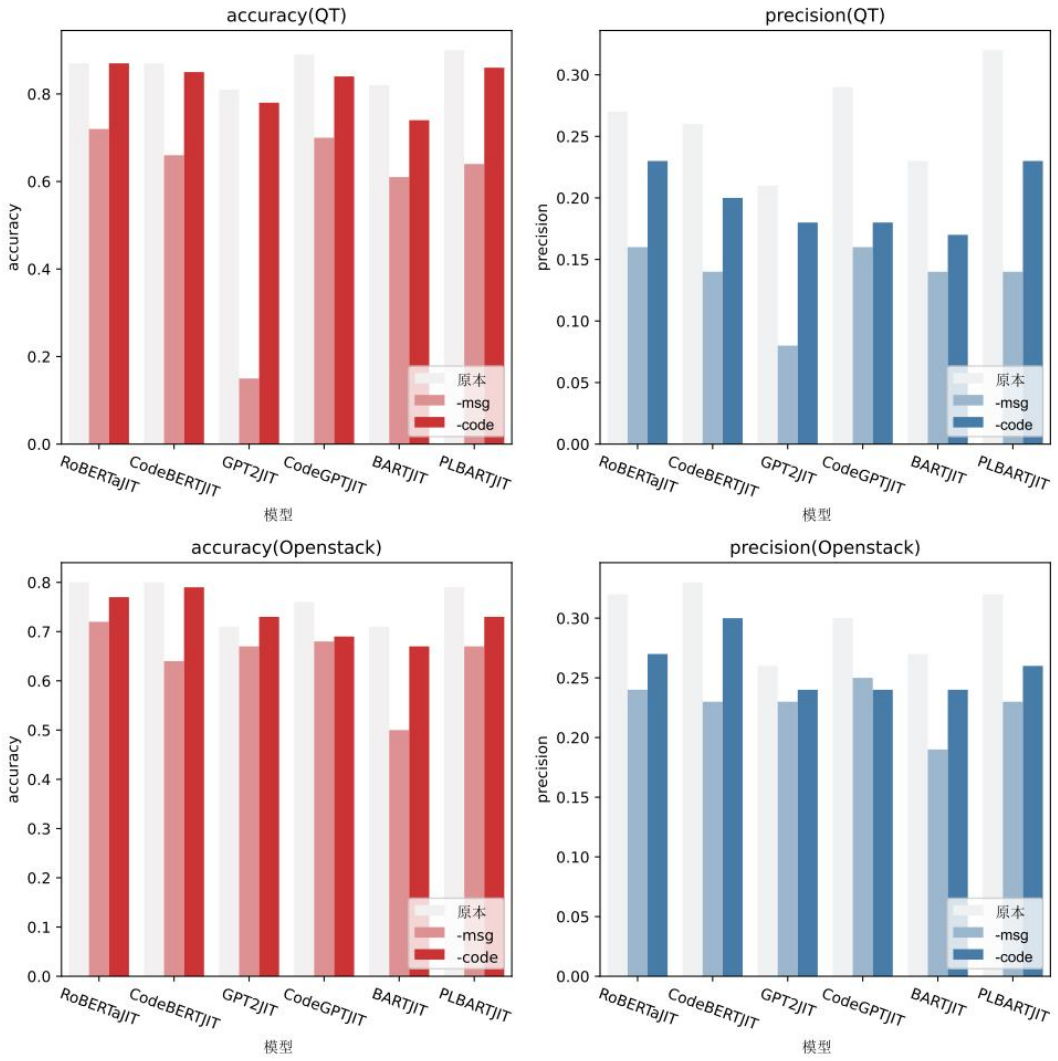


**图 13 两个数据集上进行消融实验的 accuracy 以及 precision 结果**

召回率意味着模型正确预测了一个有缺陷的提交。在 qt 和 openstack 两个数据集中，本研究对所有六个 JIT 缺陷预测模型进行“-msg”操作时，大多数模型在结果上显

示出显著的召回率提升。这表明从输入数据中移除提交信息有助于模型更准确地识别更多的缺陷。这可能与提交信息完全由开发人员提供，并且没有严格规范，可能会引入与给定提交的代码描述不完全匹配的信息有关。

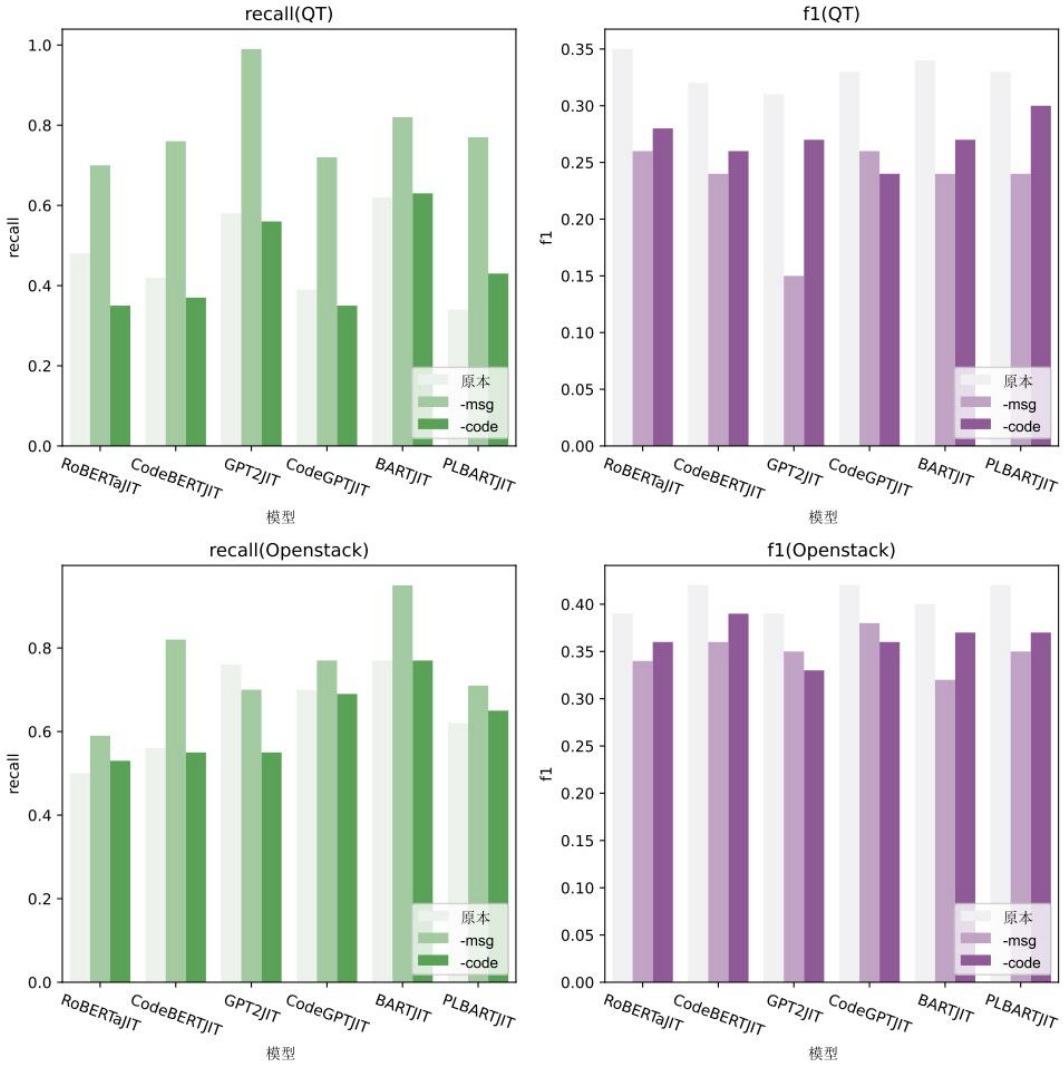


**图 14 在两个数据集上进行消融实验的 recall 以及 f1 score 结果**

另一方面，当本研究对这六个模型进行“-code”操作时，超过一半的模型在实验结果上显示召回率下降，而其他评估指标与基础模型结果只有略微差异。这进一步支持了 commit code 在缺陷检测中起着重要作用的发现。除了主要结果外，本研究在消融实验中还观察到了一个有趣的现象。无论本研究移除了 commit code 还是 commit message，其他三个评估指标（accuracy、precision 和 f1 score）均显示下降。这表明模型输入的两个部分即 commit code 和 commit message，在决定这三个评估指标的整体性能中起着关键作用。这些发现为每个输入数据组件的重要性提供了有价值的见解。

**结论：**提交代码在缺陷检测中起主要作用，提交信息和提交代码共同负责预测

accuracy、precision 和 f1 score。

### 3.4.4 不同模型在 few-shot 场景下的具体表现

这个研究问题的目标是探究在少样本平衡数据集场景下，基于不同预训练模型的 JIT 缺陷预测模型的性能。在真实的软件工程开发场景下，有缺陷的提交数目是相对较少的，因而为了保证数据平衡性，本研究对两个数据集进行了下采样，并设置了不同的训练数据规模（0、10、100、500、1000 和 2000）来探索模型的性能。

图 15 和图 16 展示了这六个模型在少样本场景下的实验结果。在 zero-shot 场景中，由于零除问题部分数据集无法计算 f1 score，因此本研究重点比较了模型在四个评估指标下的性能：auc score、accuracy、precision 和 recall。正如图 15 和图 16 所示，在 zero-shot 场景中，这六个模型的性能几乎异常，召回率和准确率都表现为极端的 0 和 1。这表明这些模型在零样本场景中不可用。然而，随着数据集规模的逐渐增加，这六个模型的性能表现变得更加一致并显著提高。

**表 7　模型 2000-shot 场景下在 qt 数据集中的实验结果**

| 模型 | auc score | accuracy | precision | recall |
|---|---|---|---|---|
| RoBERTaJIT | 0.81 | 0.87 | 0.27 | 0.48 |
| 2000-shot | 0.80 | 0.61 | 0.14 | 0.82 |
| CodeBERTJIT | 0.81 | 0.87 | 0.26 | 0.42 |
| 2000-shot | 0.79 | 0.74 | 0.17 | 0.65 |
| GPT2JIT | 0.81 | 0.81 | 0.21 | 0.58 |
| 2000-shot | 0.80 | 0.75 | 0.18 | 0.69 |
| CodeGPTJIT | 0.81 | 0.89 | 0.29 | 0.39 |
| 2000-shot | 0.81 | 0.73 | 0.18 | 0.75 |
| BARTJIT | 0.82 | 0.82 | 0.23 | 0.62 |
| 2000-shot | 0.79 | 0.63 | 0.14 | 0.81 |
| PLBARTJIT | 0.82 | 0.90 | 0.32 | 0.34 |
| 2000-shot | 0.80 | 0.58 | 0.13 | 0.86 |

**结论 1：**随着数据集规模的逐渐增加，这六个模型的性能趋于一致和正常。

本研究观察到，与基准模型相比，一些模型在评估分数上较低。然而，值得注意的是，尽管分数较低，CodeGPTJIT 和 GPT2JIT 在少样本场景下仍然优于两个最先进的模型 DeepJIT 和 CC2Vec。这从表 7 和表 8 中 CodeGPTJIT 和 GPT2JIT 在 QT 和 Openstack 数据集上的结果中可以看出。CodeGPTJIT 和 GPT2JIT 分别在 QT 和 Openstack 数据集上表现出最佳性能，表明它们在处理少样本场景时的有效性。

**结论 2：**在合适规模的平衡数据集上，基于预训练模型的少样本场景可以提升缺陷检测能力。

表 9 和表 10 分别展示了 CodeGPTJIT 以及 GPT2JIT 模型使用 2000 个训练样本的结果。从结果中可以观察到，当数据集规模缩小到原始数据集的一部分（QT 数据集的 0.07 倍和 Openstack 数据集的 0.15 倍）时，虽然模型之间的 auc 分数差异变得不明显，范围在 0 到 0.04 之间，但在准确率和召回率上存在显著差异。在较小数据集上训练模型时，准确率的显著下降可以归因于模型的收敛性。

**表 8　模型 2000-shot 场景下在 openstack 数据集中的实验结果**

| 模型 | auc score | accuracy | precision | recall |
|---|---|---|---|---|
| RoBERTaJIT | 0.79 | 0.80 | 0.32 | 0.50 |
| 2000-shot | 0.79 | 0.75 | 0.27 | 0.62 |
| CodeBERTJIT | 0.80 | 0.80 | 0.33 | 0.56 |
| 2000-shot | 0.78 | 0.53 | 0.19 | 0.83 |
| GPT2JIT | 0.79 | 0.71 | 0.26 | 0.76 |
| 2000-shot | 0.76 | 0.65 | 0.23 | 0.78 |
| CodeGPTJIT | 0.81 | 0.76 | 0.30 | 0.70 |
| 2000-shot | 0.77 | 0.71 | 0.25 | 0.69 |
| BARTJIT | 0.81 | 0.71 | 0.27 | 0.77 |
| 2000-shot | 0.78 | 0.74 | 0.27 | 0.69 |
| PLBARTJIT | 0.81 | 0.79 | 0.32 | 0.62 |
| 2000-shot | 0.77 | 0.66 | 0.23 | 0.74 |

使用相同的训练周期在较小的数据集上训练，收敛性相比于在较大数据集上训练会变得不太稳定。然而，尽管较小数据集带来了挑战，本研究发现在如此有限的数据

上训练模型，仍然可以显著提高召回率。在 QT 数据集上，召回率平均提高了 36%，在 Openstack 数据集上提高了 14%。这一发现表明，当使用平衡数据集时，基于预训练模型的 JIT 缺陷预测模型在少样本场景下仍能有效检测缺陷。

**表 9　CodeGPTJIT 在 qt 数据集 2000-shot 场景下超越其他两个基线模型**

| 模型 | auc score | accuracy | precision | recall | f1 score |
|---|---|---|---|---|---|
| DeepJIT | 0.79 | 0.73 | 0.17 | 0.70 | 0.27 |
| CC2Vec | 0.81 | 0.56 | 0.13 | 0.87 | 0.23 |
| CodeGPTJIT (2000-shot) | 0.81 | 0.73 | 0.18 | 0.75 | 0.29 |

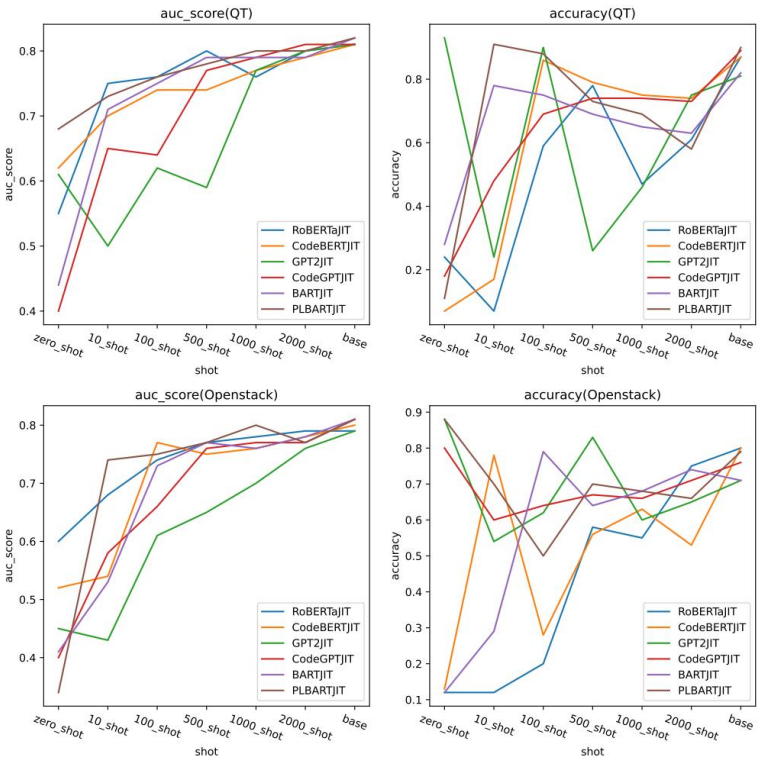


**图 15　不同 shot 场景下 qt 以及 openstack 数据集中的 auc 和 accuracy**

## 3.5 本章小节

本研究已经对针对不同 Transformer 结构预训练模型在即时缺陷预测模型上表现做出调研和实验，实验成果 A study on the impact of pre-trained model on Just-In-Time defect prediction 已被 QRS23（23rd IEEE International Conference on Software Quality，Reliability，and Security）接收。

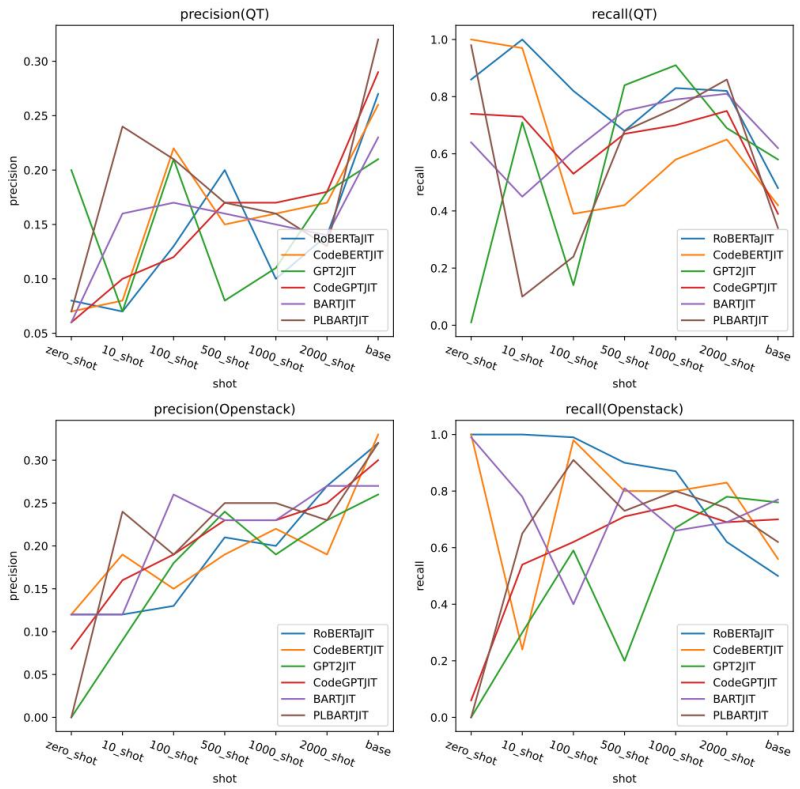


**图 16 不同 shot 场景下 qt 以及 openstack 数据集中的 precision 和 recall**

**表 10 GPT2JIT 在 openstack 数据集 2000-shot 场景下超越其他两个基线模型**

| 模型 | auc score | accuracy | precision | recall | f1 score |
|---|---|---|---|---|---|
| DeepJIT | 0.74 | 0.61 | 0.17 | 0.76 | 0.28 |
| CC2Vec | 0.76 | 0.27 | 0.11 | 0.98 | 0.19 |
| GPT2JIT | 0.76 | 0.65 | 0.23 | 0.78 | 0.36 |

这项工作的贡献是多方面的。首先，本研究不是仅仅关注单个模型的性能，而是深入研究了基于不同主干的这些模型的比较分析。

其次，本研究分析了六种模型中训练效率与模型分布的相关性，发现当骨干的预训练模型相似时，需要消耗的训练资源更接近。

第三，本研究进行了广泛的消融实验，以深入了解每个模型对各种输入的敏感性，并发现提交代码在基于提交代码和提交消息的缺陷检测中起着重要作用，这为后续任务优化提供了思路即研究人员可以专注于对提交代码的处理。

最后，本研究探讨了不同模型在 zero-shot 和 few-shot 场景下的性能。论文实验结果显示，在平衡数据集的情况下，预训练模型显示出优越的缺陷检测能力，并且该研

究获得了两个在 2000 个样本场景下优于 DeepJIT 和 CC2Vec 的模型的表现，这为在 few-shot 场景下优化缺陷预测模型提供了可能。

本研究的工作主要集中在基于不同结构预训练模型的 JIT 缺陷预测任务上。实验分析结果表明，基于不同骨干的模型构建缺陷预测模型都能得到一定的性能改进。具有相似主干的模型显示在预测分布上呈现出更接近的关系。提交代码在缺陷检测中发挥了重要作用，以及平衡的数据集即使在 few-shot 场景下，相较于基线模型依旧有更优的缺陷预测能力。这些结果为基于预训练模型优化 JIT 缺陷预测任务提供了新的见解，并强调了在构建此类模型时需要注意的因素。

# 第四章 预训练微调策略对缺陷预测任务的实证研究

为了综合评价影响基于 encoder 和 decoder 两种架构模型的 JIT 缺陷预测模型性能的微调因素，本章选择 CodeBERT 和 CodeGPT 两个典型架构的预训练模型，并基于第二章的提出的 JIT 通用框架进行实证探究。研究主要围绕以下不同调优策略展开：参数冻结和初始化设置，特征提取器影响，优化器策略，并结合 LoRA 提出一种无损性能的轻量级调优策略，并在近 10 万条公开数据集上进行实验验证。本章首先会介绍实验的目的和内容，随后介绍实施实验的环境和设计，研究内容的具体实现，最后给出实验的结果和分析。

## 4.1 实证研究目的与内容

本章使用 commit message 和 commit code 作为模型输入，并在近 10 万个扩展缺陷预测数据集上评估这两个模型的性能，比较它们在不同微调设置下的性能。实验方法如图 17 所示，实验评价指标主要为 auc 以及 f1 score。

1）参数冻结和初始化如何影响模型性能

以前基于预训练模型的缺陷预测工作倾向于进行全参数微调操作。局部参数微调对缺陷预测性能的影响尚不清楚。本文研究了两种不同的部分参数微调范式：冻结模型参数更新和在下游任务微调之前初始化参数。

2）模型对不同的特征提取器敏感性探究

在这个研究问题中，本文的目标是研究不同的特征提取器对两种架构的 JIT 缺陷预测模型的影响。研究中选择了四种不同的特征提取器：FCN（全连接网络）、LSTM、Transformer 和 CNN 以进行实验探究。

3）优化器策略对模型的具体影响

这个研究问题的目的是探索不同的超参数设置在 Adam 优化器的有效性，通过对 Adam 设置不同的权重衰减率来探究模型的性能差异。

4）基于之前的发现，为模型找到一个经济有效的微调配置探究

这个研究问题的目标是探索是否存在一种方法更有效地微调预训练模型并无损模型性能。

## 4.2 实证研究方法概述

微调基于第三章提出的通用模型的策略选择可以分为三个部分：嵌入模块设置、提取器模块设置和优化器设置。图 17 详细说明了在 CodeGPT 以及 CodeBERT 作为骨干模型的 JIT 缺陷预测模型微调过程中可以调整的具体策略。

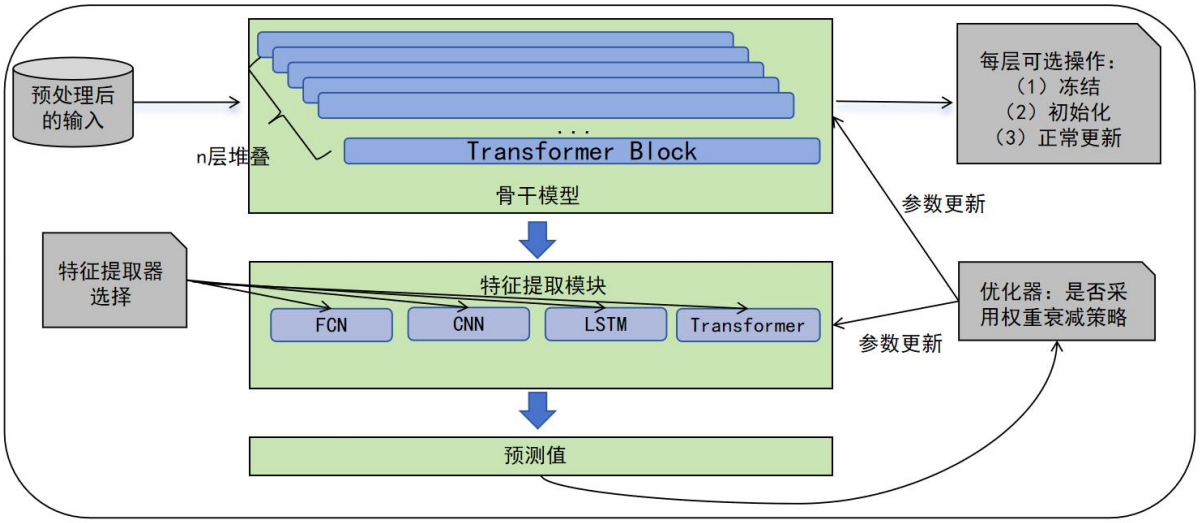


**图 17　JIT 模型框架及微调策略**

嵌入模块即预训练模型，是由堆叠的 Transformer 编码器层组成，在这一模块中可以执行三种不同的参数更新操作。这些操作包括：在微调前初始化对应的权重、在微调过程中冻结参数更新以及更新权重。在特征提取器模块中本研究选择了四种常用的特征提取网络进行探究：FCN（全连接网络）、CNN（在原来的 CodeGPTJIT 和 CodeBERTJIT 模型中使用）、LSTM 和 Transformer。在优化器模块中，Adam 优化器可选择的策略共有两种：普通 Adam 和带有权重衰减策略的 Adam。这个策略选择框架试图对基于不同结构的 JIT 缺陷预测模型性能影响因素进行全面的探索。

## 4.3 实验设置与数据

为了便于评估，本研究借用了 Zeng 等人的研究数据集，该数据集从 GitHub 上 6 个流行的开源项目（即 qt、openstack、jdt、platform、gerrit 和 go）中收集，并借助 SZZ 算法进行人工分析和标注。本研究选择了除 jdt 项目外其他项目过去三年的数据，由于 jdt 项目近三年的数据量太少，因而对于 jdt 项目选择了更长的数据周期收集的数据。用于评估的数据集详细统计信息如表 11 所示，共包含 99412 个标注提交。其中，26，

693 个是与缺陷相关的补丁，其余的是非缺陷提交。本研究将每个项目的数据按 80:20 的比例进行划分，80%的数据用于训练集，剩余 20%的数据用于测试。

**表 11　6 个开源项目数据统计信息描述**

| 项目 | 提交数目 | 缺陷数目 | 缺陷占比（%） |
|---|---|---|---|
| qt | 23912 | 3582 | 15.0 |
| openstack | 22757 | 5927 | 26.0 |
| jdt | 7773 | 3202 | 41.2 |
| platform | 11034 | 4056 | 36.8 |
| gerrit | 14927 | 1781 | 11.9 |
| go | 19009 | 8145 | 42.8 |
| 总提交数 | 99412 | 26693 | 26.9 |

本研究使用一台台式电脑作为实验环境。电脑运行 Ubuntu 18.04，配备了 24GB 内存的 RTX3090 显卡。为了减少随机性的影响，所有实验重复五次以获得平均值。在本研究中，所有研究问题中训练超参如表 12 所示，其中每组模型的训练次数为 3 轮，学习率设置为 1e-5，batch size 大小设置为 16，每次提交的 commit code 包含四个修改文件。在研究问题 1 到研究问题 3 中使用全参数微调，仅在研究问题 4 中使用 LoRA，LoRA 的 rank 设置为 8，且 CodeBERT 的第一层编码器参数不冻结。

**表 12　训练超参设置**

| 项目 | 轮次 | 学习率 | 优化器 | 批次大小 |
|---|---|---|---|---|
| qt | 1 | 1e-5 | Adam | 16 |
| openstack | 1 | 1e-5 | Adam | 16 |
| jdt | 1 | 1e-5 | Adam | 16 |
| platform | 1 | 1e-5 | Adam | 16 |
| gerrit | 1 | 1e-5 | Adam | 16 |
| go | 1 | 1e-5 | Adam | 16 |

## 4.4 实证研究实验结果与分析

### 4.4.1　参数冻结和初始化对模型性能影响探究

以前基于预训练模型的缺陷预测工作倾向于进行全参数量微调操作。局部参数微

调对缺陷预测性能的影响仍旧是未知的。本研究问题研究了两种不同的部分参数微调范式：冻结模型参数更新以及在微调之前初始化模型参数。

本节参数冻结研究在六个项目上训练和评估 CodeGPTJIT 和 CodeBERTJIT 模型性能。在冻结设置中，本研究冻结 CodeGPT 和 CodeBERT 的前 n 层编码器，其中 n 从 0 变化到 6（CodeGPT 和 CodeBERT 模型各自有 12 个 Transformer Block），为了表示的简洁，本研究在表 13 和表 14 中使用 fi 作为冻结 i 层的缩写，在图 18 和图 19 中使用 freeze_i 作为冻结第 i 层的缩写。

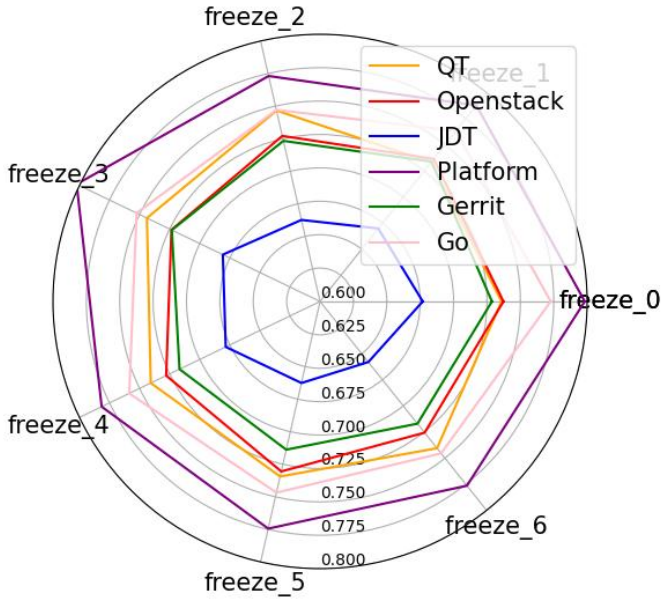


**图 18　六个数据集在 CodeGPTJIT 中冻结前 n 层解码器参数的 auc score 变化**

如表 13 和表 14 和图 18 和图 19 所示，在第一个编码器/解码器层的参数后，CodeGPTJIT 和 CodeBERTJIT 模型在所有六个项目上的 auc 评分和 f1 评分均出现下降。有趣的是，当冻结前 n 层参数时，n 大于 1，auc 评分的表现与仅冻结第一层相比没有显著差异。这种现象可以归因于模型训练过程中前向过程中的错误积累。模型的初始层倾向于学习与下游任务相关的一般知识。因此，当基于预训练模型进行微调时，更新初始层的参数可以使模型更好地掌握与特定任务相关的信息。

**结论 1：**基于 encoder 和 decoder 的两种架构模型种，第一层编码器及解码器层均在下游任务中起着最重要的作用

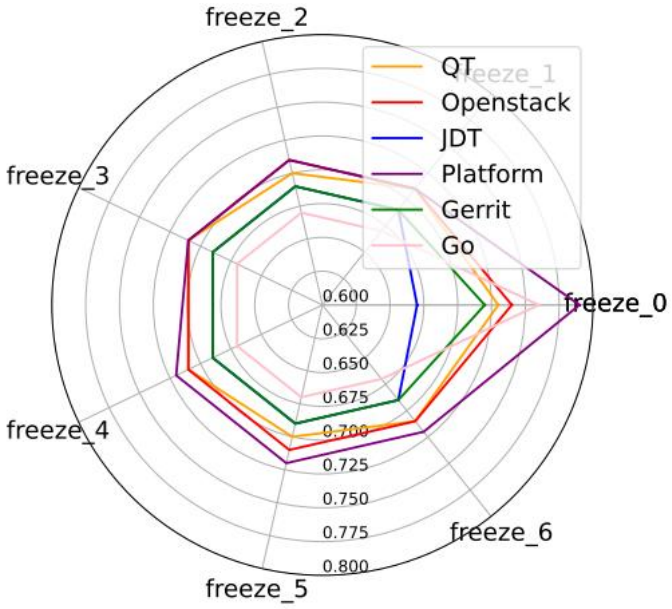


**图 19 CodeBERTJIT 中冻结前 n 层编码器参数的 auc score 变化**

在另一种模型参数配置中，本研究初始化预训练模型（CodeGPT 和 CodeBERT）的前 n 层，n 从 0 逐步累加到 6。本研究使用参数为 N（0, 0.02^2）的正态分布初始化模型权重，这种策略与后归一化设计相关，其中较小的标准差更有利于优化。考虑到不同编码器层的重新初始化，CodeGPTJIT 和 CodeBERTJIT 的实验结果如图 20 和图 21 所示。

**表 13 CodeGPTJIT 中冻结前 n 层 decoder block 参数的 f1 score**

| 项目 | f0 | f1 | f2 | f3 | f4 | f5 | f6 |
|---|---|---|---|---|---|---|---|
| qt | 0.38 | 0.38 | 0.39 | 0.38 | 0.35 | 0.37 | 0.37 |
| openstack | 0.44 | 0.44 | 0.43 | 0.43 | 0.43 | 0.43 | 0.42 |
| jdt | 0.66 | 0.62 | 0.63 | 0.65 | 0.63 | 0.60 | 0.60 |
| platform | 0.63 | 0.61 | 0.60 | 0.62 | 0.59 | 0.60 | 0.61 |
| gerrit | 0.25 | 0.22 | 0.23 | 0.22 | 0.24 | 0.22 | 0.23 |
| go | 0.64 | 0.63 | 0.63 | 0.63 | 0.64 | 0.61 | 0.63 |
| **平均** | 0.50 | 0.48↓ | 0.49↓ | 0.49↓ | 0.48↓ | 0.47↓ | 0.48↓ |

由于 CodeGPT 与 CodeBERT 具备不同的架构，在其他设置保持一致的情况下，使用相同的参数初始化策略会导致模型性能在总体上产生不同的波动变化。与冻结第一层参数性能下降不同，参数初始化策略在 CodeBERT 架构中会轻微提升模型的平均表现，在某些项目上表现出轻微的改进。而对于 CodeGPT 来说，参数初始化会导致模型平均性能的降低。在图 20 中，当重新初始化第一个编码器层时，本研究观察到这些项

目的平均 auc 和 f1 值在不进行参数初始化时为最高。

**表 14　CodeBERTJIT 中冻结前 n 层 encoder block 参数的 f1 score**

| 项目 | f0 | f1 | f2 | f3 | f4 | f5 | f6 |
|---|---|---|---|---|---|---|---|
| qt | 0.36 | 0.35 | 0.35 | 0.35 | 0.35 | 0.35 | 0.35 |
| openstack | 0.44 | 0.41 | 0.41 | 0.41 | 0.41 | 0.41 | 0.41 |
| jdt | 0.64 | 0.65 | 0.64 | 0.63 | 0.64 | 0.64 | 0.64 |
| platform | 0.61 | 0.58 | 0.58 | 0.58 | 0.58 | 0.58 | 0.58 |
| gerrit | 0.21 | 0.2 | 0.2 | 0.2 | 0.19 | 0.2 | 0.2 |
| go | 0.61 | 0.56 | 0.56 | 0.56 | 0.56 | 0.56 | 0.56 |
| 平均 | 0.48 | 0.46↓ | 0.46↓ | 0.46↓ | 0.46↓ | 0.46↓ | 0.46↓ |

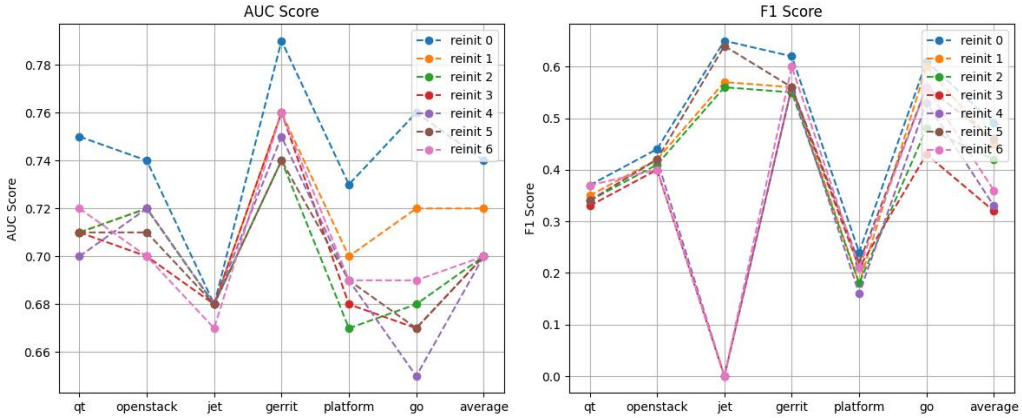


**图 20　CodeGPTJIT 中初始化前 n 层编码器参数的 auc score 以及 f1 score**

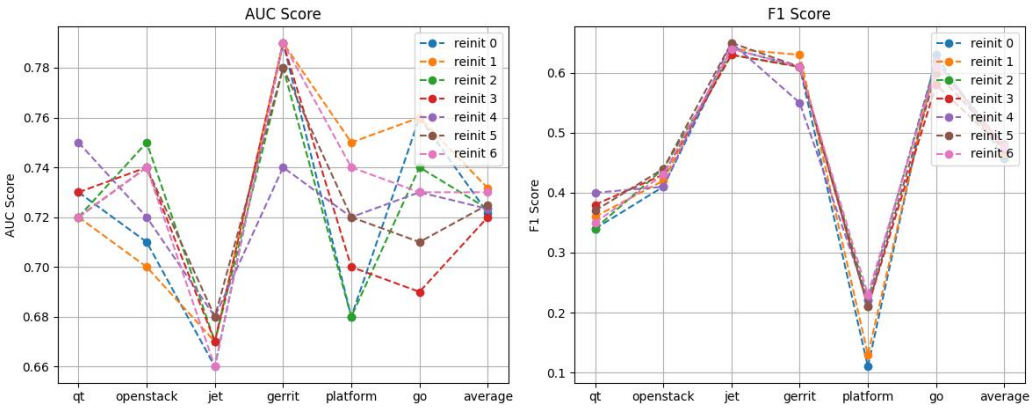


**图 21　CodeBERTJIT 中初始化前 n 层编码器参数的 auc score 以及 f1 score**

分析图 21 中的 CodeBERTJIT 结果，可以明显看出，当重新初始化前 4 个编码器层时，qt 项目的 auc 和 f1 值都增加。此外，当重新初始化前 2 个编码器层时，qt 项目的性能也有所提高。模型参数的初始化不一定会导致模型性能下降，这与 Zhang 等人对 BERT 研究中的结论一致。参数初始化的最佳值 n 因项目和数据集不同而异，这

为在微调下游任务时调整不同参数初始化策略的必要性提供了见解。

**结论 2：**为不同的开源项目定制的合适初始化策略有机会提高 JIT 缺陷预测模型的性能。

### 4.4.2 模型对不同特征提取器的敏感性探究

在这个研究问题中，本研究旨在调查不同特征提取器对 JIT 缺陷预测模型的影响。现有模型采用了不同的特征提取器。例如，DeepJIT 使用 CNN 作为特征提取器，CC2Vec 采用 LSTM，LApredict 使用逻辑回归（使用全连接网络实现）。此外，Transformer 作为特征提取的流行也被纳入考虑范围。因此，本研究选择了四种不同的特征提取器：全连接网络（FCN）、LSTM、Transformer 和 CNN。CodeGPTJIT 和 CodeBERTJIT 使用不同特征提取器的实验结果如表 15 和 16 所示。

**表 15 CodeGPTJIT 中使用不同特征提取器的 auc score 以及 f1 score**

| 项目 | FCN | LSTM | Transformer | CNN |
|---|---|---|---|---|
| **auc score** | | | | |
| qt | 0.74 | 0.73 | 0.75 | 0.75 |
| openstack | 0.73 | 0.74 | 0.73 | 0.73 |
| jdt | 0.69 | 0.68 | 0.69 | 0.68 |
| platform | 0.80 | 0.77 | 0.77 | 0.77 |
| gerrit | 0.73 | 0.72 | 0.69 | 0.72 |
| go | 0.76 | 0.76 | 0.75 | 0.77 |
| 平均 | **0.74** | 0.73 | 0.73 | **0.74** |
| **f1 score** | | | | |
| qt | 0.39 | 0.36 | 0.38 | 0.39 |
| openstack | 0.44 | 0.45 | 0.44 | 0.42 |
| jdt | 0.63 | 0.64 | 0.66 | 0.64 |
| platform | 0.64 | 0.62 | 0.60 | 0.60 |
| gerrit | 0.23 | 0.23 | 0.17 | 0.23 |
| go | 0.62 | 0.63 | 0.64 | 0.65 |
| 平均 | **0.49** | **0.49** | 0.48 | **0.49** |

本研究对使用不同特征提取器的 CodeGPTJIT 和 CodeBERTJIT 在六个项目中的 auc 评分和 f1 评分进行了比较。CodeGPTJIT 的平均 auc 评分从 0.73 到 0.74，CodeBERTJIT 的平均 auc 评分从 0.71 到 0.75，而 CodeGPTJIT 的平均 f1 评分从 0.48 到 0.49，CodeBERTJIT 的平均 f1 评分从 0.45 到 0.49。值得注意的是，两个模型在使

用 Transformer 时均获得了最差的表现，而使用 FCN 或者 CNN 时获得最佳表现。

**表 16　CodeBERTIT 中使用不同特征提取器的 auc score 以及 f1 score**

| 项目 | FCN | LSTM | Transformer | CNN |
|---|---|---|---|---|
| **auc score** | | | | |
| qt | 0.73 | 0.73 | 0.71 | 0.74 |
| openstack | 0.75 | 0.75 | 0.73 | 0.73 |
| jdt | 0.68 | 0.66 | 0.65 | 0.68 |
| platform | 0.82 | 0.79 | 0.78 | 0.80 |
| gerrit | 0.75 | 0.72 | 0.67 | 0.70 |
| go | 0.76 | 0.75 | 0.72 | 0.76 |
| 平均 | **0.75** | 0.73 | 0.71 | 0.74 |
| **f1 score** | | | | |
| qt | 0.37 | 0.37 | 0.34 | 0.37 |
| openstack | 0.45 | 0.43 | 0.43 | 0.43 |
| jdt | 0.65 | 0.63 | 0.55 | 0.65 |
| platform | 0.61 | 0.62 | 0.60 | 0.62 |
| gerrit | 0.20 | 0.23 | 0.17 | 0.20 |
| go | 0.64 | 0.64 | 0.59 | 0.64 |
| 平均 | **0.49** | **0.49** | 0.45 | **0.49** |

**结论：**复杂的特征提取器会导致模型效果不佳。对于基于两种典型 encoder 及 decoder 架构的的缺陷预测模型，使用 FCN 或者 CNN 进行特征提取可以实现最佳性能。

### 4.4.3　优化器策略对模型的具体影响探究

这个研究问题的目的是探索不同的超参数设置在 Adam 优化器中的有效性。优化器的选择至关重要，它决定了模型权值的更新策略。CodeGPTJIT 和 CodeBERTJIT 的原始代码实现中选择的优化器是 Adam，而 CodeGPT 和 CodeBERT 中使用的优化器是 AdamW，它在 Adam 中增加了权重衰减参数。Adam 是一种使用动量和尺度的自适应学习率优化算法。优化器的设计是为了解决非平稳目标具有非常嘈杂和/或稀疏梯度的问题。

在训练过程中，权向量用$\theta$表示，学习率用$\partial$表示为时间步长 t 处损失函数的梯度。$\lambda$表示模型权重更新时的权重衰减值。Adam 中的$\lambda$等于 0， AdamW 中$\lambda$等于其他非零值。设置$\lambda \in \{0, 1e-5, 1e-4, 1e-3\}$在不同设置下对六个项目进行实验。

表 17 和表 18 分别展示了 CodeGPTJIT 和 CodeBERTJIT 两种模型在不同权重衰减率下的 auc score 和 f1 score 结果。

**表 17 CodeGPTJIT 中使用不同权重衰减率的 auc score 以及 f1 score**

| 权重衰减率 | qt | openstack | jdt | platform | gerrit | go | 平均 |
|---|---|---|---|---|---|---|---|
| **auc score** | | | | | | | |
| 0 | 0.75 | 0.74 | 0.69 | 0.77 | 0.72 | 0.76 | 0.74 |
| 1e-5 | 0.75 | 0.73 | 0.70 | 0.79 | 0.71 | 0.76 | 0.74 |
| 1e-4 | 0.75 | 0.74 | 0.69 | 0.78 | 0.72 | 0.76 | 0.74 |
| 1e-3 | 0.75 | 0.72 | 0.69 | 0.77 | 0.73 | 0.75 | 0.74 |
| **f1 score** | | | | | | | |
| 0 | 0.40 | 0.44 | 0.63 | 0.59 | 0.22 | 0.64 | 0.49 |
| 1e-5 | 0.39 | 0.43 | 0.65 | 0.62 | 0.22 | 0.63 | 0.49 |
| 1e-4 | 0.37 | 0.43 | 0.66 | 0.62 | 0.22 | 0.65 | 0.49 |
| 1e-3 | 0.38 | 0.43 | 0.65 | 0.60 | 0.23 | 0.63 | 0.49 |

当添加权重衰减策略时，在 CodeGPTJIT 模型中权重衰减率对于任务平均性能影响不大，但在 CodeBERTJIT 中当增加权重衰减策略时，模型在六个数据集中的平均性能没有降低或略有提高。一些其他研究人员已经探索过在少量样本微调中，权重衰减可以使微调更稳定，本研究的实验扩展到了 BERT 风格以及 GPT 风格的 JIT 缺陷预测模型，结果表明在全量微调场景下，权重衰减策略在 BERT 风格模型中较为有效。

**表 18 CodeBERTJIT 中使用不同权重衰减率的 auc score 以及 f1 score**

| 权重衰减率 | qt | openstack | jdt | platform | gerrit | go | 平均 |
|---|---|---|---|---|---|---|---|
| **auc score** | | | | | | | |
| 0 | 0.72 | 0.71 | 0.66 | 0.79 | 0.7 | 0.77 | 0.73 |
| 1e-5 | 0.74 | 0.72 | 0.67 | 0.78 | 0.7 | 0.76 | 0.73 |
| 1e-4 | 0.74 | 0.72 | 0.66 | 0.78 | 0.71 | 0.76 | 0.73 |
| 1e-3 | 0.72 | 0.75 | 0.65 | 0.79 | 0.69 | 0.75 | 0.73 |
| **f1 score** | | | | | | | |
| 0 | 0.26 | 0.39 | 0.61 | 0.61 | 0.18 | 0.62 | 0.45 |
| 1e-5 | 0.35 | 0.43 | 0.65 | 0.61 | 0.2 | 0.64 | **0.48** ↑ |
| 1e-4 | 0.38 | 0.43 | 0.62 | 0.61 | 0.22 | 0.59 | **0.48** ↑ |
| 1e-3 | 0.35 | 0.39 | 0.54 | 0.62 | 0.19 | 0.64 | 0.46 ↑ |

**结论：**AdamW 可以用来略微提高 CodeBERTJIT 模型的性能，且不会造成

CodeGPTJIT 模型的平均性能降低。

#### 4.4.4 综合微调策略的经济有效训练设置探究

这个研究问题的目标是探索是否存在一种更有效的方法来微调预训练模型并无损模型性能。在前三个研究问题中，本研究已经探讨了影响 JIT 缺陷预测任务中性能表现的一些因素。而 LoRA 提供了一种以较低的显存微调预训练模型的方式，通过实验发现，通过组合优化策略以及结合 LoRA 进行微调，可以找到一种显存和训练时间友好的且无损性能的微调方式。

**表 19　OCJITLoRA 相比于其他基线模型在六个数据集上的 auc score**

| 数据集 | LApredict | CodeBERTJIT | OCJITLoRA | DeepJIT | CC2Vec |
|---|---|---|---|---|---|
| qt | 0.74 | 0.73 | 0.76 | 0.70 | 0.69 |
| openstack | 0.74 | 0.74 | 0.74 | 0.72 | 0.73 |
| jdt | 0.68 | 0.67 | 0.68 | 0.67 | 0.68 |
| platform | 0.75 | 0.80 | 0.81 | 0.77 | 0.77 |
| gerrit | 0.75 | 0.72 | 0.72 | 0.70 | 0.70 |
| go | 0.68 | 0.76 | 0.76 | 0.69 | 0.69 |
| 平均 | 0.72 | 0.74 | 0.75 | 0.71 | 0.71 |

本研究根据之前研究的实验发现和结论定制了 LoRA 的微调策略，以寻找一种具有成本效益的微调方案。即不冻结第一个编码器层参数，并使用权重衰减策略更新 LoRA 权重，选择 FCN 作为特征提取器。表 19 和表 20 和分别展示了在 LoRA 基础上定制微调方法相比于其他两个基线模型 DeepJIT 和 CC2Vec 的结果。

**表 20　OCJITLoRA 相比于其他基线模型在六个数据集上的 f1 score**

| 数据集 | LApredict | CodeBERTJIT | OCJITLoRA | DeepJIT | CC2Vec |
|---|---|---|---|---|---|
| qt | 0 | 0.36 | 0.38 | 0.33 | 0.29 |
| openstack | 0.13 | 0.44 | 0.43 | 0.41 | 0.43 |
| jdt | 0.42 | 0.61 | 0.64 | 0.64 | 0.65 |
| platform | 0.09 | 0.61 | 0.60 | 0.58 | 0.56 |
| gerrit | 0.09 | 0.21 | 0.22 | 0.19 | 0.17 |
| go | 0.06 | 0.61 | 0.63 | 0.57 | 0.58 |
| 平均 | 0.13 | 0.47 | 0.48 | 0.45 | 0.43 |

如表 19 和表 20 所示，本研究将基于定制 LoRA 微调的 CodeBERT 模型（称为 OCJITLoRA）的结果与原始 CodeBERTJIT 和其他三种高效方法（DeepJIT、CC2Vec

和 LApredict）进行比较。本研究可以看到，OCJITLoRA 在平均结果上能够在这些模型中实现最佳性能。相比于 CodeBERTJIT，虽然 OCJITLoRA 的改进较小，但值得一提的是，训练 CodeBERTJIT 模型需要接近 21GB 显存，而 OCJITLoRA 仅需要接近 7GB 显存。这意味着显存消耗减少到 CodeBERTJIT 的三分之一，并实现了可比甚至更优的性能。

**结论：**通过结合一些优化因素和定制 LoRA，可以在较低资源消耗下提高基于 CodeBERT 模型构建的 JIT 缺陷预测模型性能。

## 4.5 本章小节

本章针对预训练模型在即时缺陷预测中调优因素的探究也进行了调研和实验，这项工作的贡献是多方面的。首先，本研究深入研究了两种不同架构 JIT 缺陷预测模型中不同调优因素的方案并对实验结果做出详尽分析。首先，本研究深入分析了六个项目中不同参数冻结和初始化策略下的性能差异，研究结果强调了第一编码器层对模型性能的关键作用，并证明适当调整权重初始化策略可以提高性能。冻结第一个编码器/解码器层时模型性能的快速下降表明，该层的参数对于适应任务特定信息至关重要。冻结这一层的参数会阻止模型适应特定的任务，因此在微调过程中更新第一层的参数是必要的。

其次，本研究在 CodeGPTJIT 和 CodeBERTJIT 中选择不同的特征提取器来探索它们对模型性能的影响，实验结果显示，在层次化结构数据中，使用富含位置信息和序列信息的网络结构会导致模型性能较差。

第三，本研究在 CodeGPTJIT 和 CodeBERTJIT 中选择不同的权重衰减率来探索它们对模型性能的影响，实验结果显示，AdamW 和适当的权重衰减参数可以轻微提升模型性能。

最后，本研究提出了一种基于 LoRA 方法的更有效的微调方法。该技术在降低显存消耗减少模型训练时间情况下略微提高了性能。

# 第五章　基于检索增强的即时缺陷预测技术优化

本文设计了基于检索增强的即时缺陷预测研究方案，并实现了基于信息堆叠和跨模型向量融合的两种检索增强架构。并通过对 Codellama-7B 模型进行裁剪构造向量编码器，基于 FAISS 实现 L2 distance 相似度计算和召回。为了验证本文方案的有效性和泛化性，本文设计并实施了一系列实验并对结果进行了详细的分析。本章首先会介绍实验的目的和内容，随后介绍实施实验的环境和设计，模型框架的具体实现，最后给出实验的结果和分析。

## 5.1 模型结构

常见的检索增强架构，仅通过向量相似度进行文本召回，从而进行上下文扩充，本章从两个角度出发，即基于数据堆叠和基于跨模型向量融合的方式，基于信息堆叠的检索增强模型检索召回的是和当前输入 commit code 相近的 n 个提交代码，即模型在构造输入时利用两部分不同的代码信息，本次提交以及检索召回的代码。区别于无检索模型架构，这种基于信息堆叠的检索增强仅在构造输入时多了检索召回的代码，并不涉及内部模型结构的变化。基于跨模型向量融合的检索增强，检索召回的是向量编码器构造的特征向量，该模型结构涉及两种不同特征的融合即检索回向量的加权运算。

### 5.1.1　无检索模型的构建以及向量表示的获取

本章无检索模型的构建采用如子研究基于不同预训练模型的即时缺陷预测模型框架所示。对于检索增强模型中向量表示的获取，目前广泛的方式是利用一个编码器模型直接获取输入的向量表示。比较常见的方式是利用小模型如 BERT 架构的模型做 encoder 向量提取器，但由于 BERT 模型本身存在长度限制，这种方式获取的向量在处理长文本时可能会导致信息受损，信息截断或者信息不完整的问题。

本研究基于 Codellama-7B 做模型裁剪，裁剪一个参数量约为 1.5B 的模型，Codellama 模型架构如图 22 所示，模型裁剪保留了 Codellama-7b 的 embed_token，lm_head，以及最后 norm layer，以及第一层和最后一层 decoder 层，构造了一个只包含两层解码器的模型 MinusCodellama-1.5B。

Codellama 是在 Llama2 的基础上通过位置编码内插实现长度的延拓，原本的 llama2 只支持 4096 的输入长度，而 Codellama 在继续训练的基础上天然可以处理 16384 的长度。因而在利用 MinusCodellama-1.5B 模型构造 commit code 向量表示时，本研究选择了最长为 6144（即 1024*6），可以基本覆盖数据集中的输入长度而不至于在构造向量时产生截断，同时由于 1.5B 的参数量，使得本研究在构造数据集向量表示时拥有比原始 Codellama-7B 模型更快的构建效率。

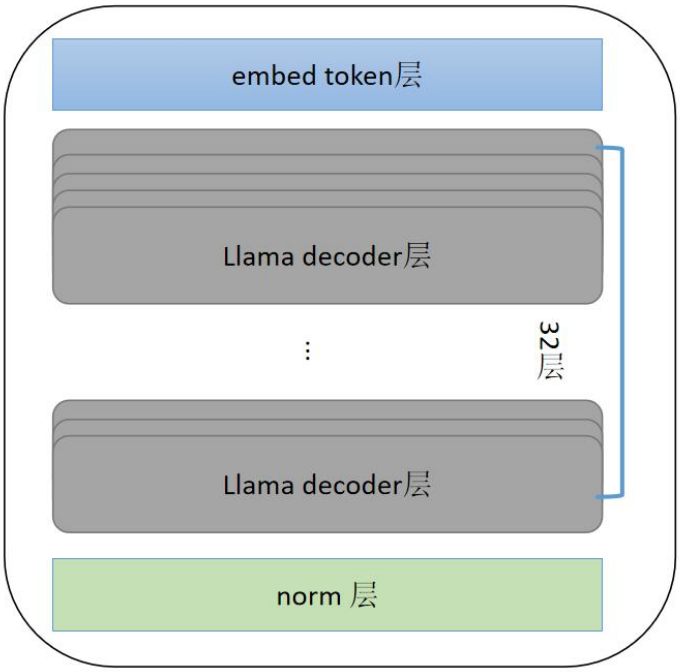


**图 22 CodeLlama 结构**

本研究处理 commit code 的具体方式为，对一次提交中提取四个修改文件代码，第 i 个文件中的代码文本作为输入，通过 MinusCodellama-1.5B 获得一个 tensor 形状为 [6144，4096] 隐藏状$Hissen_state_i$，然后对$Hissen_state_i$做平均池化操作如下列公式所示，获得第 i 个文件的向量表示$mean_embedding_i$，如公式（5.1）所示：

$$mean_embedding_i = \frac{\sum \text{Hidden_state_i}}{6144} \tag{5.1}$$

由于一次提交包含四个文件，一次提交的最终向量表示通过对 4 个向量再平均，获得最终的 commit code 向量表示$F_codellama$。

$$F_codellama = \frac{\sum \text{mean_embedding_i}}{4} \tag{5.2}$$

在获取向量表示后，需要根据向量对相似信息做出检索，检索方式的选择以及检索回的信息也是模型优化中需要考虑的一环，在设计聚合策略时会有动态和静态的区

分，寻找一种合适的聚合策略对于提升缺陷检测能力，合理利用检索信息至关重要。为了验证检索方式的有效性，本研究选择基于 L2 distance（欧几里得距离）的召回进行实验研究。

其中基于 L2 distance 的实现通过 python 中的 FAISS 库完成，L2 距离（欧几里得距离）d 的计算公式可以用以下方式表示，其中$x_i$和$y_i$分别是两个向量的第 i 维分量，而 n 为向量的维度，如公式（5.3）所示。

$$d = \sqrt{\sum_1^n (x_i - y_i)^2} \tag{5.3}$$

### 5.1.2 两种基于检索增强的模型架构

在之前的研究中，CodeGPT 和 GPT2 在少样本场景中的 JIT 模型，分别在 qt 和 openstack 数据中获得了最优的表示，因而这里选择 CodeGPT 模型作为骨干模型进行无检索模型的构建.

基于信息堆叠的检索增强模型（Retrieval-based Information Stack，为了便于描述，后文缩写为 RIS）架构如图 23 所示，此时检索召回的是和当前输入 commit code 相近的 n 个提交代码，即模型在构造输入时利用两部分不同的代码信息，本次提交以及检索召回的代码。区别于无检索模型架构，这种基于信息堆叠的检索增强仅在构造输入时多了检索召回的代码，并不涉及内部模型结构的变化。

区别于无检索模型，基于信息堆叠的检索增强模型架构仅在处理 commit code 时发生了变化。在处理 commit code 的过程中，本研究首先从本次提交中选择最近的四个修改文件。提交代码表示为 $C = [P_1, P_2, \ldots, P_{|C|}]$，其中$|C|$代表提交中的修改文件数量，$|C| = 4$。如果提交中的修改文件数量少于 4 个，则使用“empty patch”填充剩余的位置。除了本次提交的代码本身，检索召回最相近的一次提交代码也会被合并入 commit code 的输入过程，此时新的提交代码表示为 $C = [P_1, P_2, \ldots, P_{|2C|}]$，然后将每个补丁$P_i$作为提交代码中的一个序列，作为代码输入骨干预训练模型，以获得嵌入表示$e_i$，如下列公式：

$$e_i = \mathrm{BackboneModel}(P_i) \tag{5.4}$$

当提交代码中的每个补丁由预训练模型处理后，本研究使用$\oplus$操作得 commit code 的向量表示。

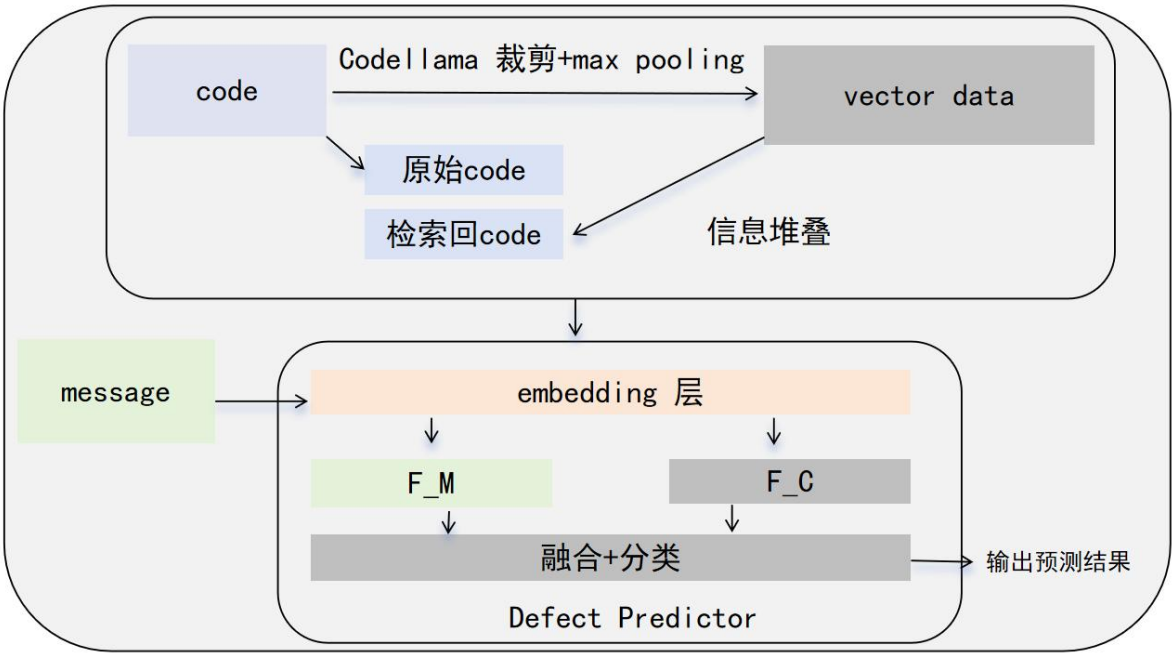


**图 23 基于信息堆叠的检索增强模型架构**

基于跨模型向量融合方式的检索增强模型（Retrieval-based Vector Fusion，为了便于描述，后文缩写为 RVF）架构如图 24 所示，区别于信息堆叠的方式，该架构直接将经由向量编码器模型编码的向量嵌入作为缺陷预测模型的内部特征$F_codellama$。其中本次提交的 commit code 处理遵循原有模型处理方式，即从每个提交中选择最近的四个修改文件。提交代码表示为 $C = [P_1, P_2, \ldots, P_{|C|}]$，其中$|C|$代表提交中的修改文件数量，$|C| = 4$。如果提交中的修改文件数量少于 4 个，则使用“empty patch”填充剩余的位置。然后将每个补丁$P_i$作为提交代码中的一个序列，作为代码输入骨干预训练模型，以获得骨干模型的嵌入表示$e_i$，如下列公式。

$$e_i = \mathrm{BackboneModel}(P_i) \tag{5.5}$$

当提交代码中的每个补丁由预训练模型处理后，本研究使用⊕操作得 commit code 的向量表示，遵循公式（5.6）。

$$C_e = e_1 \oplus e_1 \oplus \ldots \oplus e_{|C|} \tag{5.6}$$

对于跨模态融合向量，采用⊕运算将其进行堆叠，其中 n 为召回的向量特征个数。

$$F_codellama_{all} = F_codellama_1 \oplus F_codellama_2 \oplus \ldots \oplus F_codellama_n \tag{5.7}$$

对于获得的检索向量$F_codellama_{all}$，模型采用一层全连接进行动态加权操作，并将其映射为和骨干模型嵌入特征相同的维度，遵循如下公式处理：

$$weighted_t = W_{fc} \cdot F_codellama_{all} + b_{fc} \tag{5.8}$$

此刻 commit code 的特征包含了骨干模型嵌入及加权融合后的两部分，在通过如第三章所示的 CNN 进行特征提取后获取最终 commit code 的表示$Z_c$。

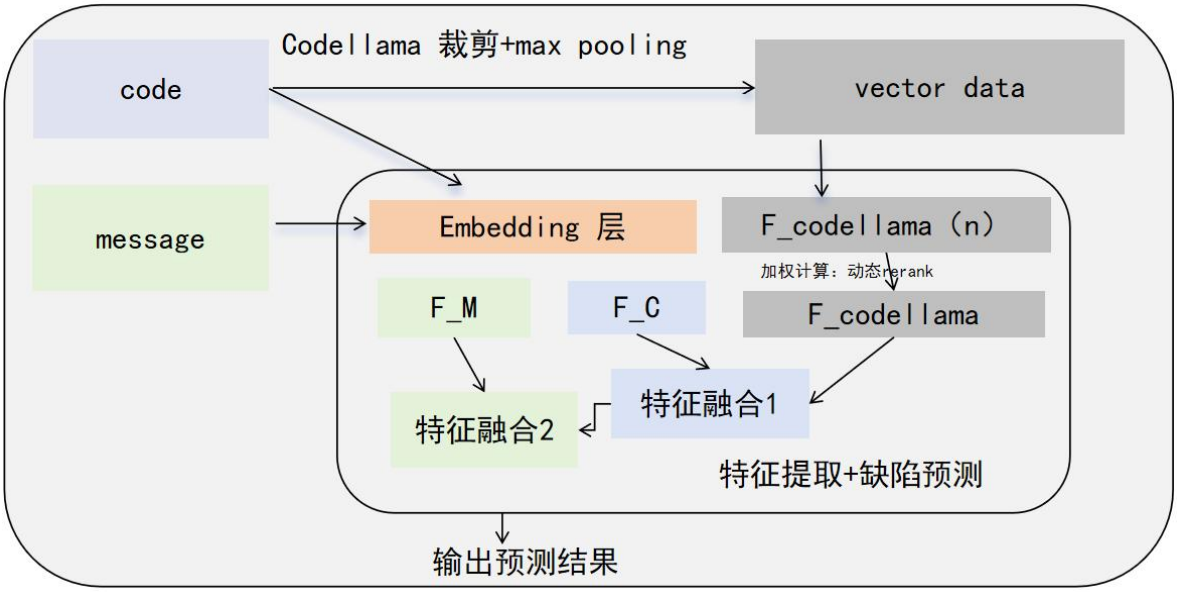


**图 24　基于向量融合的检索增强模型架构**

对于 commit message 的处理，本研究仅做简单的加权融合操作，如第三章对 commit message 相同操作。在获得 commit code 的特征向量$Z_c$和 commit message 的特征向量$Z_m$后，本研究将$Z_c$和$Z_m$融合以获得一次提交的完整特征向量，

将 Z 输入一层全连接网络作为分类器，通过 sigmoid 函数将高维向量映射到一维向量，完成二分类的预测任务。

基于跨模型向量融合的检索增加主要是通过对检索回的几个向量$F_codellama_{all}$重新分配权重后并和原本输入的代码经过小模型表征后的向量特征进行融合来增加代码特征，再将融合后的代码特征和 message 特征进行二次融合，完成即时缺陷预测模型的构建。区别于基于信息堆叠的检索增强模型，基于向量融合的检索增强通过直接引入向量表示带来了外部模型对代码的表征，从而增强模型训练时的信息流动。

## 5.2 实验的目的与内容

本章实验的主要目的为针对缺陷预测任务中存在的可用信息不足的问题，利用检索增强的方式构建缺陷预测的分类模型，并针对现有直接进行文本拼接的检索增强形

式做出改进，初步选择 CodeGPT 作为分类器，裁剪后的 Codellama-7B 作为向量编码器，并在近 10 万个扩展缺陷预测数据集上评估这模型的性能。并将所提方式在更大参数量 GPT2-medium 以及不同架构的模型 Llama3.2-1B 上进行验证实验，以证明所提方法的有效性和泛化性。评价指标为 auc，f1 score 以及马修斯相关系数 MCC。

研究内容主要分为以下不同的方面：

（1）基于向量相似召回的向量表示获取方式

首先，应用检索增强技术的前提需要获得代码及文本的合理的向量表示，目前广泛的方式是利用一个编码器模型直接获取输入的向量表示，但常用的 BERT 架构模型在获取向量表示时受到长度限制，可能导致获取的向量表示存在信息截断和损失，在获取向量表示后，需要根据向量对相似信息做出检索，检索方式的选择影响了模型的速度以及表现，本文通过选择基于 L2 distance 进行召回。

（2）两种检索增强即时缺陷预测方法的提出和有效性探究

在获取向量表示后，需要根据向量对相似信息做出检索，本文通过选择基于 L2 distance 进行召回。并基于上一研究构造的向量表示提出了基于信息堆叠的检索增强模型架构（RIS）和基于跨模型向量融合的检索增强模型架构（RVF），结合第二章中提出的模型架构，基于 CodeGPT 模型在接近 10 万条公开数据集中进行实验验证，证明检索增强应用于即时缺陷预测任务的有效性。

（3）基于向量融合的检索增强在不同参数量及不同架构下的泛化性探究

信息融合是模型优化中需要考虑的一环，涉及了模型训练的效率以及性能表现，基于信息堆叠检索增强的模型架构受限于模型处理上下文的长度以及显存随检索个数的线性增长，而基于跨模型向量融合的检索增强架构的有效性和泛化性，在同架构更大参数量 GPT2-medium 和不同架构的 Llama3.2-1B 模型上进行实验验证，以证明该架构的有效性和泛化性。

## 5.3 实验设置与数据

本研究内容所用数据统计信息如表 21 所示，其中 qt 和 openstack 项目与研究预训练模型结构对缺陷预测任务的影响中相同，jdt，platform，gerrit，go 项目与预训练微调策略对缺陷预测任务的影响探究中相同。共包含 91751 个标注提交。其中，20636

个是与缺陷相关的补丁，其余的是非缺陷提交。

**表 21　实验所用数据集统计信息**

| 项目 | 提交数目 | 缺陷数目 | 缺陷占比（%） |
|---|---|---|---|
| qt | 25704 | 1825 | 7.1 |
| openstack | 13304 | 1627 | 12.23 |
| jdt | 7773 | 3202 | 41.19 |
| platform | 11034 | 4056 | 36.76 |
| gerrit | 14927 | 1781 | 11.93 |
| go | 19009 | 8145 | 42.85 |
| **总数** | 91751 | 20636 | 22.49 |

本研究将每个项目的数据按 80:10:10 的比例进行划分，80%的数据用于训练集，剩余 20%的数据分别用于验证和测试。本研究考虑 auc score，f1 score 以及 MCC 作为评估指标。 MCC 值范围从 -1 到 +1，+1 表示完美分类，0 表示分类效果和随机猜测相同，-1 表示完全错误的分类。

**表 22　实验环境**

| GPU | Memory | 操作系统 |
|---|---|---|
| Tesla V100-SXM2-32GB | 32GB | Ubuntu 20.04 |

本文所构建的实验环境如表 23 所示，本研究使用一台台式电脑作为实验环境。电脑运行 Ubuntu 18.04，配备了 32GB 内存的 V100 显卡，实验学习率设置为 1e-5，batch size 大小设置为 16，每次提交的 commit code 包含四个修改文件。

## 5.4 实验结果与分析

### 5.4.1　基于信息堆叠的检索增强模型（RIS）具体表现

由于在输入时直接拼接了另一组召回的 commit code，模型在训练时需要更多的显存，考虑到单卡 V100 训练 OOM 的问题，这种基于信息堆叠的检索增强模型（RIS）仅能支持 n=1 的实验设置。

CodeGPTJIT 在 6 个项目中的原始无检索的实验结果如表 23 所示。

**表 23 原始无检索模型在六个项目中表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.81 | 0.33 | 0.29 |
| openstack | 0.79 | 0.38 | 0.30 |
| jdt | 0.69 | 0.67 | 0.27 |
| platfom | 0.79 | 0.63 | 0.41 |
| gerrit | 0.68 | 0.22 | 0.17 |
| go | 0.74 | 0.65 | 0.35 |
| **平均** | 0.750 | 0.48 | 0.298 |

基于信息堆叠的检索增强模型实验结果如表 24 所示，根据实验结果可以发现，无检索模型的平均 auc 为 0.750 （表 23），而检索增强模型的平均 auc 为 0.768。检索增强模型的 auc 稍有提升，说明在大多数项目中，通过引入检索代码，模型在区分正负样本的能力上有一定程度的提高。对于个别项目（如 openstack 和 gerrit），auc 有明显提升，但在其他项目（如 jdt 和 go）中 auc score 变化不大。

无检索模型的平均 f1 score 为 0.48 （表 23），而检索增强模型的平均 f1 score 为 0.493。尽管提升幅度较小，但总体来说，检索增强模型在平衡精确率和召回率方面表现更好。在项目 openstack 和 platform 中，f1 score 有显著提高，这表明在这些项目中，检索增强有效提升了模型的精度和召回率的平衡。

无检索模型的平均 MCC 为 0.298（表 23），而检索增强模型的平均 MCC 为 0.317。检索增强模型的 MCC 有所提升，表明模型的整体预测能力有所增强。在项目 openstack 和 platform 中，MCC 有显著提升，这与 f1 Score 的提升一致，表明模型在这些项目中整体表现更佳。

除此之外，本研究设计了一组扩大一次提交中 commit code 修改文件个数=8 的消融实验，来探究 JIT 缺陷预测性能的提升是由于输入代码的数量增多，还是检索回了有效的信息，消融实验的结果如表 25 所示，无检索增强模型（扩大提交文件数量）的平均 auc 为 0.760，相比于原始无检索模型（auc 为 0.750）和检索增强模型（auc 为 0.758），auc 有所提高，但提升幅度不如检索增强模型明显。这表明，单纯通过增加提交代码文件的数量也能提高模型的区分能力，但效果不如引入合理的检索信息更显著。

无检索增强模型（扩大提交文件数量）的平均 f1 Score 为 0.478，与原始无检索模

型的 0.48 基本持平，而低于检索增强模型的 0.489。说明仅仅增加提交代码文件数量在提升精确率和召回率的平衡方面效果有限。

**表 24　引入召回代码输入的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.819 | 0.346 | 0.296 |
| openstack | 0.798 | 0.385 | 0.312 |
| jdt | 0.68 | 0.678 | 0.253 |
| platform | 0.798 | 0.634 | 0.419 |
| gerrit | 0.707 | 0.236 | 0.199 |
| go | 0.747 | 0.657 | 0.356 |
| **平均** | 0.758 | 0.489 | 0.306 |

无检索增强模型（扩大提交文件数量）的平均 MCC 为 0.297，与原始无检索模型的 0.298 几乎相同，而低于检索增强模型的 0.306，这进一步表明，单纯增加提交文件数量对模型整体预测能力的提升效果有限。

**表 25　扩大 commit code 文件数量的无检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.81 | 0.33 | 0.28 |
| openstack | 0.80 | 0.37 | 0.29 |
| jdt | 0.69 | 0.68 | 0.24 |
| platform | 0.82 | 0.64 | 0.43 |
| gerrit | 0.69 | 0.24 | 0.19 |
| go | 0.75 | 0.61 | 0.35 |
| **平均** | 0.760 | 0.478 | 0.297 |

消融实验表明，尽管扩大提交代码文件的数量可以在一定程度上提升模型性能，但这种提升并不如引入合理检索信息的效果显著。这说明 JIT 缺陷预测性能的提升主要得益于检索回合理的信息，而不仅仅是增加输入代码的数量。

相比单纯增加输入代码文件的数量，合理的信息检索可以更显著地提升模型性能，尤其是在 auc score 和 MCC 上的表现。引入检索增强后，模型在多个性能指标上均有提升，说明检索回的信息对于模型性能提升具有关键作用。通过消融实验，验证了性

能提升主要来自检索回合理的信息，而不是单纯增加输入代码文件的数量。

### 5.4.2 基于跨模型向量融合的检索增强模型（RVF）具体表现

由于该架构直接引入了向量的表征（单个 4096 维度的 tensor）而非文本，因而在同样 batch size 大小的设置下，该架构可以在保持与无检索模型等价显存的资源下引入 n=64 的检索向量个数，在几乎无损训练效率的前提下实现相似信息的引入。

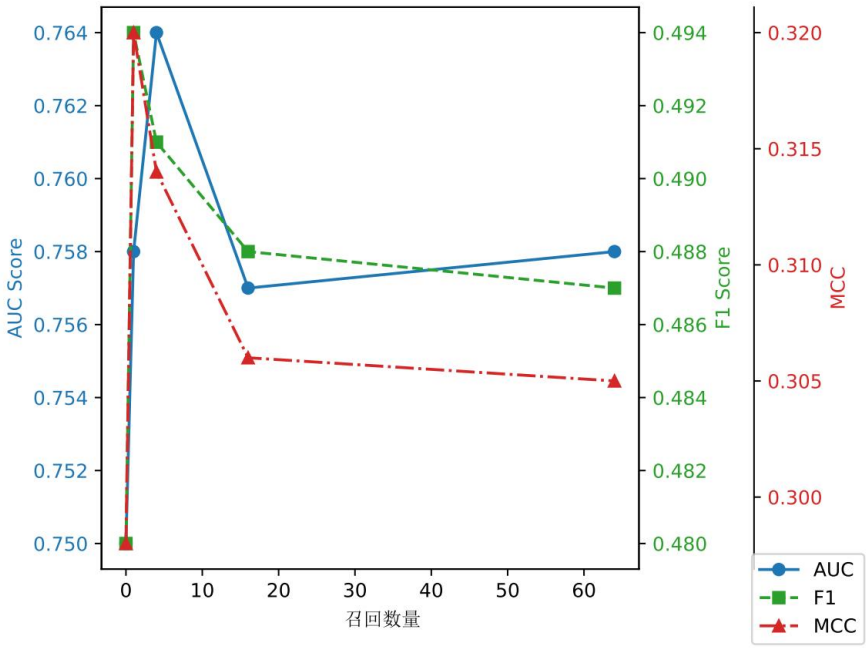


**图 25 CodeGPT-base 模型平均性能随召回向量个数变化**

本检索架构设置了 n=[1，4，16，64]进行实验探究，其中每个检索数量的实验结果分别如表 26，表 27，表 28 和表 29 所示，为了方便比较，对比表 23，即无检索模型来说，图 25 可视化了基于跨模型向量融合的即时缺陷预测模型平均性能随检索向量个数的变化，其中我们可以发现，auc 得分（蓝色）在不同检索个数下表现出轻微的变化，但总体上保持相对稳定。

具体观察如下，随着检索个数的增加，auc 得分没有明显的提升或下降，这表明增加检索个数对模型在 auc 指标上的性能影响较小。f1 得分（绿色）显示出随着检索个数增加的轻微下降趋势。在检索个数较小时，f1 得分相对较高，随着检索个数增加，f1 得分逐渐减少，这可能是由于更多的检索数据带来了更多噪声，影响了模型的精确度和召回率的平衡。

**表 26　引入召回向量数量为 1 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.810 | 0.362 | 0.321 |
| openstack | 0.806 | 0.412 | 0.34 |
| jdt | 0.683 | 0.669 | 0.283 |
| platfom | 0.803 | 0.624 | 0.4 |
| gerrit | 0.703 | 0.255 | 0.228 |
| go | 0.742 | 0.639 | 0.351 |
| 平均 | 0.758 | 0.494 | 0.320 |

**表 27　引入召回向量数量为 4 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.814 | 0.345 | 0.298 |
| openstack | 0.794 | 0.391 | 0.322 |
| jdt | 0.687 | 0.677 | 0.244 |
| platfom | 0.828 | 0.655 | 0.455 |
| gerrit | 0.713 | 0.226 | 0.188 |
| go | 0.749 | 0.654 | 0.379 |
| 平均 | 0.764 | 0.491 | 0.314 |

MCC 得分（红色）也显示出随着检索个数增加的轻微下降趋势。与 f1 得分类似，MCC 得分在检索个数较小时相对较高，随着检索个数的增加，MCC 得分逐渐减小，

**表 28　引入召回向量数量为 16 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.812 | 0.354 | 0.302 |
| openstack | 0.797 | 0.384 | 0.307 |
| jdt | 0.678 | 0.663 | 0.223 |
| platfom | 0.810 | 0.642 | 0.444 |
| gerrit | 0.706 | 0.246 | 0.209 |
| go | 0.740 | 0.640 | 0.350 |
| 平均 | 0.757 | 0.488 | 0.306 |

这表明检索更多数据对模型的预测结果相关性并没有显著的改善，反而可能引入

了更多的误差。这些趋势表明，增加检索个数虽然提供了更多的数据供模型使用，但并不一定带来显著的性能提升。

事实上，更多的检索数据可能引入了更多的噪声和误差，导致 f1 和 MCC 评分略有下降。因此，在选择检索个数时需要权衡数据量和数据质量，以找到最佳的平衡点。

**表 29　引入召回向量数量为 64 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.810 | 0.341 | 0.302 |
| openstack | 0.801 | 0.399 | 0.321 |
| jdt | 0.680 | 0.667 | 0.234 |
| platfom | 0.804 | 0.635 | 0.42 |
| gerrit | 0.723 | 0.233 | 0.209 |
| go | 0.732 | 0.645 | 0.342 |
| **平均** | 0.758 | 0.487 | 0.305 |

但总体上，在这几组检索增强的实验中，无论检索向量的数量设置为几，可以发现均比原始无检索模型更优。这表明，适度的检索增强确实能够提升模型的性能，但需要合理设置检索向量的数量以避免过多的噪声和误差的引入。在少量检索的情况下，模型的 auc、f1 和 MCC 得分相对较高，说明此时检索回的相关信息对模型的判断有积极的影响。

检索数量较少时（n=1，n=4），噪声较少，信息质量较高，模型能够更好地利用这些信息来进行预测。随着检索数量增加到中等水平（n=16），auc 得分依然保持稳定，但 f1 和 MCC 得分略有下降，此时引入的额外信息量较大，可能包含一些与当前任务不太相关的数据，导致模型性能轻微下降。在大量检索（n=64）的情况下，auc 得分依然较稳定，但 f1 和 MCC 得分进一步下降，过多的检索数据可能引入了显著的噪声，影响了模型的精确度和相关性，导致性能不如少量检索时优异。

从实验结果看，适度的检索增强（如 n=1 或 n=4）能够显著提升模型性能，且噪声较少，是较为理想的设置。增加检索数量虽然可以提供更多信息，但需要权衡数据量与数据质量，避免因过多噪声而导致性能下降。总之，适当的检索增强是提高模型性能的有效方法，但需要谨慎选择检索数量，以平衡信息量与噪声之间的关系，从而达到最佳效果。

### 5.4.3 基于跨模型向量融合的检索增强模型泛化性探究

本章在前两小节已对两种不同模式的检索增强方式做出有效性的实验验证，然而基于信息堆叠的检索增强方式具备天然缺陷：（1）检索数量的限制，（2）显存随检索召回的代码数量线性增长，由于在输入时直接拼接了另一组召回的 commit code，模型在训练时需要更多的显存，考虑到单卡 V100 训练 OOM 的问题，这种基于信息堆叠的检索增强模型（RIS）仅能在 CodeGPT 模型上支持检索召回数量 n=1 的实验。而基于跨模型向量融合方式的检索增强模型通过将召回的信息从文本转移到向量，解决了上述提到的两个问题，并且保持检索增强的性能提升无损，为了进一步验证基于跨模型向量融合方式的检索增强方法泛化性，本小节在另外两组模型上进行了更广泛的实验验证，即与 CodeGPT-base 同架构，但参数量为 355M 的 GPT2-medium，以及当前另一 decoder 架构参数量为 1.23B 的 Llama-3.2-1B 模型。

**表 30 GPT2-medium 训练超参设置**

| 项目 | 轮次 | 学习率 | 优化器 | 批次大小 |
|---|---|---|---|---|
| qt | 1 | 1e-5 | Adam | 8 |
| openstack | 1 | 1e-5 | Adam | 8 |
| jdt | 1 | 1e-5 | Adam | 8 |
| platform | 1 | 1e-5 | Adam | 8 |
| gerrit | 1 | 1e-5 | Adam | 8 |
| go | 1 | 1e-5 | Adam | 8 |

对于 GPT2-medium 模型来说，训练超参数如表 30 所示，和上一章节训练设置相比，基于该模型的实验同样设置了 n=[1，4，16，64]的向量检索个数进行实验探究。其中，无检索模型实验结果如表 31 所示，其中 auc score 平均分数为 0.75，f1 score 平均分数为 0.474，而 MCC 指标为 0.302。 每个检索数量的实验结果分别如表 31、表 32、表 33、表 34 和表 35 所示。为了方便比较，图 26 展示了 GPT2-medium 模型在如表训练设置下的平均结果可视化，其中我们可以发现，auc 得分（蓝色），f1 score（绿色）以及 MCC 评估指标（红色）在不同检索个数下表现出轻微的变化，在检索个数为 4 时具有最优的结果。

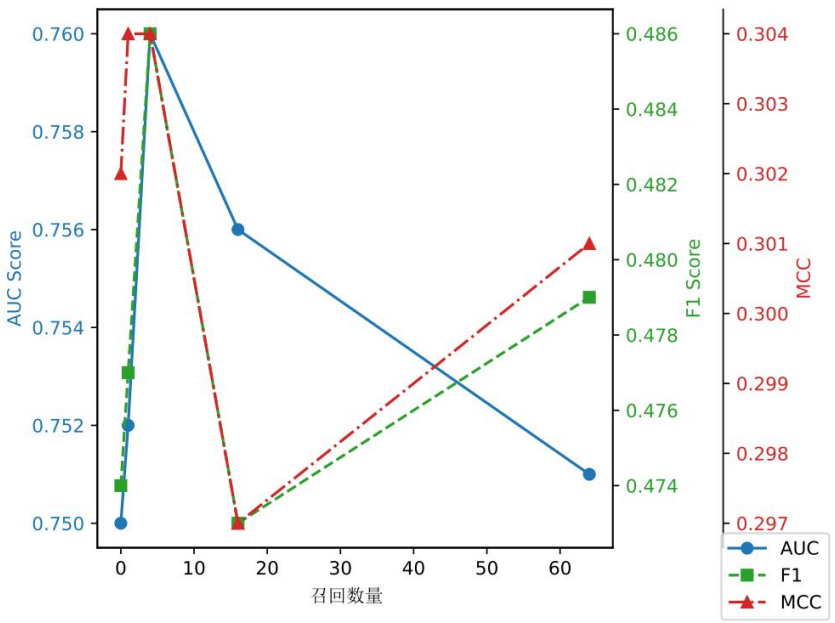


**图 26 GPT2-medium 模型平均性能随召回向量个数变化**

引入少量的召回向量（如 1 个和 4 个向量时，如表 32 和表 33）略微提升了 f1 score 和 MCC 的平均值，分别达到 0.477/0.304 和 0.486/0.304，auc 略微上升至 0.752 和 0.76。

**表 31 GPT2-medium 引入召回向量数量为 0 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.797 | 0.335 | 0.282 |
| openstack | 0.804 | 0.366 | 0.306 |
| jdt | 0.674 | 0.673 | 0.275 |
| platform | 0.809 | 0.646 | 0.441 |
| gerrit | 0.692 | 0.207 | 0.175 |
| go | 0.727 | 0.617 | 0.33 |
| **平均** | 0.75 | 0.474 | 0.302 |

具体来说，16 个召回向量时 f1 和 MCC 平均分下降至 0.473 和 0.297，而 64 个召回向量时两者分别为 0.479 和 0.301，与基准模型相差不大。这个趋势表明，大量召回信息并不总是提升模型性能，可能还会带来信息冗余的问题。这体现了相对于上一小节一致的实验结论。

当召回向量数量增加至 16（表 34）和 64（表 35）时，模型性能出现小幅波动，尤其是 f1 和 MCC 值没有随召回数量增加而持续提升。

对比各个项目的数据可以看到，不同项目对召回数量的反应并不一致。例如，在 qt

项目上，64 个召回向量时的 f1 score 和 MCC 达到了最高值（0.35 和 0.307），但 go 项目在较低召回数量时表现更好。Gerrit 项目在各召回数量上的 f1 和 MCC 表现始终较低，说明在此项目上，召回向量对模型的正向影响较小，可能因为数据特性与其他项目有所不同。

**表 32　GPT2-medium 引入召回向量数量为 1 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.795 | 0.329 | 0.284 |
| openstack | 0.779 | 0.374 | 0.29 |
| jdt | 0.673 | 0.659 | 0.277 |
| platform | 0.803 | 0.644 | 0.435 |
| gerrit | 0.694 | 0.192 | 0.151 |
| go | 0.767 | 0.666 | 0.389 |
| 平均 | 0.752 | 0.477 | 0.304 |

**表 33　GPT2-medium 引入召回向量数量为 4 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.807 | 0.327 | 0.282 |
| openstack | 0.79 | 0.384 | 0.298 |
| jdt | 0.681 | 0.674 | 0.234 |
| platform | 0.8 | 0.652 | 0.45 |
| gerrit | 0.707 | 0.217 | 0.176 |
| go | 0.772 | 0.663 | 0.382 |
| 平均 | 0.76 | 0.486 | 0.304 |

考虑到召回数量带来的显存负担和性能的增益效果有限，推荐使用 1 或 4 个召回向量，以兼顾性能提升和计算资源的合理分配。对于 MCC 分数（红色）指标来说，当检索个数为 1 和 4 时，具有平均性能提升的表现，而当检索个数 n 为 4 以上时，均显示出相较于无检索的较差性能。对于 f1 score（绿色）指标来说，同样的，当检索个数小于等于 4 时，具有平均性能提升的表现，而当检索个数 n 为 4 以上时，性能指标开始波动。

**表 34 GPT2-medium 引入召回向量数量为 16 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.792 | 0.299 | 0.266 |
| openstack | 0.802 | 0.417 | 0.339 |
| jdt | 0.692 | 0.657 | 0.267 |
| platform | 0.787 | 0.613 | 0.377 |
| gerrit | 0.702 | 0.207 | 0.164 |
| go | 0.763 | 0.646 | 0.367 |
| **平均** | 0.756 | 0.473 | 0.297 |

但总体上，在这几组检索增强的实验中，当检索向量的数量设置为 1 或 4 时，可以发现均比原始无检索模型更优。这与上一小节发现，即适度的检索增强能够提升模型的性能的结论是一致的。在少量检索的情况下，模型的 auc、f1 和 MCC 得分相对较高，说明此时检索回的相关信息对模型的判断有积极的影响。

**表 35 GPT2-medium 引入召回向量数量为 64 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.807 | 0.35 | 0.307 |
| openstack | 0.781 | 0.378 | 0.321 |
| jdt | 0.664 | 0.685 | 0.271 |
| platform | 0.796 | 0.638 | 0.425 |
| gerrit | 0.719 | 0.196 | 0.149 |
| go | 0.739 | 0.628 | 0.33 |
| **平均** | 0.751 | 0.479 | 0.301 |

对于 Llama-3.2-1B 模型，考虑到单卡 V100 显存限制，本章在对该模型进行训练时采用了 LoRA 方法，其中模型训练参数设置如表所示，其中 LoRA 设置如表 36 所示。

**表 36 LoRA 训练设置**

| LoRA rank | LoRA Alpha | Target_modules |
|---|---|---|
| 8 | 16 | [ "q_proj", "k_proj", "lm_head","o_proj","v_proj"] |

基于该模型的实验同样设置了 n=[1，4，16，64]的向量检索个数进行实验探究。其中，无检索模型实验结果如表 37 所示，其中 auc score 平均分数为 0.735，f1 score

平均分数为 0.45，而 MCC 指标为 0.275。

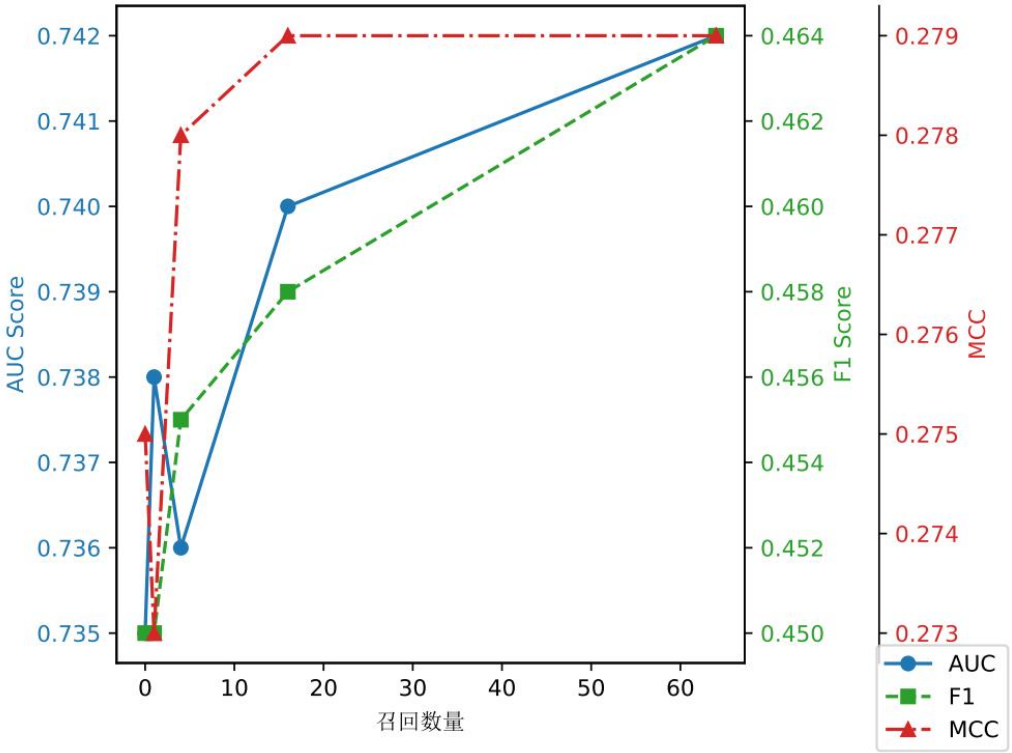


**图 27　Llama3.2-1B 模型平均性能随召回向量个数变化**

图 27 展示了 Llama-3.2-1B 模型在上述训练设置下的平均结果可视化。图中显示，auc score（蓝色）、f1 score（绿色）以及 MCC 评估指标（红色）在不同检索个数下呈现出轻微的变化。具体来看，随着检索个数的增加，三个指标的变化趋势较为平稳，并且在检索个数为 64 时，三个指标均达到了最优的结果。尤其在 auc score 和 f1 score 上，召回数量为 60 时显示出较为显著的提升。

引入 1 个召回向量（见表 38）后，模型的表现出现了一些微小的变化。auc score 得分略微下降至 0.738，f1 score 略有提升至 0.45，而 MCC 则轻微下降至 0.273。这些变化表明，虽然引入召回向量对某些评估指标有所影响，但整体性能变化不大。当召回向量的数量增加至 4 个（见表 39）和 16 个（见表 40）时，auc 和 f1 score 的显示出轻微提升。

总体来看，引入召回向量对于大部分项目的表现影响有限，但在某些特定的任务和数据集上，召回向量的选择和数量仍然能够发挥一定的作用，尤其在检索任务对结果敏感的场景中，适度的召回向量数目仍然可以提升模型的性能。

**表 37 Llama3.2-1B 引入召回向量数量为 0 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.758 | 0.244 | 0.223 |
| openstack | 0.776 | 0.367 | 0.293 |
| jdt | 0.682 | 0.651 | 0.259 |
| platform | 0.79 | 0.646 | 0.444 |
| gerrit | 0.711 | 0.181 | 0.154 |
| go | 0.693 | 0.608 | 0.275 |
| 平均 | 0.735 | 0.45 | 0.275 |

**表 38 Llama3.2-1B 引入召回向量数量为 1 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.771 | 0.254 | 0.226 |
| openstack | 0.78 | 0.366 | 0.298 |
| jdt | 0.687 | 0.618 | 0.27 |
| platform | 0.788 | 0.645 | 0.446 |
| gerrit | 0.704 | 0.197 | 0.157 |
| go | 0.7 | 0.617 | 0.238 |
| 平均 | 0.738 | 0.45 | 0.273 |

实验结果表明，Llama-3.2-1B 模型在引入召回向量时，整体性能变化相较于在小参数量模型上性能提升不大，且大部分情况下，增加召回向量的数量未能带来显著的性能提升，虽然在检索个数为 64 时三个指标均获得了性能最优。基于实验结果，即时在检索向量较少时也能获得性能提升，如检索个数为 4 时。

**表 39 Llama3.2-1B 引入召回向量数量为 4 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.773 | 0.257 | 0.231 |
| openstack | 0.778 | 0.363 | 0.303 |
| jdt | 0.681 | 0.65 | 0.252 |
| platform | 0.78 | 0.648 | 0.447 |
| gerrit | 0.712 | 0.197 | 0.163 |
| go | 0.695 | 0.616 | 0.272 |
| 平均 | 0.736 | 0.455 | 0.278 |

在实际应用中，可能在不同模型做分类时，性能最优随检索个数变化略有差异，但总提升推荐检索个数在 4 左右，避免因过多的向量召回引入不确定性噪声而导致性能产生波动。

**表 40　Llama3.2-1B 引入召回向量数量为 16 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.759 | 0.258 | 0.232 |
| openstack | 0.777 | 0.386 | 0.317 |
| jdt | 0.692 | 0.646 | 0.271 |
| platform | 0.799 | 0.651 | 0.45 |
| gerrit | 0.716 | 0.189 | 0.152 |
| go | 0.696 | 0.617 | 0.254 |
| 平均 | 0.74 | 0.458 | 0.279 |

**表 41　Llama3.2-1B 引入召回向量数量为 64 的检索增强模型表现**

| 项目 | auc score | f1 score | MCC |
|---|---|---|---|
| qt | 0.777 | 0.28 | 0.25 |
| openstack | 0.777 | 0.369 | 0.296 |
| jdt | 0.685 | 0.693 | 0.27 |
| platform | 0.801 | 0.653 | 0.454 |
| gerrit | 0.716 | 0.181 | 0.152 |
| go | 0.695 | 0.608 | 0.254 |
| 平均 | 0.742 | 0.464 | 0.279 |

### 5.4.4　相较于基线模型的总体性能提升

本章将所提方式与所选基线模型 DeepJIT 与 CC2Vec 进行对比，考虑到 auc，f1 以及 MCC 值的三个指标进行分析，这里列出了 RIS+CodeGPT 以及 RVF+CodeGPT 以及基线模型在 6 个项目中的具体结果。

其中表 42，43 和 44 分别展示了 RIS+CodeGPT 在召回相似提交数目为 1，以及 RVF+CodeGPT 在召回向量数目为 4，RVF+gpt2-medium 在召回向量数目为 4，RVF+Llama-3.2 在召回向量数目为 64 的 auc，f1 score 以及 MCC 具体实验结果。根据表中数据，从 auc、f1 score 和 MCC 三个方面分析不同模型结合这两种检索增强方案

相较于基线模型（DeepJIT 和 CC2Vec）的改进效果。

对与 auc 指标（表 42）来说，DeepJIT 和 CC2Vec 在 6 个项目中的平均指标分别为 0.729 和 0.728，其中表现最好的 RIS+CodeGPT(1)的 auc 平均值达到 0.758，相较于 DeepJIT 提升了 3.98%，相较于 CC2Vec 提升了 4.12%。

**表 42 所提优化方案与基线模型相比的 auc score**

| 方法 | qt | openstack | jdt | platform | gerrit | go | 平均 |
|---|---|---|---|---|---|---|---|
| DeepJIT | 0.792 | 0.749 | 0.67 | 0.771 | 0.703 | 0.689 | 0.729 |
| CC2Vec | 0.792 | 0.759 | 0.665 | 0.761 | 0.699 | 0.692 | 0.728 |
| RIS+CodeGPT(1) | **0.819** | **0.798** | 0.680 | 0.798 | 0.707 | **0.774** | 0.758 ↑ |
| RVF+CodeGPT(4) | 0.814 | 0.794 | **0.687** | **0.828** | **0.713** | **0.749** | **0.764** ↑ |
| RVF+gpt2-medium(4) | 0.807 | 0.79 | 0.681 | 0.8 | 0.707 | 0.772 | 0.76 ↑ |
| RVF+Llama-3.2(64) | 0.777 | 0.777 | 0.685 | 0.801 | 0.716 | 0.695 | 0.742 ↑ |

RVF+CodeGPT(4)的 auc 平均值为 0.764，相较于 DeepJIT 提升了 4.80%，相较于 CC2Vec 提升了 4.95%。对于其他两个模型 gpt2-medium 以及 Llama3.2 结合 RVF 架构，对比基线模型均在 auc 指标上获得了全面的提升，且在不同项目上均具有一致性。这表明在不同数据集项目中，检索增强对模型的预测能力有显著促进。

对与 f1 指标（表 43）来说，DeepJIT 和 CC2Vec 的 f1 平均值分别为 0.425 和 0.406，RIS+CodeGPT(1)的 f1 平均值为 0.489，相较于 DeepJIT 提升了约 15.06%，相较于 CC2Vec 提升了 20.44%。RVF+CodeGPT(4)的 f1 平均值为 0.491，相较于 DeepJIT 提升了 15.53%，相较于 CC2Vec 提升了 20.94%。对于其他两个模型 gpt2-medium 以及 Llama3.2 结合 RVF 架构，对比基线模型均在 f1 指标上获得了全面的提升，且在不同项目上均具有一致性。总体来说，在 f1 指标上 RIS 和 RVF 对基线模型的提升较为显著，尤其在代码检索效果更差的项目中，表现出更强的泛化能力和鲁棒性。

**表 43 所提优化方案与基线模型相比的 f1 score**

| 方法 | qt | openstack | jdt | platform | gerrit | go | 平均 |
|---|---|---|---|---|---|---|---|
| DeepJIT | 0.272 | 0.278 | 0.638 | 0.595 | 0.197 | 0.571 | 0.425 |
| CC2Vec | 0.211 | 0.297 | 0.631 | 0.563 | 0.167 | 0.564 | 0.406 |
| RIS+CodeGPT(1) | **0.346** | 0.385 | **0.677** | 0.634 | **0.236** | **0.657** | 0.489 ↑ |
| RVF+CodeGPT(4) | 0.345 | **0.391** | **0.677** | **0.655** | 0.226 | 0.654 | **0.491** ↑ |
| RVF+gpt2-medium(4) | 0.327 | 0.384 | 0.674 | 0.652 | 0.217 | 0.663 | 0.486 ↑ |
| RVF+Llama-3.2(64) | 0.28 | 0.369 | 0.693 | 0.653 | 0.181 | 0.608 | 0.464 ↑ |

对与 MCC 指标（表 44）来说，DeepJIT 和 CC2Vec 的 MCC 平均值分别为 0.243 和 0.230，RIS+CodeGPT(1)的 MCC 平均值为 0.306，对比 DeepJIT 提升了约 25.93%，对比 CC2Vec 提升了 33.04%。RVF+CodeGPT(4)的 MCC 平均值为 0.314，对比 DeepJIT 提升了约 29.22%，对比 CC2Vec 提升了 36.52%。对于其他两个模型 gpt2-medium 以及 Llama3.2 结合 RVF 架构，对比基线模型均在 MCC 指标上获得了全面的提升，且在不同项目上均具有一致性。在 MCC 指标上，结合 RIS 和 RVF 结构对性能提升幅度尤为显著，尤其在衡量模型对不同项目适应性和模型的稳健性方面提供了更高的数值支撑，表明两种检索增强方法有较高的正确率。

综合 auc、f1 和 MCC 三项指标的表现，RIS+CodeGPT(1)和 RVF+CodeGPT(4)均相较于基线模型表现出显著提升，尤其在 f1 和 MCC 指标上有较大优势。这表明在即时缺陷预测任务场景中，基于检索增强的模型能够有效提升模型的预测性能和模型的泛化能力。

**表 44　所提优化方案与基线模型相比的 MCC**

| 方法 | qt | openstack | jdt | platform | gerrit | go | 平均 |
|---|---|---|---|---|---|---|---|
| DeepJIT | 0.245 | 0.226 | 0.23 | 0.366 | 0.153 | 0.24 | 0.243 |
| CC2Vec | 0.201 | 0.242 | 0.243 | 0.317 | 0.124 | 0.251 | 0.230 |
| RIS+CodeGPT(1) | 0.296 | 0.312 | **0.253** | 0.419 | **0.199** | 0.356 | 0.306 ↑ |
| RVF+CodeGPT(4) | **0.298** | **0.322** | 0.244 | **0.455** | 0.188 | **0.379** | **0.314** ↑ |
| RVF+gpt2-medium(4) | 0.282 | 0.298 | 0.234 | 0.45 | 0.176 | 0.382 | 0.304 ↑ |
| RVF+Llama-3.2(64) | 0.25 | 0.296 | 0.27 | 0.454 | 0.152 | 0.254 | 0.279 ↑ |

## 5.5 本章小节

本章首先介绍了本次实验目的和研究内容。随后给出了两种基于检索增强的即时缺陷预测架构的模型架构具体描述。实验成果“一种基于预训练模型的检索增强即时缺陷预测方法”已申请为发明专利并受理。本研究主要集中在缺陷预测任务中利用检索增强的方式构建分类模型，并考察了不同的性能评估指标如 auc score、f1 score 和 MCC。本研究首先采用了基于预训练模型的无检索模型（CodeGPTJIT）作为基准模型，并使用了 Codellama-7B 的裁剪版本 MinusCodellama-1.5B 来获取代码向量表示。在 CodeGPT 模型中进行两种不同检索增强架构的实验，以及检索有效性的消融实验。并将实验结果迁移到 GPT2-medium 以及 Llama3.2-1B 上进行基于跨模型向量融合的检索增强模型泛化性及有效性验证。

本研究设计了基于信息堆叠（RIS）和基于跨模型向量融合（RVF）两种检索增强模型。基于信息堆叠的模型在输入阶段加入了检索回来的代码片段，而基于向量融合的模型则直接引入检索到的向量特征进行模型训练。实验结果显示，引入检索增强后，模型在大多数项目中的性能指标有所提升，尤其在 auc score 和 MCC 上表现更为明显。除此之外，本研究还在 CodeGPT 模型中进行了扩大提交文件数量的消融实验，以及不同数量检索向量的实验， 结果表明，相比于单纯增加提交文件数量，合理引入检索信息能更显著地提升模型性能，尤其是在处理数据质量和噪声方面的平衡。基于跨模型向量融合的检索增强框架（RVF）有效解决了基于信息堆叠的检索增强架构（RIS）中存在的两个问题：（1）检索数量的限制（2）显存随检索召回的代码数量线性增长。基于跨模型向量融合方式（RVF）的检索增强模型通过将召回的信息从文本转移到向量实现检索增强的性能的提升。为了进一步验证基于跨模型向量融合方式的检索增强方法泛化性，本研究在另外两组模型上进行了更广泛的实验验证，即与 CodeGPT-base 同架构，但参数量为 355M 的 GPT2-medium，以及当前另一 decoder 架构，参数量为 1.23B 的 Llama3.2-1B 模型。

总体实验结果显示，合理的检索增强架构能够有效提升模型的分类准确性和整体性能，特别是在处理缺陷预测这类数据不足的任务中。总体来说，本研究不仅提供了深入的模型分析和评估，还通过实验验证了检索增强在缺陷预测任务中的实际效果。并给出了在基于检索增强缺陷预测的结构中召回数量的分析和建议。

# 总结与展望

即时（Just-in-time）缺陷预测的目标旨在判定一个提交在未来是否会引入缺陷。这种提交级别的预测对于帮助从业者在软件开发过程中将有限的 SQA 资源优先放在风险最大的提交上非常有用。然而当前研究往往局限于基于深度学习方式及预训练模型的简单应用，对于如何搭建有效的模型结构及影响 JIT 缺陷预测任务性能的因素尚未进行深入探究。此外，当前的 JIT 缺陷预测架构无法解决可用缺陷数据不足以及缺陷信息利用不足的问题，显示了即时缺陷预测任务仍旧有优化空间。

针对上述关于即时缺陷预测任务研究存在的问题，本文面向基于预训练模型的即时缺陷预测分析与优化技术，通过提出一个基于预训练模型的 JIT 缺陷预测架构，探究不同模型结构以及模型调优等各方面因素分析，详尽分析影响即时缺陷预测任务性能的因素。实验发现 commit code 在很大程度上决定了以不同预训练模型为骨干的所有模型的缺陷预测能力。针对这一发现及微调策略的有效性证明吗，本研究并提出了两种基于 commit code 的检索增强架构，即信息堆叠和跨模型向量融合的检索增强信息架构，所提模型均能实现 JIT 缺陷预测任务的性能提升，并在更大规模及不同架构模型中就进行了完备的实验验证，从而证明所提方法有效性和可泛化性。

## 论文总结

本文主要工作内容如下：

(1) 本文提出了一个基于预训练模型的 JIT 缺陷预测架构，通过对六种不同结构的预训练模型进行实证研究与分析，深入研究了这些模型在 JIT 缺陷预测任务中的性能及影响因素。

(2) 本文通过消融实验揭示了 commit code 在缺陷预测中的重要性，提出在模型优化中重点关注提交代码处理，以增强预测性能的建议。此外，本研究还发现平衡数据集在 few-shot 场景下可以实现模型性能的提升。

(3) 本文深入研究了两种不同风格 JIT 缺陷预测模型中的调优因素，实验表明第一模型第一层在模型微调中至关重要。冻结第一层参数显著降低性能，强调该层参数在任务适应性中的关键作用，并探讨合适的微调策略为 JIT 缺陷预测任务提供重要参考。

(4) 本文提出了结合 LoRA 的高效微调方案，优化训练资源与性能平衡。通过保留第一层参数可更新性，配合权重衰减策略，实现计算效率与性能提升的平衡，为 JIT 缺陷预测模型的优化提供了有效手段。

(5) 本研究基于第二章和第三章的实证研究，在即时缺陷预测中引入基于 commit code 的检索增强方法，设计并实现两种基于信息堆叠（RIS）与跨模型向量融合（RVF）的检索增强架构，实验结果证明，所提两种检索增强架构均能提升缺陷预测任务表现，证明合理检索增强可有效提高模型性能。

(6) 本文所提基于跨模型向量融合检索增强架构，通过将召回文本转变为召回向量，有效解决了基于信息堆叠架构中受显存限制和信息丢失的问题，实验验证发现跨模型向量融合方法在减少噪声与提升数据质量方面效果显著，较信息堆叠方法更适合缺陷预测任务。

(7) 本文在不同模型（GPT2-medium 和 Llama-3.2-1B）上验证跨模型向量融合的泛化能力，进一步证实了该方法在多种模型上均能提升性能，且 Llama3.2-1B 在基于 LoRA 的训练设置下依旧显示了其性能提升，验证了这一缺陷预测策略的较好的泛化性。

## 未来展望

本文面向基于预训练模型的即时缺陷预测任务，搭建了通用的模型架构并对任务影响因素进行详尽的实证分析，设计并实现两种检索增强架构（信息堆叠与跨模型向量融合）以提升 JIT 缺陷预测任务的性能。然而，当前研究仍在诸多方面有优化空间且有待改进，本文总结对未来的展望如下：

(1) 探索跨模型特征融合的更高效方式：当前研究的跨模型向量融合方法在性能提升上效果显著，未来可以尝试不同的特征聚合策略，以进一步丰富信息整合的深度，提高检索增强架构的性能。

(2) 改进数据增强技术：针对 JIT 缺陷预测中的数据稀缺和质量问题，未来可以研究更有效的数据增强技术和降噪策略，以获取更具代表性和泛化能力的数据集。

(3) 构建领域自适应的动态微调策略：在不同项目或编程语言中，提交代码结构及模式差异较大，未来可开发自适应微调技术，提高模型在实际应用中的灵活性和泛化性。

# 参考文献

[1] 蔡亮, 范元瑞, 鄢萌,等. 即时软件缺陷预测研究进展[J]. 软件学报, 2019, 30(5):20.

[2] Hinton, Geoffrey E. How neural networks learn from experience.[J]. Scientific American, 1992, 267(3):144-51.

[3] Restricted S, Related S . Learning Deep Architectures for AI .

[4] Hinton GE, Osindero S, Teh YW. A fast learning algorithm for deep belief nets. Neural Comput. 2006 Jul;18(7):1527-54. doi: 10.1162/neco.2006.18.7.1527. PMID: 16764513.

[5] Krizhevsky A, Sutskever I, Hinton G. ImageNet Classification with Deep Convolutional Neural Networks[J]. Advances in neural information processing systems, 2012, 25(2).

[6] Ioffe S, Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift[J]. JMLR.org, 2015.

[7] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” nature, vol. 521,no. 7553, p. 436, 2015.

[8] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[J]. IEEE, 2016.

[9] Yang X, Lo D, Xia X, et al. Deep Learning for Just-in-Time Defect Prediction[C] IEEE International Conference on Software Quality. IEEE, 2015.

[10] Pornprasit C, Tantithamthavorn C. JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction[J]. 2021.

[11] Yan M, Xia X, Fan Y, et al. Just-In-Time Defect Identification and Localization: A Two-Phase Framework[J]. IEEE Transactions on Software Engineering, 2020, PP(99):1-1.

[12] Toderici G, O'Malley S M, Hwang S J, et al. Recurrent Neural Network Regularization.

[13] Hoang T, Dam H K, Kamei Y, et al. DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction[C].2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR). ACM, 2019.

[14] Pappas N, Popescu-Belis A. Multilingual Hierarchical Attention Networks for Document Classification[J]. 2017.

[15] Hoang T, Kang H, Lo D, et al. CC2Vec: distributed representations of code changes[J]. 2020.

[16] Zeng Z, Zhang Y, Zhang H, et al. Deep just-in-time defect prediction: how far are we?[C]. ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis. ACM, 2021.34(2):181−196.

[17] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[J]. arXiv, 2017.

[18] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[J]. 2018.

[19] Liu Y, Ott M, Goyal N, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach[J]. 2019.

[20] Feng Z, Guo D, Tang D, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages[C] 2020.

[21] Radford, A., Wu, J., Child, R., et al. Language Models are Unsupervised Multitask Learners[J].

[22] Lewis, M., Liu, Y., Goyal, N., et al. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension[J].

[23] Ahmad, W., Chakraborty, S., Ray, B., et al. Unified Pre-training for Program Understanding and Generation[J].

[24] M. Fu and C. Tantithamthavorn, “Linevul: A transformer-based line-level vulnerability prediction,” in 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), 2022, pp. 608–620.

[25] Wu, B., Liu, S., Feng, R., et al. Enhancing Security Patch Identification by Capturing Structures in Commits[J]. 2015, 14(8).

[26] Sun, J., Xing, Z., Lu, Q., et al. ``Silent vulnerable dependency alert prediction with vulnerability key aspect explanation," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 970--982.

[27]Zhang, Q., Yu, B., Fang, C., et al. ``Gamma: Revisiting template-based automated program repair via mask prediction," in Proceedings of 38th IEEE/ACM International Conference on Automated Software Engineering (ASE2023), 2023.

[28]Mosbach, M., Andriushchenko, M. and Klakow, D. ``On the stability of fine-tuning {BERT:} misconceptions, explanations, and strong baselines," CoRR,vol. abs/2006.04884, 2020.

[29]Li, X., Xiong, H., An, H., et al. ``{RIFLE:} backpropagation in depth for deep transfer learning through re-initializing the fully-connected layer," CoRR, vol. abs/2007.03349, 2020.

[30]Lee, C., Cho, K., and Kang, W. ``Mixout: Effective regularization to finetune large-scale pretrained language models," in {8th International Conference on Learning Representations, {ICLR} 2020, Addis Ababa, Ethiopia, April 26-30, 2020}.

[31]Aghajanyan, A., Gupta, S., Zettlemoyer, L. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning[C/OL]. Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Online: Association for Computational Linguistics, 2021: 7319-7328[2024-10-31].

[32]Hu, E. J., Shen, Y., Wallis, P., et al. LoRA: Low-Rank Adaptation of Large Language Models[A/OL]. arXiv, 2021[2024-10-31].

[33]Mahabadi, R. K., Henderson, J., Ruder, S. Compacter: Efficient Low-Rank Hypercomplex Adapter Layers[A/OL]. arXiv, 2021[2024-10-31].

[34]Guo, Y., Gao, X., Zhang, Z., Chan, W. K., & Jiang, B. (2023). A study on the impact of pre-trained model on Just-In-Time defect prediction. 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), 105－116.

[35]Zhang, T., Wu, F., Katiyar, A., et al. ``Revisiting few-sample bert fine-tuning," in Proceedings of the International Conference on Learning Representations(ICLR 2021), 2021.

[36]Chen D, Fisch A, Weston J,et al.Reading Wikipedia to Answer Open-Domain

Questions.2017[2023-11-23].DOI:10.18653/v1/P17-1171.

[37] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks[J]. 2019.DOI:10.18653/v1/D19-1410.

[38] Li B, Zhou H, He J, et al. On the Sentence Embeddings from Pre-trained Language Models. 2020[2023-11-23]. DOI:10.18653/V1/2020.EMNLP-MAIN. 733

[39] Gao T, Yao X, Chen D. SimCSE: Simple Contrastive Learning of Sentence Embeddings[J]. 2021.DOI:10.48550/arXiv.2104.08821.

[40] Yan Y, Li R, Wang S, et al.ConSERT: A Contrastive Framework for Self-Supervised Sentence Representation Transfer[J]. 2021.DOI:10.18653/v1/2021.acl-long.393.

[41] Zhou Y, Liu S, Siow J, et al. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks[J]. 2019.DOI:10.48550/arXiv.1909.03496.

[42] Guo D, Ren S, Lu S,et al. GraphCodeBERT: Pre-training Code Representations with Data Flow[C]. International Conference on Learning Representations.2021.

[43] Liu, Y. Chen, X. Xie, J. K. Siow, and Y. Liu, “Retrieval-augmented generation for code summarization via hybrid gnn,” in International Conference on Learning Representations, 2020.A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, “Pre-trained contextual embedding of source code,” 2019.

[44] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu et al., “Graphcodebert: Pre-training code representations with data flow,” arXiv preprint arXiv:2009.08366, 2020.

[45] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, “Intellicode compose: Code generation using transformer,” in Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020, pp. 1433–1443.

[46] L. Buratti, S. Pujar, M. Bornea, S. McCarley, Y. Zheng, G. Rossiello, A. Morari, J. Laredo, V. Thost, Y. Zhuang et al., “Exploring software naturalness through neural language models,” arXiv preprint arXiv:2006.12641, 2020.

[47] R.-M. Karampatsis and C. Sutton, “Scelmo: Source code embeddings from language models,” arXiv preprint arXiv:2004.13214, 2020.

[48] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” arXiv preprint arXiv:2109.00859, 2021

[49] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Unified pre-training for program understanding and generation,” arXiv preprint arXiv:2103.06333, 2021.

[50] A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, “Learning and evaluating contextual embedding of source code,” in International Conference on Machine Learning. PMLR, 2020, pp. 5110–5121.

[51] Liu, Shangqing & Wu, Bozhi & Xie, Xiaofei & Meng, Guozhu & Liu, Yang. (2023). ContraBERT: Enhancing Code Pre-trained Models via Contrastive Learning. 2476-2487. 10.1109/ICSE48619.2023.00207.

[52] Johnson, Jeff, Matthijs Douze, and Hervé Jégou. "Billion-scale similarity search with gpus." IEEE Transactions on Big Data (2019).

[53] Guo, Ruiqi, et al. "Accelerating large-scale inference with anisotropic vector quantization." International Conference on Machine Learning. PMLR, 2020

[54] Khandelwal U, Levy O, Jurafsky D,et al.Generalization through Memorization: Nearest Neighbor Language Models[J]. 2019.

[55] Ouz B, Chen D, Wu L,et al.Dense Passage Retrieval for Open-Domain Question Answering.2020[2023-11-23].DOI:10.18653/V1/2020.EMNLP-MAIN.550.

[56] Guu K, Lee K, Tung Z, et al. REALM: Retrieval-Augmented Language Model Pre-Training[J]. 2020.DOI:10.48550/arXiv.2002.08909.

[57] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, “Memorizing Transformers.” arXiv, Mar. 16, 2022. Accessed: Aug. 21, 2023.

[58] S. Borgeaud et al., “Improving language models by retrieving from trillions of tokens.” arXiv, Feb. 07, 2022. Accessed: Aug. 22, 2023.

[59] C. Yu, G. Yang, X. Chen, K. Liu, and Y. Zhou, “BashExplainer: Retrieval-Augmented

Bash Code Comment Generation based on Fine-tuned CodeBERT." arXiv, Jun. 27, 2022. Accessed: Nov. 20, 2023.

[60]Lu S, Duan N, Han H,et al. ReACC: A Retrieval-Augmented Code Completion Framework[J]. 2022.DOI:10.48550/arXiv.2203.07722.

[61]Kim S, Jr. Whitehead EJ, Zhang Y. Classifying software changes: Clean or buggy? IEEE Trans. on Software Engineering, 2008,

[62]A large-scale empirical study of just-in-time quality assurance[J]. IEEE Transactions on Software Engineering, 2013, 39(6):757-773.

[63]Sliwerski J, Zimmermann T, Zeller A. When do changes induce fixes? In: Proc. of the 2nd Working Conf. on Mining Software Repositories. New York: ACM Press, 2005. 2428.

[64]Kim SH, Zimmermann T, Pan K, Jr. Whitehead EJ. Automatic identification of bug-introducing changes. In: Proc. of the 21st Int' l Conf. on Automated Software Engineering. Washington: IEEE, 2006. 81−90.

[65]Da Costa DA, McIntosh S, Shang W, Kulesza U, Coelho R, Hassan AE. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. IEEE Trans. on Software Engineering, 2017,43(7):641−657.

[66]Neto EC, Da Costa DA, Kulesza U. The impact of refactoring changes on the SZZ algorithm: An empirical study. In: Proc. of the 25th Int' l Conf. on Software Analysis, Evolution and Reengineering. Washington: IEEE, 2018. 380−390

[67]Basili VR, Perricone BT. Software errors and complexity: An empirical investigation. Communications of the ACM, 1984,27(1): 42−52.

[68]Hatton L. Reexamining the fault density component size connection. IEEE Software, 1997,14(2):89−97.

[69]Mockus A, Votta LG. Identifying reasons for software changes using historic databases. In: Proc. of the 16th Int' l Conf. on Software Maintenance. Washington: IEEE, 2000. 120−130.

[70]Schneidewind NF, Hoffmann HM. An experiment in software error data collection and

analysis. IEEE Trans. on Software Engineering, 1979,5(3):276−286.

[71] Shihab E, Hassan AE, Adams B, Jiang ZM. An industrial study on the risk of software changes. In: Proc. of the 20th Int’l Symp. on the Foundations of Software Engineering. New York: ACM Press, 2012. No.62.

[72] Kamei Y, Shihab E, Adams B, Hassan AE, Mockus A, Sinha A, Ubayashi N. A large-scale empirical study of just-in-time quality assurance. IEEE Trans. on Software Engineering, 2013,39(6):757−773.

[73] McIntosh S, Kamei Y. Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction. IEEE Trans. on Software Engineering, 2018,44(5):412−428.

# 攻读硕士期间取得的成果

## 攻读硕士学位期间发表的论文

[1] Guo, Y., Gao, X., Zhang, Z., Chan, W. K., & Jiang, B. (2023). A study on the impact of pre-trained model on Just-In-Time defect prediction. 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), 105 – 116.

[2] Guo, Y., Gao, X., & Jiang, B. (2024). An Empirical Study on JIT Defect Prediction Based on BERT-style Model (arXiv:2403.11158). arXiv. (Technical Report)

[3] Guo, Y., Yin, L., Jiang, B., & Zhang, J. (2024). TODO: Enhancing LLM Alignment with Ternary Preferences (arXiv:2411.02442). arXiv. (ICLR2025 Under Review)

[4] 姜博，郭钰香，高小鹏，王世海, & 刘斌. (2024). 一种基于预训练模型的检索增强即时缺陷预测方法 [专利申请号 2024117714271]. 北京航空航天大学.

# 致谢

一晃已经两年半过去，我在北航的硕士求学之路即将走到终点。回顾过去两年多的学习生活，从一个初出茅庐本科毕业什么都不太懂的科研小白，到在姜老师指导下中稿第一篇论文，再到和别人合作完成第一篇顶会投稿，我的求学之路充满了幸运二字。与此同时，在北航的校园生活里认识了太多太好的同门，同学以及友人和师长。在我毕业论文的结束语里，我想把我诚挚的感谢给每一位指导过，帮助过，合作过以及交流过的人。

首先，我要感谢的是我的本科学校四川大学，包容开放的环境使得我在对周遭认知建立雏形的时候得以形成一个健全健康的人格，在川大的培养下我才能有进入北航学习的机会。其次我要感谢的是母校北京航空航天大学，两年半的学习让我对计算机科学有了更深入的了解和体会。与此同时，北航为我们提供了优秀的资源，不管是科研还是就业，都使我有幸能成长为现在的独立个体。

其次，我想感谢我的导师高小鹏教授，以及指导我学习科研的姜博教授。姜博教授是我在学术道路上的引路人。在研究生学习生活过程中，他给我们以科研指导和写作指导，并对我们每个人极尽包容，每次和同门一起聊天都会觉得读研能遇到这样好的指导老师而感到幸运。

同时我要感谢我在学习过程中遇到的合作者们，和他们一起完成科研工作使得自己受益良多，也正因他们不管是生活还是就业上的帮助我才能有幸走到今天。另外，我也要感谢现视研群友，北航艾欧泽亚之家以及其他的朋友们，他们在我生活中扮演着不可或缺的角色，不管是生活上的苦恼，平时的交流，以及偶尔的约饭打本，都使得我获得了有趣的生活经历。

最后，我要感谢我的母亲，她是我这一生中最值得敬爱的人，从小到大无条件的包容支持和理解让我获得了现在的成长，也学会了包容支持和理解别人，无论何时，我都会感谢我的母亲，这世界上最伟大的女性。